

Introduction to RISC BASED COMPUTER ARCHITECTURE

AArch32 and AArch64

```
main:  
// the "return" pattern  
push {lr} // save lr on the stack  
// implement your application  
mov r0, #0 // set the return value in r0  
pop {pc} // set the pc to the lr  
// the "exit" pattern
```

e3a01000

e3a0200a

e3a03000

e3a04005

e0535004

b0800002

b2833001

baffffffb

ebffffff

e52de004

e3a0400f

e3a0500a

e0856004

e0535004

Contents

1	Introduction	1
1.1	Historical Context	1
1.2	Complex Instruction Set Computers	1
1.3	Reduced Instruction Set Computers	2
1.3.1	The Mobile Revolution	4
1.4	This book	4
1.4.1	Approach	4
1.4.2	Book Resources	5
1.4.3	CPUlator	5
1.4.4	<code>qemu-system</code>	6
1.4.5	<code>qemu-system-aarch64</code> example	6
1.5	Chapter Summary	8
1.6	Chapter Exercises	8
2	Program Representation and Execution	11
2.1	Introduction to the Executable and Linkable Format	11
2.2	ELF Layout	12
2.2.1	Mapping C Program Components to ELF Sections	13
2.2.2	Unmapped C Program Data	14
2.3	Process address space of an ELF program	14
2.3.1	Sections to Segments	15
2.3.2	The Stack	16
2.3.3	Memory Mapping	16
2.3.4	The Heap	16
2.4	Chapter Summary	17
2.5	Exercises	17
3	Assembly Language Programming	19
3.1	Basic Instructions: Adding and Subtracting	19
3.1.1	Discussion	20
3.1.2	Two's Complement	21
3.1.3	Mathematical Instructions	22
3.1.4	A note on <code>{cond}{S}</code>	22
3.1.5	Multiplication using the <code>mul</code> family	23
3.1.6	Logical Operators	24
3.2	Program Flow and the Program Counter	25
3.2.1	Sequential Execution	25
3.2.2	The Program Counter Register (<code>pc</code>)	25

3.2.3	Program Exit	26
3.3	Chapter Summary	28
3.4	Chapter Exercises	28
4	Conditional Execution	29
4.1	Branching	29
4.1.1	<code>if</code> test as subtraction	29
4.2	The Current Program Status Register	30
4.2.1	NZCV flags	30
4.2.2	Carry-out	31
4.2.3	Overflow	31
4.2.4	Compare and test	32
4.2.5	Signed and Unsigned Conditions	33
4.2.6	Updating NZCV	35
4.2.7	Combining condition codes and NZCV updates	35
4.2.8	Branching	35
4.3	Chapter Summary	36
4.4	Exercises Summary	36
5	Branching and Iteration	37
5.1	Introduction to Program Control Flow	37
5.1.1	Control Flow in Assembly	37
5.1.2	The Role of the Program Counter (<code>pc</code>)	37
5.1.3	Branch Instruction Encoding and Range	37
5.1.4	Branching and the Pipeline	38
5.1.5	Why Control Flow Matters	39
5.2	ARM Branch Instruction Set	39
5.2.1	Core Branch Instructions	39
5.2.2	ARM64 Enhancements	40
5.3	Conditional Execution	40
5.3.1	Condition Flags Overview	40
5.3.2	Using Flags in Practice	41
5.4	Loop Constructs and Patterns	42
5.4.1	While Loop in Assembly	42
5.4.2	For Loop Using <code>subs</code>	42
5.4.3	Do-While Loop	43
5.4.4	Nested Loops and Flowcharts	44
	Visualizing Control Flow	44
5.5	Exercises	45
5.5.1	Basic Load and Store Operations	45
5.5.2	Byte vs Word Access	45
5.5.3	Offset Addressing	46
5.5.4	Pointer Arithmetic	46
5.5.5	Pointer Traversal in Assembly	46
5.5.6	Finding Maximum in an Array	46

CONTENTS

6	Addressing Memory	49
6.1	Load and Store	49
6.1.1	What to know when addressing memory	49
6.1.2	Instruction Syntax	49
6.2	C Program	50
6.2.1	C Pointers	50
6.3	ARM Assembly “Pointers”	51
6.3.1	Initializing the base-register	51
6.3.2	Pointing to the i -th element	52
6.4	Integer Addressing Modes	53
6.4.1	Offset Addressing	53
6.4.2	Post-index Addressing	54
	Post-Index Patterns	55
6.4.3	Pre-index Addressing	57
	Stacks	59
6.5	Character Addressing Modes	60
6.5.1	Iterating over strings	60
6.5.2	Zero-extending bytes	61
6.6	Revisiting the Program Counter	62
6.7	pc-Relative Addressing	63
6.7.1	ADR and ADRP	63
6.8	Literal Pools	63
6.8.1	Pool Placement and Range Limits	64
6.8.2	Viewing Literal Pools	65
6.9	Mixed Instructions and Data in ELF	65
6.9.1	ELF Sections vs. Segments	66
6.9.2	ARM-Specific ELF Annotations	66
6.9.3	Implications for the Programmer	66
6.10	Chapter Summary	67
7	Functions and The Stack	69
7.1	Introduction	69
7.2	Sample C Code	69
7.3	ARM Procedure Call Standard (APCS)	70
7.4	***caller prepares parameters*** UPDATE THIS!!	71
7.5	Preserved and Unpreserved Registers	73
7.6	callee reads the parameters	73
7.6.1	Post-index stack addressing	73
7.6.2	Offset stack addressing	74
7.7	callee saves <code>lr</code>	75
7.7.1	Orphaned instructions and <code>return</code>	75
7.7.2	Branch with link <code>bl</code> and the link register <code>lr</code>	75
7.8	callee saves its working set of registers	77
7.9	Stack Overflow and Underflow	79
7.10	Stack Overflow	79
7.10.1	Causes of Stack Overflow	80
7.10.2	Example of Stack Overflow in Recursive Function	80
7.10.3	Effects of Stack Overflow	80

7.10.4	How to Prevent Stack Overflow	81
	Limit Recursion Depth	81
	Optimize Stack Usage	81
	Use Tail Recursion	82
	Use the Stack Wisely	82
	Monitor Stack Usage	83
7.10.5	Conclusion on Stack Overflow	83
7.11	Prologue and Epilogue	83
7.11.1	Prologue: Setting Up the Stack Frame	83
7.11.2	Epilogue: Returning Control to the Caller	84
7.12	Register Allocation and Stack Management	84
7.13	PC-Relative Addressing and Literal Pools	85
7.13.1	PC-Relative Addressing	85
7.13.2	Literal Pools	85
7.14	ARM32 vs. ARM64 Stack Conventions	86
7.15	Exercises on Programming the Stack	86
7.15.1	Exercise 1: Stack Management in Recursive Functions	86
7.15.2	Exercise 2: Understanding Caller-Callee Stack Interaction	86
7.15.3	Exercise 3: Stack Overflow Prevention	87
7.15.4	Exercise 4: Prologue and Epilogue Function Design	87
7.15.5	Exercise 5: Exploring Stack Underflow	88
7.15.6	Exercise 6: Stack Size Limitation and Stack Guard	88
8	Integrating libc	89
8.1	Linux System Calls	89
8.1.1	Linux libc	91
8.1.2	Printing to the screen using <code>printf</code>	91
8.2	Chapter Exercises using <code>libc</code> functions	94
9	Binary Representation of Instructions	97
9.1	Motivating Example	97
9.2	Instruction Classes	98
9.3	Data Processing Instructions	99
9.3.1	Non-Multiply Instruction Format	99
	Src2: The Second Source	100
9.3.2	Worked Examples	101
9.3.3	Multiply instructions	103
9.4	Memory Instructions	103
10	Single-Cycle and Multi-Cycle Microarchitecture	105
10.1	Overview	105
10.2	From ISA to Microarchitecture	105
10.2.1	What the ISA Describes	105
10.2.2	What the Microarchitecture Implements	107
10.2.3	The Instruction Execution Cycle	108
10.3	Part I: Single-Cycle Microarchitecture	110
10.3.1	Core Concept	110
10.3.2	Main Hardware Components	111
	Instruction Encoding and Decoding	111

CONTENTS

Register File Ports in Detail	113
10.3.3 Dataflow of an Instruction (Example)	116
10.3.4 Timing and Performance	118
10.3.5 Advantages and Disadvantages	120
10.4 Part II: Multi-Cycle Microarchitecture	121
10.4.1 Motivation	121
10.4.2 Additional Internal Registers	123
10.4.3 State-by-State Operation	125
10.4.4 Advantages of the Multi-Cycle Approach	128
10.4.5 Performance Comparison	128
10.4.6 Comparison Summary	129
10.5 Putting It All Together	129
10.5.1 Evolution Toward Pipelining	129
10.6 Summary	130
10.7 Exercises	130
11 Pipeline Microarchitecture	131
11.1 Overview and Motivation	131
11.2 The Five Pipeline Stages	132
11.2.1 Tracing One Instruction Through All Five Stages	133
11.3 Pipelined Datapath	135
11.3.1 Pipeline Registers and Signal Naming	135
11.3.2 Pipelining the Write Address (WA3)	135
11.3.3 Register File Write and Read Timing	136
11.3.4 PC Increment and Optimization	136
11.4 Pipelined Control	137
11.4.1 Deriving Control Signals	137
11.4.2 Pipelining the Control Signals	137
11.5 Data Hazards	138
11.5.1 The Read-After-Write Problem	138
11.5.2 Result Forwarding	139
11.6 The Hazard Unit	140
11.6.1 Architecture	140
11.6.2 Match Signals	141
11.6.3 Forwarding Logic	142
11.7 Load-Use Hazard and Pipeline Stalls	143
11.7.1 Why <code>ldr</code> Cannot Be Forwarded	143
11.7.2 Stalls and Bubbles	143
11.7.3 Hazard Unit: LDR Stall Logic	144
11.7.4 The Compiler's Role: Avoiding Hazards in Software	145
11.8 Control Hazards	145
11.8.1 The Branch Problem	145
11.8.2 Predict-Not-Taken	146
11.8.3 Resolving Branches in the Execute Stage	147
11.8.4 Handling PC Writes from Write-Back	147
11.8.5 Complete Hazard Unit Control Logic	147
11.8.6 Reducing the Branch Penalty Further	148
11.8.7 Beyond Predict-Not-Taken: Smarter Branch Predictors	148

11.9 Exceptions in a Pipelined Processor	149
11.9.1 The Problem: Multiple In-Flight Instructions	149
11.9.2 Precise vs. Imprecise Exceptions	150
11.9.3 Implementing Precise Exceptions in a 5-Stage Pipeline	150
11.10 Structural Hazards	151
11.10.1 The Memory Conflict	151
11.10.2 Harvard Architecture Eliminates the Memory Hazard	151
11.10.3 Register File Read/Write in the Same Cycle	152
11.10.4 Summary	152
11.11 Real-World ARM Pipelines	152
11.12 Complete Pipeline Trace: Putting It All Together	153
11.12.1 The Program	153
11.12.2 Cycle-by-Cycle Table	154
11.12.3 Pipeline Snapshot at Cycle 6	154
11.12.4 Annotated Walkthrough	154
11.13 Performance Analysis	155
11.13.1 Ideal CPI and Throughput	155
11.13.2 Pipeline Efficiency	155
11.13.3 Stall Penalties	156
11.13.4 Worked Example: Computing Effective CPI	156
11.13.5 Comparison Summary	157
11.14 Summary	158
11.15 Exercises	159
12 CPU Caches	163
13 Virtual Memory	165
14 Parallel Architectures	167
15 Conclusion	169

Introduction

1.1 Historical Context

The evolution of CPU architectures over the years since the 1970s has taken two distinct pathways. The first pathway, and the one taken by Intel with their x86,¹ was to squeeze more and more [circuitry] onto the silicon fabric of the CPU. Starting from around 29000 transistors in 1978, the Arrow Lake variant (launched in 2024) has over 4 billion transistors today [Com25].

The astonishing growth of transistor density is summarized in Fig. 1.1. As you can see from this log-linear graph, growth in transistor density followed the Moore' Law doubling model quite closely, only beginning to slow then plateau in from 2008 onwards.

In addition to the growth in transistor density, the instruction set of the x86 in 1978 had about 100 instructions, whereas today, this number is closer to 1600.

In addition, clock speeds have increased by a factor of 1200 - from 5 MHz to 6 GHz today. Finally pipeline depth: has gone from 4 to about 30 stages in 2024. Pipeline microarchitectures will be discussed in more detail in Chapter 11.

As you can see, the Intel x86 trend has been to continuously pack more and more complex circuitry onto the CPU silicon.

1.2 Complex Instruction Set Computers

The Intel x86 as mentioned in the previous section is an example of a *CISC* (Complex Instruction Set Computing) architecture. This architecture uses a large set of *complex* instructions, each of which is capable of performing multiple operations in a single instruction. In other words, a CISC instruction is often the *encapsulation* of a set of lower-level operations.

CISC instructions typically offer the programmer a higher level of *abstraction* than, as we shall see, the RISC (Reduced Instruction Set Architecture) discussed next. The Intel x86 Instruction Set Architecture (ISA) supports *variable length* instructions. This means that instructions can occupy less than the full word size of the CPU. Variable length instructions are a space saving solution - reducing the overall footprint of the executable and thereby reducing the amount of memory required to store the process code. X86 instruction lengths can vary in the range 1 to 15 bytes. This means that the number of bytes used

¹first introduced in 1978

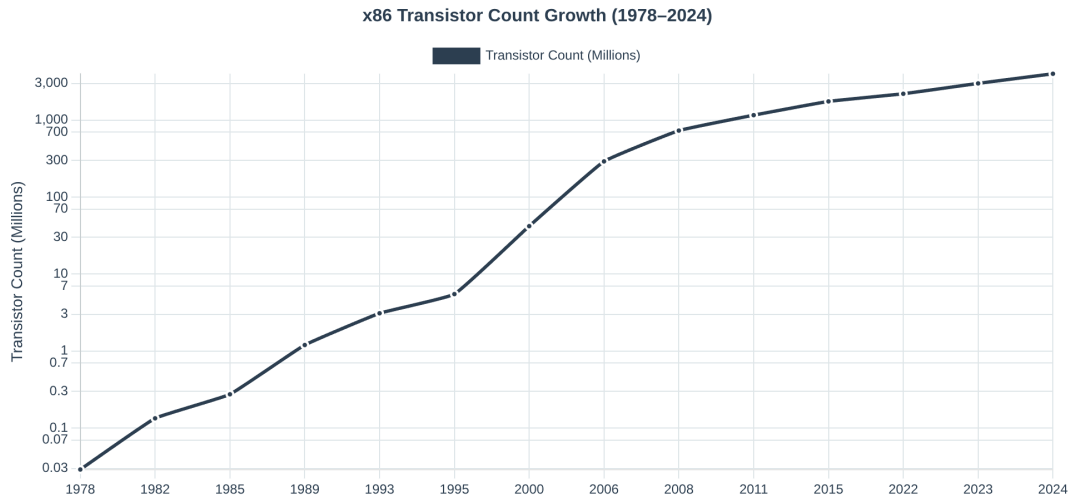


Figure 1.1: The growth of the number of transistors in the x86 architecture from 1978 to the present time. This is a log scale with the y-axis showing a measurement in millions.

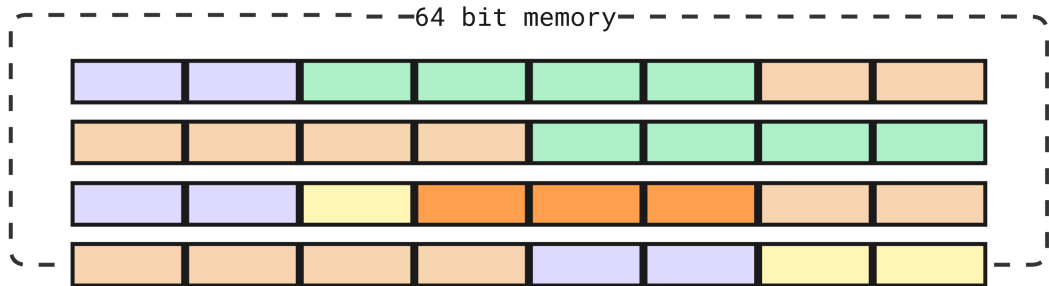


Figure 1.2: Variable length instructions in 64-bit x86 memory. Illustrating 2-byte (purple), 4-byte instructions (green), 6-byte in orange and 1-byte (yellow).

to encode a single instruction can differ, unlike some other architectures that use fixed-length instructions (eg, RISC ISAs)

So instead of padding all instructions out to the same length, variable length instructions allow for more efficient use of main memory.

We present an illustrative model of variable length memory in Fig .1.2. in 64-bit x86 memory. Here, 2-byte instructions in purple, 4-byte instructions in green, 6-byte in orange and one byte in yellow.

1.3 Reduced Instruction Set Computers

CISC Instruction Set Architectures (ISAs) were, until relatively recently, the dominant consumer CPU platform. Excluding desktop Apple Mac systems (which used the PowerPC), Windows desktop computers used Intel x86 exclusively. For most end-users, the CISC capabilities of the ISA were an ever-increasing advantage. It made the standard desktop PC more and more powerful and capable, and allowed software engineers to design and build ever

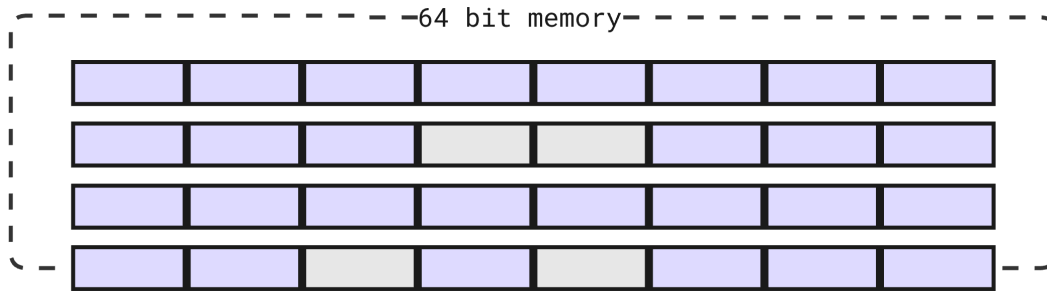


Figure 1.3: RISC fixed length instructions in 64-bit x86 memory. All instructions (purple) are exactly 64-bits in length, with any unused space padded (gray). The same logic applies to RISC 32-bit ISAs (although to a lesser extent)

more CPU demanding applications (CAD/CAM, animation, modelling and simulation, numerical analysis and so on).

However, as the x86 became the dominant desktop solution, there were plenty of alternative CPU architectures available. These architectures (for example, Apple PowerPC, IBM RS/6000) were notable for one major difference to CISC: they were designed with a *reduced* instruction set (RISC) architecture. For example, the Apple Mac Powerbook G5 used a RISC CPU (from IBM) had only around 500 instructions in the totality of the ISA.

Why might a computer architecture be designed with a deliberately smaller instruction set size than a comparable Intel CPU? There are several reasons for this approach, but there are 3 keys ones:

1. **Energy:** RISC CPUs consume less energy because the architecture has much less circuitry
2. **Speed:** RISC instructions *generally* take fewer CPU cycles to execute than CISC instructions because each instruction has less overhead as is considerably simpler in its circuitry implementation.
3. **Complexity:** RISC instructions are always the same size (the word size of the CPU - 32/bit or 64/bit). Because there are no variable length instructions in CISC, there is no need for the complex circuitry to deal with this.

The fixed length instruction size model is shown in Fig. 1.3. In this example all instructions (purple) are exactly 64-bits in length, with any unused space padded (gray) out. The same logic applies to RISC 32-bit ISAs (although to a lesser extent).

Not only is the ISA reduced in terms of the number of instructions, but we may say that each of these instructions is simpler (this being a relative term of course) to design and implement. Indeed, early versions of the ARM RISC did not contain native CPU support for floating point numbers. It was not until 2001 that on-CPU support for floating point numbers were added to the ARMv5TE architecture.

1.3.1 The Mobile Revolution

Of course, we are all familiar with the phenomenal growth in adoption of mobile devices. From the early 1990s models (typically running custom OS code), to the complex and capable Android and Apple devices of today. There are now only *two* mobile OS standards - Android and iOS. In parallel to the rise of mobile devices, a new RISC mobile architecture emerged from a company called Advanced RISC Machines (ARM).

Although ARM was around even in the 1980s, the release of ARM7TDMI 32-bit RISC architecture was an enabling factor for OEMs to launch complex mobile architectures. ARM7TDMI had only 200 unique instructions. This feature alone gave ARM architectures an unrivalled competitive advantage over Intel in the mobile market.

ARM CPUs consumed far less energy than CISC cpus, requiring very little cooling hardware within the device, and enabling battery powered devices to run complex operating systems such as Android (with the linux kernel). Today, ARmv9-A still has only 500–600 instructions, a substantial difference to Intel, and one that ensures that ARM continues to dominate both the mobile market, and the productivity laptop market segment (ARM powers Mac hardware).

1.4 This book

1.4.1 Approach

This book introduces the reader to Computer Architecture. Any book that covers this subject matter needs to decide if it will focus on CISC or RISC ISAs. We have decided to use RISC in the chapters dedicated to assembly programming (Chapters 5 through to Chapter 8), and in the chapters dedicated to microarchitectures (Chapters 10-??).

There are several reasons for this decision, viz:-

1. ARM32 assembly, in particular, is a straightforward language and one that is easy to understand.
2. Both ARM32 and ARM64 are *load and store* architectures (more on that later). So the instructions are WYSIWYG ², unlike intel x86, where a single instruction can mask a complex implementation.
3. ARM is making ever greater inroads into the data centre. For example, AWS reported in December 2024 that more than 50% of its new CPU capacity added in the previous two years was ARM based. Percentages from the other cloud providers is likely to be similar.

As energy consumption continues to grow in significance for data centre operators, the role of RISC based systems, and ARM64 in particular, is becoming more and more significant.

²WYSIWYG - *what you see is what you get*

1.4.2 Book Resources

All the code samples used in this book, along with scripts, READMEs and documentation, can be freely cloned from the Book Github Repo.

► Try this: Clone the Book Github Repo

Before we begin, make sure you've cloned the Book Github Repo

Browse the repo, and take a look at the various samples contained there. For example,

1. In the **C** directory we have some code for you to compile and analyze.
2. In the **arm-assembly** directory there is some examples of different ARM32 assembly programs for you to compile and run.

In the assembly language chapters that follow, we will assume the reader is using either <https://cpulator.01xz.net/?sys=arm> (for an in-browser, ARM32 emulator with limited functionality), or they have installed **qemu-system** emulator for a complete and accurate emulation of ARM32/ARM64 CPUs. Of course, we do not assume the reader has an Apple Mac (Air or Powerbook). But if you do use Apple, then you do not need an emulator, all you need is a text editor and compiler (**gcc**) will do fine.

1.4.3 CPULator

We recommend two strategies for those starting to learn assembly language programming for the first time. The first resource is called *CPULator*. CPUlator is a standard web browser *simulator* for the M7/ARM32 ISA. It is an excellent introduction to assembly programming because it has a decent UI that allows you to step through code, add breakpoints, step into and out of label code etc. CPUlator also allows the programmer to view and modify registers, the flag register, stack pointer, and program counter registers, as well as viewing memory during and after program execution.

There are many useful resources to help the new programmer get started with CPUlator. One youtube channel we recommend is called *LaurieWired*. Laurie has an excellent ARM32 video playlist we recommend you work through these carefully in order to better understand CPUlator.

You will find the link to *LaurieWired* along with some other useful web resources in the **README.md** file in the **csci-comp-arch** repository mentioned already.

Please note, although CPUlator provides an excellent emulation learning environment, by its nature, it cannot fully emulate system calls (and does not support any references to **libc**). Therefore, once you develop your competence in ARM assembly programming, you will need to compile and run actual binaries of your own. This is not possible in CPUlator. For this, we need either a true ARM platform (such as Raspberry Pi 3/4/5, Apple Macbook or Air), or a complete system emulator that allows you to boot into full hardware emulation. For this we recommend using **qemu-system**, which we discuss next.

1.4.4 **qemu-system**

Finally, let us consider a situation where we want to develop software for an architecture different to the one we are working at. For example, consider a situation where a developer is building software for a ARM64/ARM7TDMI Raspberry Pi, but works on a Linux x86 laptop. How can this be achieved? In this situation, virtual machines are not useful as they cannot execute software that is compiled for a different CPU architecture.

In this situation, there are only two solutions:

- Provide every developer with their own Raspberry Pi hardware. But what do we do when we are building OS-level software for expensive mobile platforms? We cannot give every developer their own expensive mobile device for testing.
- Instead, we need to use a software layer that acts as an *emulator* which fully reproduces the platform we wish to develop on. This includes kernel, file system, binary tool chains and so on. This is the purpose of **qemu**.

As can be seen in Fig. 1.4, we use **qemu-system-aarch64** to run a Debian Linux (kernel 5.12) Emulating ARM7DTI/2 core on top of the host running Arch Linux 6.14 (kernel) on a 4-core CPU x86 ISA. This combination of different kernels, file systems and ISAs is not possible in containerization or virtualization.

1.4.5 **qemu-system-aarch64** example

We conclude this section with a concrete example of how to use **qemu-system-aarch64**. Firstly, note that **qemu-system-*** is an full system emulator for a specific ISA. In this case, **aarch64** which is the ARM64 architecture. There are many useful **qemu** resources for you to study - but start at <https://www.qemu.org/> for an excellent overview of the tools and tool chains available.

Consider the following **qemu-system-aarch64** command:

```
qemu-system-aarch64
-machine raspi3b
-cpu cortex-a72
-smp 4 -m 1G
-kernel kernel.img
-dtb treeblob.dtb
-drive "file=bullseye.img,format=raw,index=0,if=sd"
-append "rw earlyprintk loglevel=8 console=ttyAMA0
,115200 dwc_otg.lpm_enable=0 root=/dev/mmcblk0p2
rootdelay=1"
-device usb-net,netdev=net0 -netdev user,id=net0,
hostfwd=tcp::5555-:22
-nographic
```

When we execute this **qemu-system-aarch64** instruction on our host (and assuming **kernel.img**, **treeblob.dtb** and [Debian] **bullseye.img**

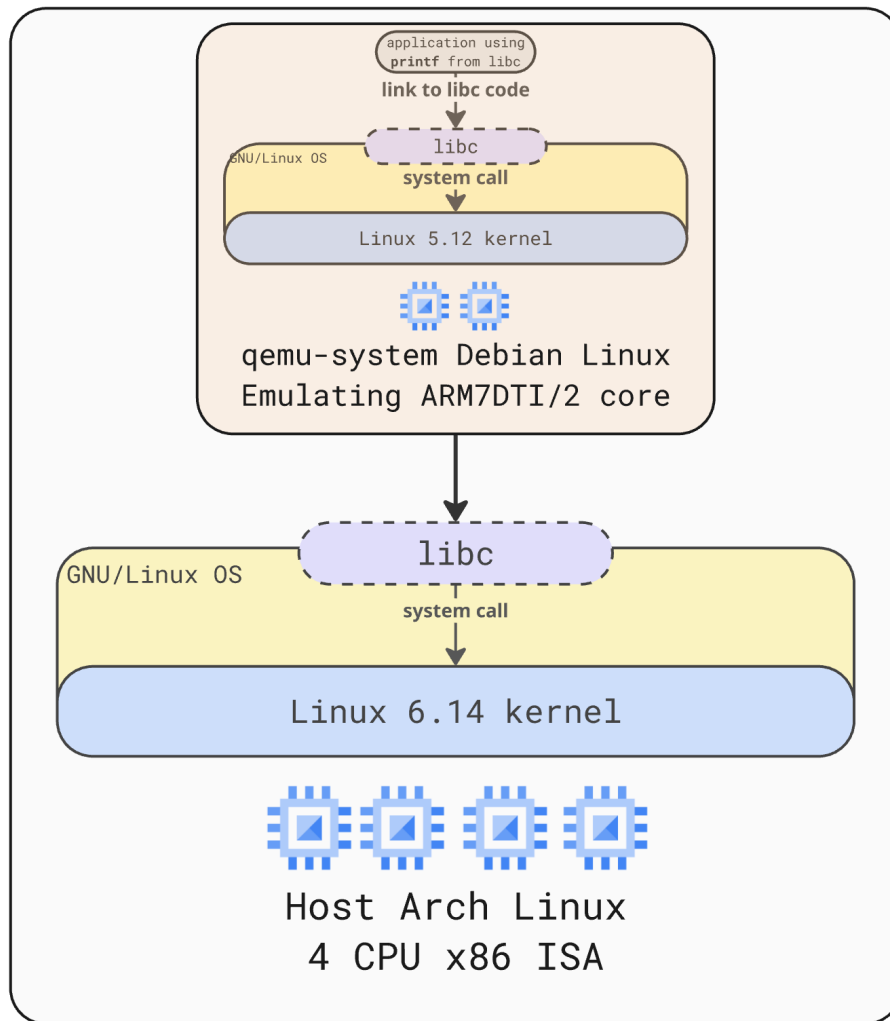


Figure 1.4: Using `qemu-system-aarch64` to run a Debian Linux (kernel 5.12) Emulating ARM7DTI/2 core on top of the host running Arch Linux 6.14 (kernel) on a 4-core CPU x86 ISA

are available locally), then we will shortly start a Raspberry Pi 3B ARM32 cortex-a72 with the kernel and filesystem of our choice. You can find this script in the book github repository (located in `qemu/start.sh`).

We can then use `ssh` to connect to `localhost` on port 5555 and our instance is ready to begin using for our project development!

1.5 Chapter Summary

In this chapter we have introduced the reader to CISC versus RISC ISAs. We discussed the design principles that motivate the development of CISC ISAs: that ISA complexity and circuitry complexity outweigh energy consumption performance. Conversely, we noted that RISC ISAs are relatively less complex and significantly less energy demanding.

We do not need to consider CISC ISAs any further as the remainder of this book will look exclusively at either ARM32 or ARM64 systems in general.

You are advised to develop a strong understanding of these two architectures and the advantages and disadvantages of each. In this chapter we also introduced you to the source code resources for you to use, along with some suitable emulation platforms. In the early chapters of this book, **CPULATOR** is ideal because it gives us a nice development and debugging experience inside a standard web browser. But as we develop our program complexity, we will start using `qemu-system` in order to use this full set of `libc` tools present in the Linux OS of our choice.

In the following chapter we will discuss the internal structure of an executable program. It is important to understand this structure well, because when we begin assembly language programming the structures of our executable will be explicitly named and referenced.

1.6 Chapter Exercises

Exercise 1.6.1 *Both ARM7DTI and ARM64 are **load and store** ISAs. Write a brief note comparing load and store with the “traditional” CISC memory access ISA instructions. Give an example to illustrate your answer.*

Exercise 1.6.2 *The ARM specification discusses emulation. It refers to a limited emulation mode known as **ARM Semihosting**. Write a brief explanation on this and give an example to support your answer. How does CPULATOR support **ARM Semihosting**?*

Exercise 1.6.3 *CPULATOR is an ARM **simulator** whereas `qemu-system-aarch64` is an ARM **emulator**. Write a brief note explaining the difference between these two concepts and give some examples.*

Exercise 1.6.4 *Here is a full list of the `qemu-system` emulators on our system:*

<code>qemu-system-aarch64</code>	<code>qemu-system-ppc</code>
<code>qemu-system-alpha</code>	<code>qemu-system-ppc64</code>
<code>qemu-system-arm</code>	<code>qemu-system-riscv32</code>

<code>qemu-system-avr</code>	<code>qemu-system-riscv64</code>
<code>qemu-system-hppa</code>	<code>qemu-system-rx</code>
<code>qemu-system-i386</code>	<code>qemu-system-s390x</code>
<code>qemu-system-loongarch64</code>	<code>qemu-system-sh4</code>
<code>qemu-system-m68k</code>	<code>qemu-system-sh4eb</code>
<code>qemu-system-microblaze</code>	<code>qemu-system-sparc</code>
<code>qemu-system-microblazeel</code>	<code>qemu-system-sparc64</code>
<code>qemu-system-mips</code>	<code>qemu-system-tricore</code>
<code>qemu-system-mips64</code>	<code>qemu-system-x86_64</code>
<code>qemu-system-mips64el</code>	<code>qemu-system-xtensa</code>
<code>qemu-system-mipsel</code>	<code>qemu-system-xtensaeb</code>
<code>qemu-system-or1k</code>	

Taking any two of these platforms (except `qemu-system-i386` and `qemu-system-x86_64`) compare and contrast them under the following headings:

1. *Clock speeds*
2. *ISA size (number of instructions)*
3. *Energy consumption*
4. *Operating System / kernel*
5. *Number of units still running*
6. *typical workload*

Program Representation and Execution



2.1 Introduction to the Executable and Linkable Format

The process by which a C program is eventually transformed into a binary image (that can be loaded and run on a modern computer), is rather complex.

In simple terms, and for a C program called `main.c`, we can define 3 key phases that turn your C programs into executable code:

1. Compiler to Assembly (output: `main.s`) The compiler translates the program into low-level assembly code specific to the target CPU.
2. Assembler to Object File (output: `main.o`) The assembler converts assembly into machine code, producing a file that may have unresolved external references (symbols).
3. Linker to Executable (output: `a.out` ELF file) The linker combines one or more object files and libraries, resolves symbols, lays out memory, and produces a final executable in the ELF format. The basic flow of this process is shown in figure 2.1.



Figure 2.1: C program to ELF executable compilation stages (simplified)

Irrespective of the programming language, almost all unix-like systems (with the exception of OSX) store executable code in the **Executable and Linkable Format** (ELF).¹ The ELF format is relatively complex, containing a number of sections that describe a program's contents and a smaller number of segments that determine how those contents are loaded into memory. The discussion that follows is necessarily a simplification. Readers wishing to understand more should start with the `objdump` man pages on their local Linux OS.

¹ELF gABI Specification

```

/*
 * simple.C
 * a file to show the layout of objects
 * in the a.out executable
 * using objdump -s -C
 */
#include <stdio.h>

// initialized global variable (r/w)
int global_var1 = 100;

// uninitialized global variable (r/w)
int global_var2;

// initialized global variable (r/w) string
char global_var3[] = "This is a global string";

int main(int argc, char* argv[]) {
    int local_var1 = 99;
    int local_var2;
    // reference to printf is in .dynstr
    // the string ("This is ..") is r/o in .data
    printf("Locals are %d %d\n", local_var1, local_var2);
    printf("Globals are %d %d %s\n",
        global_var1,
        global_var2,
        global_var3);
    return 0;
}

```

Figure 2.2: Simple C to show the layout of objects in an ELF file

To demonstrate the layout of an ELF file, we have developed a simple simple C program (shown in Fig 2.2), that has a number variable types, including, global initialized int, global uninitialized int, global initialized character array, initialized function local variable, and uninitialized function local variable.

2.2 ELF Layout

When the C program shown in Fig. 2.2 is compiled and linked, ready for execution, the file (by default called **a.out**), will have the structure shown in Fig. 2.3

As you can see from the ELF layout shown in Fig 2.3, the structure of the ELF file contains the *static* properties of the C code - in other words - the properties of the program at rest on the disk. What's missing from this picture is the *dyanmic* properties of the program when it gets loaded into memory at *process* start time. We discuss this in the next section.

Next we will map the various parts of our program into the ELF sections as defined in Fig. 2.3

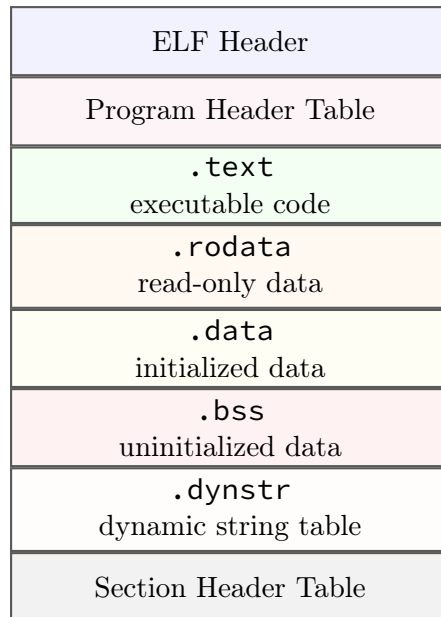


Figure 2.3: Simplified ELF file layout

2.2.1 Mapping C Program Components to ELF Sections

The C program defined in Fig. 2.2 is not especially interesting or useful. The purpose of the program is to show you where the various elements of a C program are placed in the ELF section model introduced in Fig. 2.3. We should also remember, according to the ELF standard, some sections are *read only (ro)* whereas some are *read write (wr)*

1. **.text**: This *read-only* section stores all the instructions from the program in the order in which they are defined in the C program. For example:

```
int local_var1 = 99;
int local_var2;
printf(...)
printf(...)
```

2. **.rodata**: This section also stores *read-only* data such as strings that are passed into a function (for example, `printf`). So this section contains "Locals are %d %d \n" and "Globals are %d %d %s \n". These strings are read-only because they can not be modified once compiled into the program code.
3. **.data**: This section is used to store initialized global data, so the variables `global_var1` and `global_var3` are placed into this section. Note that both these variables are modifiable. This is true of any variable data placed into **.data**. We will see this section in much more detail in the coming chapters.
4. **.bss**: This section contains *uninitialized* global data. So in this section we expect to find `global_var2`
5. **.dynstr**: This section is used to record what dynamic runtime interpreter

is needed during program execution. For a program that contains dynamically linked shared libraries, for functions such as `printf`, the section `.dynstr` will typically contain the string `printf.libc.so.6`, along with several other entries

2.2.2 Unmapped C Program Data

If you've been following the preceeding section closely, you will notice that some of our C program data was not mapped to an ELF section. For example:

```
int local_var1 = 99;
int local_var2;
```

The variables `local_var1` and `local_var2` are not stored in any data section in the ELF file. So where are they stored? They are both stored on the *stack*.

Each thread executing inside a process gets its own stack in memory. This means that should a process have more than one thread, each thread will be allocated its own stack segment in virtual memory. The stack is a very important and specialized region of memory. It is highly *dynamic*, growing and shrinking over the lifetime of the process. In section 2.3.2, we briefly review the key ideas of the stack, but we dedicate a complete chapter to it - Chapter 7, where important matters such as local variables, saving and restoring registers, and the implementation of procedures, is discussed in detail.

► Try this: Using `objdump` to view ELF sections

If you have access to a Linux OS you can use `objdump` to view the ELF contents of an executable file. If you don't use a Linux OS, you can still do this at the GCP command line!

Before you begin, make sure you've cloned the Book Github Repo

- Change into the directory `C` directoy
- Compile `simple.C` using `gcc` or `clang` - for example: `gcc simple.C`
- Use `objdump -s -C a.out` to view the ELF sections of the file.
- Look out for the `.dynstr` section ... what do you see?
- Look out for the `.data` section ... what do you see?

Fun question - do `gcc` and `clang` produce significantly different object files?

2.3 Process address space of an ELF program

When the ELF executable becomes a *process*, it gains a number of additional memory resources, such as stack, heap and a shared library section. These are the sections needed for correct process initialization and execution.

2.3.1 Sections to Segments

Sections describe the logical contents of a program within the file. They are defined in the Section Header Table and are used primarily by the compiler and linker.

Segments, on the other hand, define how the program is loaded into memory. They are specified in the Program Header Table and are the structures actually interpreted by the operating system loader at runtime.

The connection between the two is established by the linker: The linker groups sections into segments based on their required memory properties (e.g., read, write, execute). Each segment may contain parts of one or more sections. The operating system loads segments, not sections, into memory. This is why the terminology in Fig. 2.4 switches from *sections* to *segments*

The heap and stack sections are *dynamic* regions: they grow and shrink as the process executes. When the process exits, the stack, heap and shared library sections are *unmapped*, in other words, they are returned to the kernel. This arrangement is shown in Fig. 2.4.

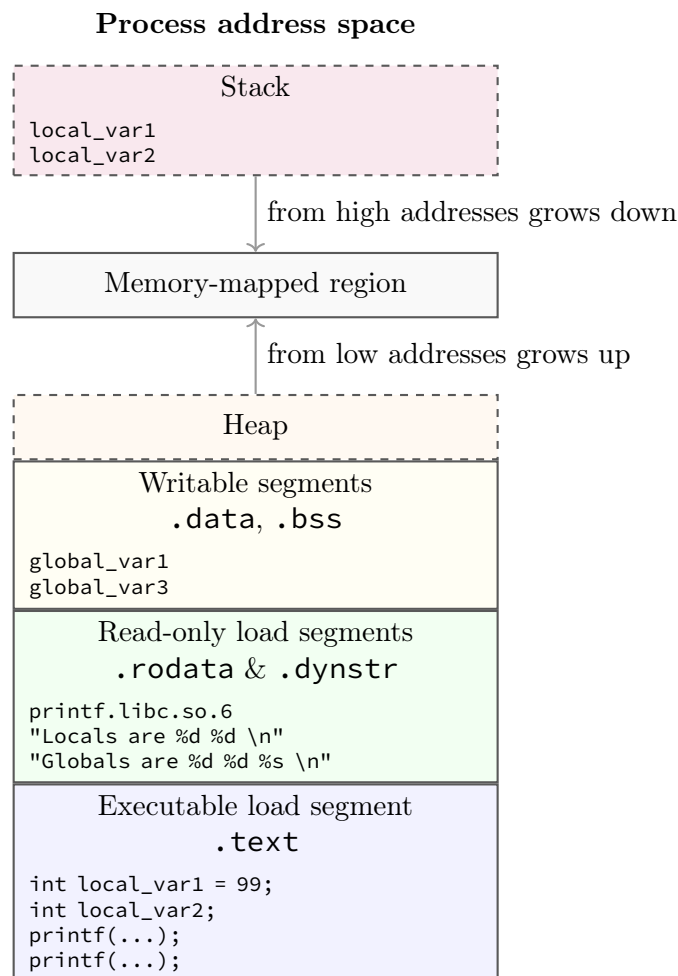


Figure 2.4: Simplified process address space for an ELF program. Fixed size segments are shown with solid borders, dynamic sized segments (stack and heap) are shown with dashed borders. A simplified placement of data and code is also shown.

2.3.2 The Stack

Function local variables are always placed on the stack. As each thread gets its own stack space, such local variables are always private to the thread and are not shared between threads. Contrast this with data stored in writable segments (`.data`, `.bss`). These data are seen by all threads and, unless access is properly controlled, a race condition may arise. A race condition is a situation where two or more threads are altering a value at the same time, possibly resulting in corruption or incorrect values.

In most ISAs, the stack grows downward, from high addresses to low addresses. Technically this is called the Full Descending stack model, and in Chapter 7 we discuss the correct way to add and remove data from the stack. For now, let us just make a few observations about thread stacks

1. Stacks are a finite resource. By default, on a typical Linux based system, the default size is 8192kb, or 8MB, per thread
2. Stacks are used for *temporary* or *short-lived data*, for example data needed only for the duration of a function. This short-lived data must be returned to the operating system via proper management of the stack pointer (`sp`) before the function returns.
3. Failure to manage the stack pointer correctly can lead to a stack leak or a stack overflow. We will discuss these further in Chapter 7, along with the ARM instructions we use to manage the stack.

2.3.3 Memory Mapping

The memory-mapped region of a process is the portion of the virtual address space used for mappings created with system facilities such as `mmap`. In an Executable and Linkable Format program, this region sits between the heap and the stack (conceptually), though its exact position and growth pattern are determined by the kernel's virtual memory layout.

One of the primary uses of the memory-mapped region is to map shared libraries into the process address space. When an ELF executable depends on dynamic libraries, the dynamic linker loads these libraries by mapping their segments directly into the process's address space.

This allows multiple processes to share the same physical memory for read-only portions (such as code), improving efficiency. These mappings typically include read-only segments (e.g., `.text`, `.rodata`) and sometimes read-write regions for relocation and global state.

2.3.4 The Heap

The heap is a region of dynamic memory that typically grows upward, from lower to higher virtual addresses. Memory in the heap is allocated at runtime using functions such as `malloc`, which are implemented in the C standard library and ultimately rely on system mechanisms like `brk` or `mmap`.

Heap allocations are often long-lived, potentially persisting for the lifetime of the process unless explicitly freed. The heap is a per-process resource,

shared by all threads within the same process. As a result, concurrent access to heap-allocated data must be properly synchronized (e.g., using mutexes).

In principle, there is no fixed upper bound on heap size. In practice, it is constrained by factors such as the process's virtual address space limits (e.g., `ulimit`), operating system policies (including `cgroups`), and available physical memory.

We will not focus extensively on heap allocation in this book, as it requires interaction with either system calls or the C standard library. However, Chapter 8 introduces how to integrate the C library into an ARM assembly program (e.g., for using `printf`), and the same approach can be extended to support dynamic allocation functions such as `malloc`.

2.4 Chapter Summary

2.5 Exercises

Exercise 2.5.1 *Why is it beneficial for multiple processes to share the same physical memory pages for shared libraries? In your answer, include implications for memory usage and overall system performance.*

Exercise 2.5.2 *How does virtual memory allow each process to have its own address space while still sharing physical memory? In your answer, explain the role of address translation.*

Exercise 2.5.3 *Why is it important that the stack grows downward and the heap grows upward in a process address space? What problem does this design help prevent?*

Exercise 2.5.4 *What would happen if each process had to load its own private copy of every shared library it used? Discuss both memory usage and system scalability.*

Exercise 2.5.5 *How does the operating system ensure that shared memory (such as shared libraries) is protected from unintended modification by processes?*

Exercise 2.5.6 *Why are memory-mapped files considered more efficient than traditional file I/O in some cases? Provide at least two reasons.*

Exercise 2.5.7 (Using `ulimit`)

Exercise 2.5.8 (Using `objdump`)

Assembly Language Programming

We begin our study of assembly by focusing on ARM7DTI which is a ARM32-bit platform. The ARM64 platform contains many additions and updates, and in most real-world scenarios you will be building and deploying software on this ISA. However, the 32-bit variant is interesting from the pedagogical point of view because it is a smaller and significantly simpler ISA.

As we proceed through the following chapters we will compare C programming and assembly to understand how tasks in high level programming languages are implemented in low level assembly.

3.1 Basic Instructions: Adding and Subtracting

Let us begin by doing a very simple task: adding two numbers together and storing the result in a variable. Let's choose 5 and 10 as our two numbers, and c as the variable. Mathematically this is simply $c = 5 + 10$, and in C:

```
int c = 5 + 10;
```

How do we implement this in assembly? We need to use the assembly assignment instruction (**mov**) and the addition mathematical function (**add**) instruction:

```
mov r1, #5      // move 5 into r1
add r3, r1, #10 // add r1 and 10
```

You notice immediately that in the assembly code we added in *two* steps: first, assignment using **mov** then add using **add**.

Why did we not do this:

```
add r3, #5, #10 // why not this?
```

This instruction will not assemble and will generate an error. What is the error?

```
Error: bad expression -- 'add r3,#5,#10'
```

It tells us that the *first* operand to **add** cannot be an immediate (in other words, it cannot be a number, it must be a register)

Why this limitation? As we will see in Chapter 9 this constraint is due to the fact that there is *no space* in a 32-bit instruction to encode two arbitrary immediate values. In fact, even encoding one arbitrary immediate has limitations which will be discussed in detail in Chapter 9.

3.1.1 Discussion

Notice some interesting features of the assembly code. Let's deal with each of these in turn:

1. We use *registers* to store data. Registers are word-size storage areas on-board the CPU itself. Because they are on the CPU, register updates happen in one CPU clock cycle (in other words, zero latency).

In ARM7DTI there are 15 *general purpose* registers. These registers are named **r0-r14**. Register **r15** is a special purpose register and should not be used by the programmer.

As there are only 15 general purpose registers, we must be careful not to use them unnecessarily, otherwise we may run out of registers.

2. Two new instructions have been introduced: **mov** and **add**. Notice also that when we want to use an integer value, we use the **#** sign in front of it. For example, **#10**. This is a language requirement. Numbers like this are called *immediates*. Later we will explain the derivation of the word, but for now, we will accept this term as-is.

3. In ARM7DTI all instructions are either right-to-left or left-to-right ordered¹. The **mov** instruction is right-to-left ordered. It means take the *value* from the right hand side and place it into the *location* on the left hand side. Is it possible to have a **mov** that uses *two* locations (eg, **mov r1, r2**)? Yes. Is it possible to have a move with two values **mov #3, #4**? Of course, no.

*Notice that **mov** is a two operand instruction*

4. The **add** instruction is also a right-to-left instruction. Notice that **add** is a *three* operand instruction. So the code **add r3, r1, r2** is understood as “add the value in **r2** to the value in **r1** and place the result in **r3**”. Does this instruction change the value of either **r1** or **r2**? No. Does it change the value of **r3**? Yes.

Some variants of **add** that achieve the same result:

```
mov r1, #5
add r3, r1, #10
```

and

```
mov r1, #5
add r3, r1
```

Notice here that **r3** does not contain 15 but rather it contains 10.

5. Finally, notice the following illegal instructions:

¹most are right-to-left ordered

```
// illegal
add r3, #10, r1 // cannot add a reg to a constant
// causes this compiler error
Error: constant expression expected -- 'add r3,#10,r1'
```

and

```
// illegal
add r3, #10, #5
// causes this compiler error
Error: bad expression -- 'add r3,#10,#5'
```

We cannot add a register to a constant, nor can we add a constant to a constant. The reason for these constraints is not exactly as you might suppose. We will discuss these types of constraints in Chapter ?? (Machine Code), but suffice it to say, the constraints are due to the nature of ARM7DTI RISC ISA, and in particular, the constraint of fixed word-size instruction length.

3.1.2 Two's Complement

In C, we use the type `unsigned` to specify that the variable cannot take on a value of less than zero.²

If we deal only with unsigned integers, then all the bits of the word size are allocated in representing the number. However, as you know, there are many negative-type numbers, and we must have a way of representing them within the 32-bit word.

Over the years, there were many attempts to represent negative numbers inside computing hardware. Today, the standard is *two's complement* method which we outline here:

A two's complement negative number is computed as follows:

1. Take the bit pattern of the positive number and invert all the bits ($0 \rightarrow 1$ and $1 \rightarrow 0$)
2. Add one (bit) to the resulting number.

Let us look at an example: we want to represent -5 . To make the study of this a little easier, let us deal with a 6-bit computer (the principles generalize to 32- and 64-bit systems in exactly the same way):

1. $5 = 000101 \rightarrow 111010$
2. Now, add 1 to $111010 = 111011$

The largest non-negative number expressible in a computer of word size N bits, is $2^N - 1$. This means in a six bit CPU, the largest non-negative number is $2^6 - 1 = 63$.

Notice that 111011 can be thought of in two different ways. As a non-negative number it is

²However, due to the implicit type conversion rules in C, an `unsigned` can silently change to `signed` if it is later assigned a negative value

$$1 * (2^5) + 1 * (2^4) + 1 * (2^3) + (0 * 2^2) + (1 * 2^1) + (1 * 2^0) = 59$$

However, because the most-significant bit (MSB) is 1, this number is *also* a negative number. The negative number is computed as follows:

$$-1 * (2^5) + 1 * (2^4) + 1 * (2^3) + (0 * 2^2) + (1 * 2^1) + (1 * 2^0) = -5$$

Let us test to see if *addition* under two's complement works correctly. We know that $27 + -5 = 22$. Does it work? Let's see:

$$\begin{array}{r} 011011 \\ +111011 \\ \hline \text{---} \\ 1 \overset{\text{carry}}{\longleftarrow} 010110 \end{array}$$

As $010110 = 22$ we can see that addition and subtraction of positive and negative two's complement works correctly.

Note, in the example above, we had a *carry-out* of the MSB. This arises because *there was no more space* left in the 6-bit computer for the full answer. (in other words, the result needed a 7-bit computer, which we were not using)

3.1.3 Mathematical Instructions

Registers can hold positive or negative numbers without the need to use words such as **signed**, or **unsigned**:

```
mov r1, #10 // move 10 into r1
mov r2, #-5 // move negative 5 into r2
mov r3, r2  // move (r2) negative 5 into r3
```

Let us look briefly at **sub** instructions.

```
mov r1, #10 // move 10 into r1
mov r2, #-5 // move negative 5 into r2
mov r3, #5  // move 5 into r3
add r4, r1, r2 // result is 5, stored in r4
sub r4, r1, r3 // result is 5, stored in r4
sub r5, r2, r1 // result is -15, stored in r4
// etc
```

Table 3.1 shows some of the more commonly used ARM7DTI mathematical operations. It is by no means complete. Please refer to [Ins25] for a comprehensive treatment of ARM7DTI instructions.

3.1.4 A note on **{cond}{S}**

You may notice in Table 3.1 and Table 3.2 the text **{cond}{S}** appended to the end of the mathematical instruction (for example **sub{cond}{S}**). We should first note that there are two different things going on here, both of which are optional:

Instruction	Type	Description
<code>add{cond}{S}</code>	Arithmetic	Adds two registers or a register and an immediate value, storing the result in a destination register.
<code>sub{cond}{S}</code>	Arithmetic	Subtracts one register or immediate value from another, storing the result in a destination register.
<code>mul{cond}{S}</code>	Multiplication	Multiplies two registers and stores the 32-bit result in a destination register.
<code>mla{cond}{S}</code>	Multiplication	Multiplies two registers, adds a third register to the product, and stores the result in a destination register.
<code>umull{cond}{S}</code>	Multiplication	Unsigned multiply long, multiplies two 32-bit registers to produce a 64-bit result, stored in two registers.
<code>smull{cond}{S}</code>	Multiplication	Signed multiply long, similar to <code>UMULL</code> but for signed integers.
<code>adc{cond}{S}</code>	Arithmetic	Adds two registers or a register and an immediate value with carry, storing the result in a destination register.
<code>sbc{cond}{S}</code>	Arithmetic	Subtracts one register or immediate value from another with borrow, storing the result in a destination register.

Table 3.1: Top 10 ARM32 Mathematical Instructions

1. `instr{cond}` indicates the instruction should execute only if `{cond}` is *true*. This is called a conditional instruction. For example `mullt` consists of `mul` and `lt`. Here `lt` means *less than*, and signifies that the `mul` operation should only execute *if the condition flags permit*.
2. `instr{s}` indicates the instruction outcome of the instruction (less-than, zero, greater-than etc) should be sent to the *conditional registers* so that other instructions may access the information.

We will discuss both `instr{cond}` and `instr{s}` in the next chapter. For now, to keep things simple, we omit these optional appendages and focus on the mathematical operation itself.

3.1.5 Multiplication using the `mul` family

We now turn to multiplication operations. There are two main multiplication instructions in ARM assembly ISA: `mul` and `mla`. We will briefly discuss each. We use `mul` when we want to store the result of the operation into some register as follows:

```
mov r1, #10
mov r2, #5
mul r4, r1, r2
```

We can understand this line 3. as “*multiply the value in `r1` by the value in `r2` and store the result in `r3`*”

Here, `r4` now contains 50. This is equivalent to the C code:

```
int c = 10 * 5;
```

In C we often want a *multiplication-accumulation* function as:

```
int c = 10 * 5;
d += c;
```

Because multiplication-accumulation is such a common mathematical operation, ARM assembly has a special *four* operand instruction, (`mla`) to do this:

```
mov r1, #10
mov r2, #5
mla r4, r1, r2, r5
```

Note here that `r5` is the accumulation register, and is an accumulation of the product of `a` and `b` plus whatever was in `r5` previously.

3.1.6 Logical Operators

Finally, let us briefly mention the logical Operators in ARM7DTI ISA:

Table 3.2: ARM32 Logical Operator Instructions

Instruction	Description
<code>and{cond}{S} Rd, Rn, <Operand2></code>	Bitwise AND between <code>Rn</code> and <code>Operand2</code> , result in <code>Rd</code>
<code>orr{cond}{S} Rd, Rn, <Operand2></code>	Bitwise OR between <code>Rn</code> and <code>Operand2</code> , result in <code>Rd</code>
<code>eor{cond}{S} Rd, Rn, <Operand2></code>	Bitwise XOR between <code>Rn</code> and <code>Operand2</code> , result in <code>Rd</code>
<code>bic{cond}{S} Rd, Rn, <Operand2></code>	Bitwise AND of <code>Rn</code> with NOT of <code>Operand2</code> , result in <code>Rd</code>
<code>orn{cond}{S} Rd, Rn, <Operand2></code>	Bitwise OR of <code>Rn</code> with NOT of <code>Operand2</code> , result in <code>Rd</code>

3.2 Program Flow and the Program Counter

3.2.1 Sequential Execution

Assembly programs have a wide number of structural characteristics in common with high level languages (such as C). These structural similarities are not total. There are plenty of points of difference. For the programmer writing assembly code for the first time, one of the most obvious differences is in program flow. In assembly, there is no *scope-control* equivalent.

This means that your assembly code will start at the entry point and continue sequentially until your program either exits or branches. We will discuss branching in Chapter 4.

In the example below, the program begins executing at line 1. and stops at line 6.

```
[numbers=left, stepnumber=1]
mov r1, #10
mov r2, #-5
mov r3, #5
add r4, r1, r2
sub r4, r1, r3
sub r5, r2, r1
```

3.2.2 The Program Counter Register (pc)

In ARM7DTI and ARM64 there is a dedicated register known as the *Program Counter* register (**pc** register), the job of which is to point to the address in memory of the currently executing instruction ³. Suppose in the above example, the instructions are placed into memory in the following sequential hexadecimal addresses:

```
//Address  Instruction
00FFFF00  mov r1, #10
00FFFF04  mov r2, #-5
00FFFF08  mov r3, #5
00FFFF0C  add r4, r1, r2
00FFFF10  sub r4, r1, r3
00FFFF14  sub r5, r2, r1
```

Then the program counter will change over time as shown in Fig. ??:

The **pc** is modifiable by the programmer. But we need to be extremely carefull in doing so, because changing the programmer counter to address *A* will cause the instruction at address *A* to be read and processes by the CPU. *A* should always contain a valid instruction - if it does not - the program will crash.

³this is a slight simplification which we refine in Chapter 10

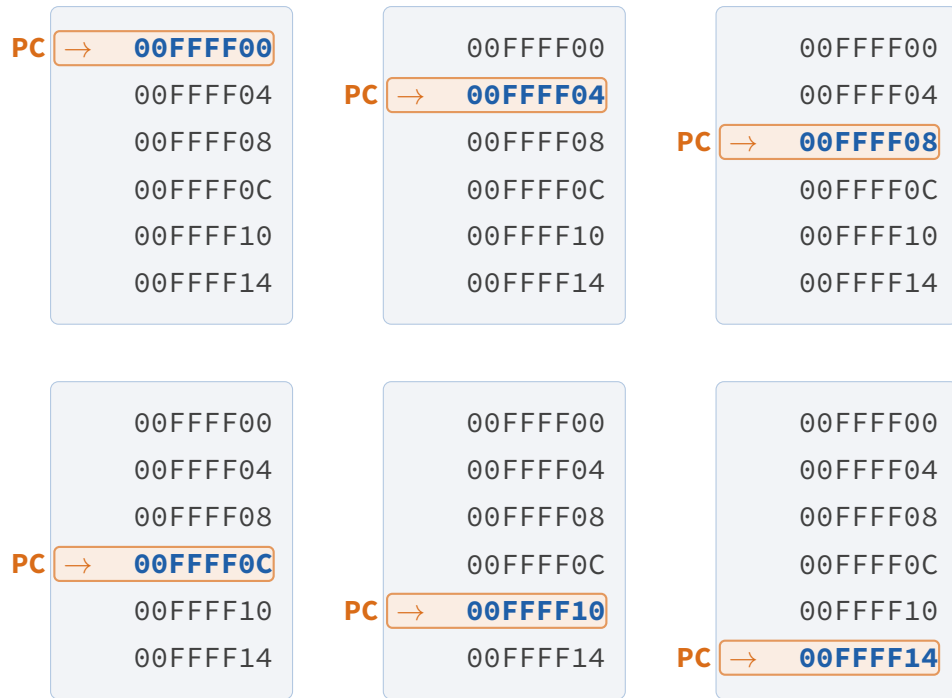


Figure 3.1: Program counter movement through memory addresses.

```
int main(int argc, char* argv[] {
    // method 1 - use "return"
    return 0;

    // method 2 - use "exit"
    exit(0);
}
```

Figure 3.2: We can use `return 0` or `exit(0)` to return from `main`. The best practice is to use `return`

3.2.3 Program Exit

A C program can exit and return control to the operating system in one of two ways, both shown in Fig 3.2.

Note in Fig. 3.2 - by convention - a program that returns 0 to the operating system (eg, `return 0`) indicates it exited normally. A non-zero value (eg, `exit(3)`) indicates the program encountered an error or some abnormal ending.

If the function call stack is deep and unwinding it via return values is inconvenient, a function can use `exit()` to terminate the program in the function. For example see Fig 3.3:

In assembly there is a direct equivalent to the C `return` and `exit()`. We will briefly show the options in assembly, but the instructions we will explain in detail in future chapters.

```
int main(int argc, char* argv[] {  
    A();  
}  
void A() {  
    B();  
}  
void B() {  
    C();  
}  
void C(){  
    if (fatal_error) {  
        // just exit here  
        exit(2);  
    }  
}
```

Figure 3.3: Due to the deep nesting of function calls, it is easier here to `exit(2)` rather than unwinding the function call stack

```
main:  
    // the "return" pattern  
    push {lr} // save lr on the stack  
    // implement your application  
    mov r0, #0 // set the return value in r0  
    pop {pc} // set the pc to the lr  
  
    // the "exit" pattern  
    // implement your application  
    mov r0, #0 // set the return value  
    bl exit // call exit from libc
```

Figure 3.4: In assembly we can `return 0` by using using the first pattern and call `exit(0)` using the second pattern.

3.3 Chapter Summary

In this chapter we introduced the basic concepts of ARM7DTI assembly programming. We reviewed the main mathematical and logical operations, the role of the Program Counter (`pc`). We also discussed how to terminate an assembly language program (`return` versus `exit()`) and the tradeoffs of each approach. We introduced some important topics that will be covered later in this book, in particular:

1. Stack manipulation using `push` and `pop`.
2. Branch with link using `bl`
3. Setting the `pc`.
4. Conditional execution and/or conditions register updates using `instr{cond}{S}`

We will return to these interesting and important topics in future chapters.

3.4 Chapter Exercises

Exercise 3.4.1 *Implement the following C program in assembly:*

```
int main(int argc, char* argv[] {
    // we encountered an error
    return 16;
}
```

Exercise 3.4.2 *Convert this assembly program to C and compile it:*

```
mov r0, #5
mov r1, #50
mov r2, #0
mla r3, r2, r1, r4
```

Use `printf` to test the result is correct.

Exercise 3.4.3 *Modify the previous program to “return” to the operating system with a 0.*

Exercise 3.4.4 *Write the assembly code to use the logical `and` operator to test if a number is odd or even. If the number is odd place 0 into `r0` and if it’s even, place 1 in `r0`.*

Exercise 3.4.5 *This code does not properly “return” to the operating system. Why not? What does this code actually do?*

```
push {lr}
mov r0, #0
pop {lr}
```

Conditional Execution

4.1 Branching

All programming languages allow us to control the flow of execution based on some logical test. Of course, we are all familiar with this typical program form:

```
if (condition_is_true) {
    // execute instructions a,b,c,d
}
else {
    // execute instructions e,f,g,h
}

// execute instructions i,j,k,l
```

In C we use **if** and **else**, along with scope delimiters **{ }**. Scope delimiters are used to specify how many instructions are to be included in the **if** part, and how many in the **else** part. In C, when there are no curly branches present following the **if** and/or **else**, then the scope is limited to the end of the next line delimited by **;**

More formally, we can say that code containing **if** and **else** has a *branch* structure. You may have seen *flowcharts*, which are useful in understanding the logical branches in a program. Let's look at the above C program in the form of a flowchart as shown in Fig 4.1

Although in a C program we use **if/else** keywords to create a branch in the instruction flow, in ARM7DTI assembly we use a combination of instructions to effect this. But before we look at the appropriate ARM instructions, let us think a little more deeply about **if** tests.

4.1.1 **if** test as subtraction

You may not have realised it, but all **if** tests can be thought of as subtraction operations. For example, suppose we have $a = 5$ and $b = 10$, then

1. **if** $a > b$ is true if $a - b > 0$ which means the result is not zero and not negative
2. **if** $a \geq b$ is true if $a - b > 0$ or $a - b = 0$ which means the result is either zero or not negative
3. **if** $a == b$ is true if $a - b = 0$ which means the result is zero
4. **if** $a < b$ is true if $a - b < 0$ which means the result is negative

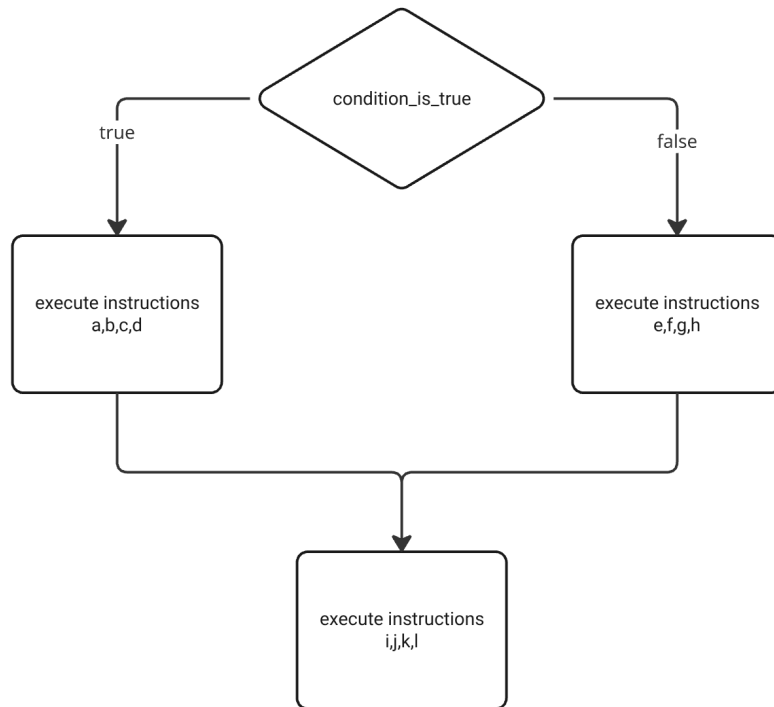


Figure 4.1: Flow chart for the C program showing two *branches*

5. `if a <= b` is true if $a - b < 0$ or $a - b = 0$ which means the result is negative or zero

Notice in the above that we use only two *indicators*: the zero indicator and the negative indicator. In C we don't have to test for these indicators, the compiler generates the code behind the scenes to do the subtraction. But in ARM7DTI assembly we must understand this process, because there is no direct `if/else` equivalent, there is only *subsection* of the operands and the *test* of the outcome. So next we look at these *indicators*. Where are they stored and how do we access them.

4.2 The Current Program Status Register

4.2.1 NZCV flags

The ARM7DTI processor has a dedicated register known as the Current Program Status Register **CPSR**, which is updated with the outcome status of mathematical operations. As mentioned above, this outcome status is (a) result was a negative, (b) result was a zero. We also add two new outcome status indicators (c) the result includes a *carry-out*, and (d) the result includes a *overflow*. Although the **CPSR** is a 32-bit register, only the last 4 least significant bits (LSBs) are of interest to us. Collectively, these 4-bits are often referred to as the **NZCV** bits of the **CPSR** register.

The CPSR register cannot be directly read or written to by the programmer. (only the ALU can update the CPSR).

4.2.2 Carry-out

Two's complement representation is covered in Chapter 3; here we focus specifically on how carry-out and overflow set the **NZCV** flags. Consider a 6-bit example: $25 + -5$

$$\begin{array}{r}
 011011 \\
 +111011 \\
 \hline
 1 \xleftarrow{\text{carry out}} 010110
 \end{array}$$

Note, we can say a carry-out occurs whenever we require one bit *beyond* the MSB to store the result. So in this case, the **NZCV** register is **0010**

4.2.3 Overflow

Although an overflow may seem to be similar to a carry-out, it is completely different. We get an overflow whenever we add two numbers of the same sign, and the result is a number of a different sign. For example, adding $-32 + -1$ in a 5-bit computer:

$$\begin{array}{r}
 100000 \\
 +111111 \\
 \hline
 1 \xleftarrow{\text{carry out and overflow}} 011111
 \end{array}$$

Notice here that the sign changed - we added two negative numbers and the result was a positive number (+31). This is both an overflow and a carry-out. So in this case, the **NZCV** register is **0011**.

Overflow does not *always* result in a carry out. For example

$$\begin{array}{r}
 010000 \\
 +010000 \\
 \hline
 100000
 \end{array}$$

In the above example, we have $16 + 16 = 32$ so there is no carry-out (because 32 can be represented in a 6-bit word), yet the MSB bit has changed from positive to negative. It is important to note that, in this case, an overflow *does not necessarily* mean an error. It is only an error if your calculations are *signed*. If your calculations are *unsigned* then the result is correct and there is no problem.

In conclusion, overflow arises when two operands of the *same* sign are added, and the result is a number of a *different* sign ($+n + +m = -s$ or $-n + -m = +s$)

If you are not interested in the operand signs, then the overflow event does not matter but carry-out certainly do matter.

Now that we understand how the **NZCV** gets updated, and what it can be used for (ref. Table 4.1), we now look at the ARM7DTI approach for implementing the humble **if**, by using the **cmp** instruction.

4.2.4 Compare and test

To implement **if** we use the **cmp** instruction followed by a test appended to the instruction that follows the **cmp** (see Table 4.1). **cmp** has two operands, the left and the right. It subtracts the right operand from the left operand and updates **NZCV** with the outcome of the operation. The actual numerical result is not stored anywhere.

In **cmp**, both the left and right operands can be registers (this is usually the way), or the left operand can be a register and the right operand an immediate. For example:

```
mov r1, #10
mov r2, #5
mov r3, #20
cmp r3, #20 // NZCV = 0110
cmp r2, r1  // NZCV = 1000
cmp r1, r2  // NZCV = 0010
```

Note that each of the above 3 **cmp** Instructions will update the **NZCV** flags. This means that the result will be overwritten and lost. Therefore, if we use **cmp**, we never follow it immediately with another **cmp**. We first must *examine* the **NZCV** and make a decision. Lets look at an example in C:

```
int i = 10;
int r = 5;

if (i > r) {
    r = i;
}
```

We can say that the **r = i** is a conditional operation - it should only happen if **i > r**. Let's do this in assembly:

```
mov r1, #10
mov r2, #5
cmp r1, r2
movgt r2, r1
```

Note the new version of **mov** here: **movgt** which means *move if greater than*. If what is greater than what? if **r1** is greater than **r2**. How is this determined? by subtracting **r2** from **r1** and checking the **NZCV = 0000** (not negative and not zero).

The full list of conditionals is shown in Table 4.1

Almost all ARM7DTI instructions, with the exception of **cmp**, can optionally have *one* conditional code appended. The conditional code is omitted

Condition	Meaning	Flags Tested
eq	Equal (zero)	$Z = 1$
ne	Not equal (non-zero)	$Z = 0$
cs/hs	Carry set / Unsigned higher or same	$C = 1$
cc/lo	Carry clear / Unsigned lower	$C = 0$
mi	Negative (minus)	$N = 1$
pl	Positive or zero (plus)	$N = 0$
vs	Overflow set	$V = 1$
vc	Overflow clear	$V = 0$
hi	Unsigned higher	$C = 1$ and $Z = 0$
ls	Unsigned lower or same	$C = 0$ or $Z = 1$
ge	Signed greater than or equal	$N = V$
lt	Signed less than	$N \neq V$
gt	Signed greater than	$Z = 0$ and $N = V$
le	Signed less than or equal	$Z = 1$ or $N \neq V$

Table 4.1: Full set of ARM7DTI conditional codes

then the instruction *always* executes.

For example:

```
addlt r2, r4 // add r4 to r2 NVCV = 1000
subeq r2, r4 // add r4 to r2 NVCV = 0110
mulhi r2, r3, r4 // multiply r3 by r4 if NVCV = 0010
```

4.2.5 Signed and Unsigned Conditions

To make the following examples easier to understand we will use a simple 3-bit computer. The ideas here apply equally to a computer of word size of any integer.

Let us think through the example of a signed *greater than* test. When we say *signed*, what this means is that *if* the number has the **MSB=1** it must be treated as a negative number. We will later compare this to an unsigned test using the same bit pattern.

Let's look at the first example, using **gt** (for example, **movgt**). As you can see, **gt** is a signed greater than test, and for it to be true the **NZCV** flags must be: **Z=0** and **N=V**. The **Z=0** requirement is obvious. But less obvious is **N=V**. Let's look at a 3-bit example: **r0=-1** and **r1=2**.

When the instruction **cmp r0, r1** runs, we have $-1 + -2 = -3$.

$$\begin{array}{r}
 111 \\
 +110 \\
 \hline
 101
 \end{array}$$

The result is correct, $101 = -3$. But do the flags agree with requirement for **gt**?

- $Z=0$ true
- $N=V$ false (negative but no overflow)

No - there is a disagreement, because the result is negative without an overflow. Therefore, $r0 < r1$ and the condition is false.

Therefore, we would expect the instruction `movgt r4, #5` to *not* execute.

How about reversing the test? `cmp r1, r0`. When it runs we have $2 - -1 = 3$.

$$\begin{array}{r} 001 \\ +010 \\ \hline 011 \end{array}$$

The result is correct, $011 = 3$. But do the flags agree with requirement for `gt`?

- $Z=0$ true
- $N=V$ true (no negative, no overflow)

Yes - there is agreement, as the result is not negative and not overflow. Therefore, $r1 > r0$ and the condition is true. Therefore, we would expect the instruction `movgt r4, #5` to execute.

Just for clarity, let us think again about the previous example from an *unsigned* point of view. Again, $r0=7$ and $r1=2$, let's see what happens when we compare them and test if $r0$ is `gt r1` (it is). This becomes $7 - 2 = 5$, or

$$\begin{array}{r} 111 \\ +110 \\ \hline 1 \xleftarrow{\text{carry out}} 101 \end{array}$$

Does this agree with the definition for unsigned greater than (`hi`)?

- $Z=0$ true
- $Z=0$ true (no overflow)

Yes, $7 > 2$. Remember that although for us, we think of these two numbers as unsigned, when `cmp r0, r1` runs, it multiplies the second operand by -1 and then adds it to the first operand. In unsigned arithmetic this always produces a negative second operand.

You can see the contradiction here: when `cmp r0, r1` is followed by `gt`, the result was false (-1 is not greater than 2), but when we use `hi` we find the result is true! ($7 > 2$).

If our algorithm uses only unsigned arithmetic, then we should use `hi` and `ls` for our greater than and less than or equal to. Conversely, if we are using signed arithmetic, then use `le` and `gt` respectively.

4.2.6 Updating NZCV

`cmp` *always* updates the `cpsr` (NZCV bits) with the outcome of the instruction. But there are other instructions that can produce Zero, Negative, Carry and Overflow outcomes. For example, a calculation such $m - n > 0$ always results in a carry (`C=1`). How do we update NZCV ourselves? It can only be done *indirectly* - by appending `s` to the instruction in question. For example:

```
mov r0, #5
mov r1, #3
subs r0, r1 // results in C=1
sub r0, r1  // does not change NZCV
```

Notice in the example above, `subs r0, r1` should be understood as a subtraction instruction that updates NZCV with the outcome of `r0` minus `r1` (from the ALU). Whereas `sub r0, r1` performs no such update. Therefore, when you want to make a decision about the outcome of an operation, append `s` to the instruction in question.

4.2.7 Combining condition codes and NZCV updates

Is it possible to combine both condition codes and NZCV updates in one instruction? Yes. For example: `addlts r0, r1` will update the NZCV with the outcome of the `addlt` so long as the *previous* instruction resulted in a signed `lt` outcome (`Z = 0` and `N = V`).

4.2.8 Branching

Now that we have seen how `if` is implemented in assembly as the pair `cmp` + `instrcond`, we should return to scope control. Let's look at an example: where 5 instructions should execute if the `lt` condition is true:

```
mov r0, #-1
mov r1, #2
mov r3, #0
cmp r0, r1
// the next 5 instructions
// only execute if lt is true
movlt r4, #5
movlt r5, #10
mullt r6, r5, r4
sublt r3, r6
movlt r5, r6
```

The above structure does not resemble how we write `if` statements in C. What we would like to do is to “jump” to the a block of code if the condition is true, and if it's false, then it should continue executing from the condition. This is the important role that *branching* plays. We will discuss branching in Chapter 6.

4.3 Chapter Summary

In this chapter we have introduced the ARM7DTI equivalent of the C program `if` keyword. We noted that `if` in ARM7DTI is always at least *two* instructions: the `cmp` instruction followed by one or more instructions with the condition code appended (see Table 4.1).

In this chapter we discussed several examples of how signed and unsigned conditions are handled. We noted that in some cases, signed and unsigned tests of the same condition type (eg, `gt` and `hi`) produce different results. Remember that although the bit patterns in memory are the same, the results in the both `gt` and `hi` are tested against different parts of the NZCV. We also looked at how the programmer can indirectly update NZCV (by appending `s` to the instruction), and we also briefly noted that condition codes and NZCV updates can be combined into one instruction such as `addlts r0, r1`.

4.4 Exercises Summary

Exercise 4.4.1 *Using three few different examples test to see if a carry-out always arises whenever we have a positive and a negative addition where the result is positive. Does it hold true?*

Exercise 4.4.2 *Derive an ARM7DTI assembly example where NZCV = 0011 (Positive result, with carry-out and overflow). Test it in CPUlator.*

Exercise 4.4.3 *Derive an ARM7DTI assembly example where NZCV = 1010 (Negative result, with carry-out, no overflow). Test it in CPUlator.*

Exercise 4.4.4 *Give informal examplea from the real world, where (a) an overflow would be a concern, and (b) where an overflow would not be a concern.*

Exercise 4.4.5 *Write and ARM7DTI program to demonstrate how `lt` and `lo` conditional codes produce different results for the same bit pattern.*

Branching and Iteration

5.1 Introduction to Program Control Flow

In previous chapters, we have examined how to load and store values to and from memory. However, real programs do not simply execute instructions in a straight line; they must make decisions and repeat operations. In higher-level languages, this is accomplished with `if` statements, `while` loops, `for` loops, and function calls. In ARM Assembly, we must implement these control flow mechanisms using *branch* instructions.

5.1.1 Control Flow in Assembly

At the heart of control flow in ARM Assembly are instructions that affect the Program Counter (`pc`). The `pc` determines which instruction the CPU executes next. Most instructions implicitly increment `pc` to the next instruction, but branch instructions explicitly update it to a new target.

Control Flow Instructions in ARM Assembly:

- `b label` – Unconditional branch to `label`.
- `bl label` – Branch with link; jumps to a function and stores return address in the Link Register (`lr`).
- `bx lr` – Return from function by branching to the address in `lr`.
- `bne`, `beq`, `bgt`, etc. – Conditional branches that execute based on status flags.

5.1.2 The Role of the Program Counter (`pc`)

In ARM32, due to the fetch-decode-execute pipeline, reading from the `pc` actually gives the address of the instruction *two steps ahead*, i.e., `pc + 8`. In ARM64, this is simplified to `pc + 4`. This offset is important when performing `pc`-relative calculations such as loading data from memory based on the current program position.

5.1.3 Branch Instruction Encoding and Range

The `b` and `bl` instructions use relative addressing with a signed immediate value. In ARM32:

- The offset is 24 bits wide and shifted left by 2, allowing for a range of $\pm 2^{25}$ bytes, or $\pm 32\text{MB}$.

In ARM64:

- The offset is 26 bits wide, also shifted left by 2, resulting in a branch range of $\pm 128\text{MB}$.

If a branch target is outside this range, the address must be loaded into a register, and an indirect branch performed:

```
ldr x0, =target_address
br x0 // ARM64 indirect branch
```

5.1.4 Branching and the Pipeline

Modern ARM cores use deep pipelines to maximize throughput. However, branches can disrupt the pipeline:

- If a branch is correctly predicted, the pipeline continues smoothly.
- If mispredicted, the processor must flush instructions and refetch from the correct path, causing a delay.

Core	Branch Misprediction Penalty
Cortex-A53	~10 cycles
Cortex-A76	~15 cycles
Neoverse V1	20+ cycles

Table 5.1: Misprediction penalties in common ARM cores

Why the penalty grows with pipeline depth

These cores have 8- to 15-stage pipelines, so more instructions are fetched on the wrong path before the mistake is detected. The pedagogical 5-stage pipeline in Chapter 11 has a 2-cycle penalty because the branch resolves in the Execute stage with only two instructions fetched speculatively behind it. The hardware mechanism — flushing the Decode and Execute pipeline registers — is identical; only the depth (and therefore the cost) differs.

► What is PlayARM?

PlayARM is a browser-based ARM microarchitecture simulator. Write ARM assembly in the built-in editor, then step through it cycle by cycle — watching the 5-stage pipeline canvas, register file, CPU flags, control signals, memory accesses, and TLB translations update in real time. Throughout this book, purple **Try it in PlayARM** boxes point you to programs you can run directly in the simulator to reinforce each concept.

► Watch a Branch Flush the Pipeline

Run the program below in PlayARM. Use **Slow** speed and watch the **Pipeline Activity** panel when the **BNE** instruction is in the **EX** stage. If the branch is *taken*, notice that the instructions fetched after it are flushed — the pipeline must restart from the branch target.

```
mov r0, #3
loop:
    sub r0, r0, #1
    cmp r0, #0
    bne loop
mov r1, #99
```

Count how many extra cycles the branch adds each time it is taken compared to when it falls through on the final iteration.

5.1.5 Why Control Flow Matters

Control flow constructs (branches, loops, and function calls) enable the creation of algorithms, decision-making structures, and code reuse. Efficient use of branches is critical in performance-sensitive applications, such as signal processing or real-time embedded systems.

In the sections that follow, we will explore how these ideas translate into practical assembly code, including:

- Creating branches based on comparisons.
- Writing loops with different structures (while, do-while, for).
- Minimizing the performance cost of branching using techniques such as conditional execution and loop unrolling.

5.2 ARM Branch Instruction Set

In ARM Assembly, branching is the fundamental method of controlling program flow. Branch instructions modify the Program Counter (**pc**), allowing a jump to another instruction in the program. These jumps may be conditional or unconditional, direct or indirect, and are essential for implementing loops, conditionals, and function calls.

5.2.1 Core Branch Instructions

The most basic branching operations in ARM Assembly are:

- **b label** – Unconditional branch to **label**.
- **bl label** – Branch to **label** and save return address in the Link Register (**lr**).
- **bx lr** – Return from subroutine (branch to address in **lr**).
- **blx reg** – Branch with link to address in **reg**, also supports interworking between ARM and Thumb states.

```
bl my_function    // Call function
...
my_function:
    ; function body
    bx lr         // Return
```

The ‘bl’ instruction automatically stores the address of the next instruction in `lr`, making it possible to return using `bx lr`.

5.2.2 ARM64 Enhancements

Modern ARM64 introduces new branching capabilities that are both compact and powerful:

- `b.cond label` – Compact conditional branch (e.g., `b.eq`, `b.ne`, `b.ge`).
- `br xN` – Branch to address in register `xN`.
- `ret xN` – Return from subroutine (equivalent to `bx lr`).
- `braa xN, xM` – Branch with return address authentication (used for security).

Example – ARM64 return mechanism:

```
bl some_function
...
some_function:
    ; do something
    ret         // return to caller
```

These enhancements are particularly relevant in performance- and security-critical applications, such as operating system kernels and cryptographic routines.

ARMv8.3 also introduces **Pointer Authentication Codes (PAC)**, introducing the use of `braa` for secure, authenticated control transfers.

we need to develop this braa point - why do we need it? what problem does it address does it need its own section?

5.3 Conditional Execution

Modern ARM processors support **conditional execution**, a powerful feature that allows certain instructions to execute only if specific conditions (based on the status flags) are met. This eliminates the need for short branches in many cases and helps to reduce pipeline disruptions.

5.3.1 Condition Flags Overview

ARM architecture maintains a special register called the **Application Program Status Register (APSR)**. This register stores condition flags that reflect the result of the most recent arithmetic or logical operation.

- **N (Negative)** – Set if the result is negative (bit 31 of result is 1).

- **Z (Zero)** – Set if the result is zero.
- **C (Carry)** – Set if there was a carry out (for unsigned arithmetic).
- **V (Overflow)** – Set if there was a signed overflow.

Example – Flag update by comparison:

```
cmp r0, r1 // Updates N, Z, C, and V
bge next  // Branch if r0 >= r1 (N == V)
```

These flags are implicitly updated by arithmetic instructions such as **add**, **sub**, **cmp**, and **movs**. Conditional branches and conditional instruction execution depend on the values of these flags.

► Watch the N, Z, C, V Flags Update

Enter the program below and step through it in PlayARM. After each **cmp** or **sub**, check the **CPU Flags** panel and note which of the **N, Z, C, V** flags are set.

```
mov R0, #5
mov r1, #5
cmp R0, r1
mov r2, #10
mov R3, #3
sub R4, R3, r2
cmp R4, #0
```

After **cmp R0, r1**: **Z** should be 1 (equal).

After **sub R4, R3, r2**: **N** should be 1 (negative result).

Verify each flag matches the condition table above.

5.3.2 Using Flags in Practice

ARM supports a full set of conditional suffixes that can be applied to many instructions, allowing them to execute only when a condition is true.

Common condition suffixes:

Suffix	Meaning	Condition
eq	Equal	$Z = 1$
ne	Not equal	$Z = 0$
lt	Less than (signed)	$N \neq V$
ge	Greater than or equal (signed)	$N = V$
cs	Carry set (unsigned $\geq$)	$C = 1$
cc	Carry clear (unsigned $<$)	$C = 0$

Table 5.2: ARM Condition Suffixes

Example – Using conditional arithmetic:

```
cmp r0, r1
addgt r2, r0, r1 // if r0 > r1
sublt r2, r1, r0 // if r0 < r1
```

This technique is particularly useful when minimizing branch penalties. Rather than using `bgt` or `blt` to jump around the code, the instructions themselves are conditionally suppressed or executed.

5.4 Loop Constructs and Patterns

Looping structures are essential for repeating instructions in programs. In C, loops are expressed as `while`, `for`, and `do-while` constructs. ARM Assembly does not have these keywords, so all loops must be built explicitly using branches and comparison instructions.

This section explains how to implement these high-level loop constructs using ARM's conditional branches and registers.

5.4.1 While Loop in Assembly

A `while` loop in C repeatedly checks a condition before executing the loop body.

C code:

```
[language=C]
int i = 10;
while (i > 0) {
    // loop body
    i--;
}
```

ARM Assembly:

```
mov r1, #10          // i = 10
b test_condition     // jump to condition check

loop_body:
    ; your loop logic here
    subs r1, r1, #1   // i--

test_condition:
    cmp r1, #0
    bgt loop_body     // loop if i > 0
```

Note: 'subs' subtracts and updates flags; 'cmp' checks the loop condition.

5.4.2 For Loop Using `subs`

A `for` loop has explicit initialization, condition, and iteration components.

C code:

```
[language=C]
for (int i = 5; i > 0; i--) {
    // loop body
}
```

ARM Assembly:

```

mov r1, #5           // i = 5

loop:
    ; loop body here

    subs r1, r1, #1    // i--
    bgt loop           // loop if i > 0

```

This form is compact and efficient. It uses ‘subs’ to both decrement the counter and update the flags for ‘bgt’.

► For Loop: Count Down from 5

Translate the **for** loop above directly into PlayARM. Use **Step** mode and watch **r1** count down from 5 to 0 in the **Register File** panel. Observe that the **Z** flag goes high on the final iteration (when **r1** hits 0), causing **BNE** to fall through instead of branching.

```

mov r1, #5
loop:
    sub r1, r1, #1
    cmp r1, #0
    bne loop
mov r2, #1

```

After `mov r2, #1` executes, **r1** should be 0 and **r2** should be 1 — confirm both in the **Register File**.

5.4.3 Do-While Loop

The **do-while** loop guarantees that the body executes at least once.

C code:

```

[language=C]
int i = 5;
do {
    // loop body
    i--;
} while (i > 0);

```

ARM Assembly:

```

mov r1, #5

loop:
    ; loop body here
    subs r1, r1, #1
    bgt loop           // repeat if i > 0

```

Unlike the previous loops, this structure places the condition check *after* the body, ensuring at least one execution.

5.4.4 Nested Loops and Flowcharts

Nested loops involve placing one loop inside another. These are common in array or matrix processing.

C code:

```
[language=C]
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 2; j++) {
        // inner loop body
    }
}
```

ARM Assembly:

```
mov r0, #0          // i = 0
outer_loop:
    cmp r0, #3
    bge end_outer

    mov r1, #0        // j = 0
    inner_loop:
        cmp r1, #2
        bge end_inner

        ; inner loop body here

        add r1, r1, #1
        b inner_loop

    end_inner:
        add r0, r0, #1
        b outer_loop

end_outer:
```

Each loop must use separate registers for indexing ('r0', 'r1') and must have clearly marked start and end labels.

Visualizing Control Flow

To understand nested loop structure better, we can sketch a flowchart:

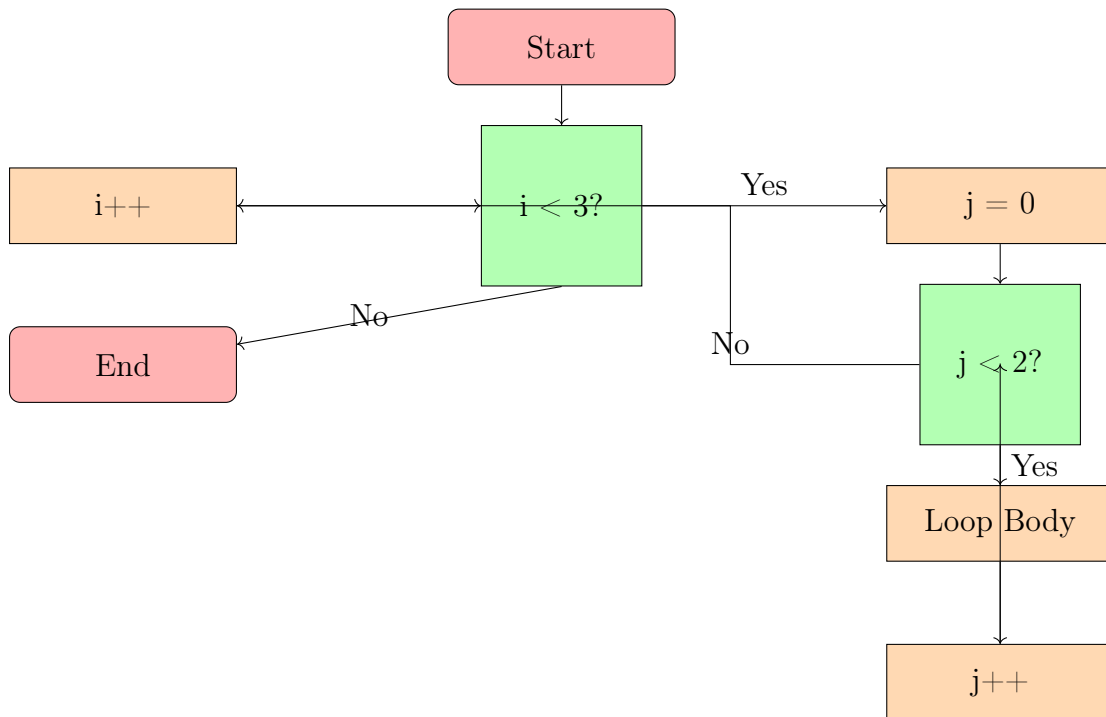


Figure 5.1: Flowchart of Nested Loops

5.5 Exercises

5.5.1 Basic Load and Store Operations

Exercise 5.5.1 Write an ARM assembly program that does the following:

1. Declares an integer variable `x` initialized to 25.
2. Loads `x` into a register.
3. Adds 10 to `x`.
4. Stores the result back in memory.

Use `ldr` and `str` instructions to perform memory operations.

5.5.2 Byte vs Word Access

Exercise 5.5.2 Given the following memory declaration:

```
.data
array: .word 0x12345678
```

Perform the following operations in ARM assembly:

1. Load the full word into a register and print its value.
2. Load only the least significant byte using `ldrb` and print its value.
3. Store `0xAB` into the least significant byte of `array` without modifying other bytes.

5.5.3 Offset Addressing

Exercise 5.5.3 *Given an integer array in memory:*

```
.data
numbers: .word 5, 10, 15, 20
```

Write an ARM assembly program that:

1. *Loads the second element of the array into a register.*
2. *Adds 5 to the second element.*
3. *Stores the modified value back into the array.*

Use offset addressing mode in the `ldr` and `str` instructions.

5.5.4 Pointer Arithmetic

Exercise 5.5.4 *Convert the following C code into ARM assembly:*

```
unsigned values[5] = { 2, 4, 6, 8, 10 };
unsigned *p = values;
*(p + 2) = *(p + 2) + 5; // Modify the third element
```

1. *Identify how memory addressing is done in both C and ARM assembly.*
2. *Use a base register and appropriate addressing mode to modify the third element.*

5.5.5 Pointer Traversal in Assembly

Exercise 5.5.5 *Write an ARM assembly program that:*

1. *Defines an array of 6 integers.*
2. *Uses a base register (pointer) to iterate through each element.*
3. *Doubles each value in the array.*

5.5.6 Finding Maximum in an Array

Exercise 5.5.6 *Write a C function that finds the maximum value in an array using pointers:*

```
unsigned find_max(unsigned *arr, int size) {
    unsigned max = *arr;
    for (int i = 1; i < size; i++) {
        if (*(arr + i) > max) {
            max = *(arr + i);
        }
    }
    return max;
}
```

1. *Translate this function into ARM assembly.*

2. *Use a loop with indexed addressing mode to traverse the array.*

Addressing Memory

6.1 Load and Store

The ARM32/64 Microarchitecture uses a *load and store* memory access model. This means that before an item from memory can be used in the CPU, the *address* of the item must first be *loaded* into a *base register*. To write (*store*) a value back to memory, we must use the address contained in the base register.

This means obtaining the address of the first element of the variable you wish to access, placing this address in a base register, and computing the access locations of subsequent elements, as an offset of the base register.

Later we will discuss the ARM instructions required for this, but first we will consider a C program as the basis for understanding how memory operations take place.

6.1.1 What to know when addressing memory

We have to plan carefully in order to address memory correctly. Our approach needs to be aware of the following factors:

1. *Data Type*: There are two possible choices here. Addressing 32-bit words or addressing 8-bit bytes
 - (a) For accessing signed or unsigned integer data we read from memory using **ldr** (LoaD Register) and write to memory using **str** (StoRe Register)
 - (b) For accessing ASCII character data we read from memory using **ldrb** (LoaD Register Byte) and write to memory using **strb** (StoRe Register Byte)
2. *Addressing modes*:
 - (a) Offset addressing
 - (b) Post-index Addressing
 - (c) Pre-index Addressing

Offset, Post- and Pre- index Addressing will be discussed in Section 6.4, but first we will discuss the general syntax of the memory instructions.

6.1.2 Instruction Syntax

Let us briefly examine the syntax of the memory access instructions:

Instruction	Meaning
<code>ldr ToReg, [FromAddr]</code>	Load the integer (word) found at address FromAddr into the register ToReg
<code>str FromReg, [ToAddr]</code>	Store the integer found in the register FromReg into the value at address ToAddr
<code>ldrb ToReg, [FromAddr]</code>	Load the byte from address FromAddr into the register ToReg
<code>strb FromReg, [ToAddr]</code>	Store the byte found in the register FromReg into the value at address ToAddr

Figure 6.1: Memory Instructions

Let's consider an example from Table 6.1, we will use the load register (`ldr`)

```
ldr r1, [r0]
```

This should be understood as *read the value from the memory location pointed to by `r0`, and place it into `r1`.*

6.2 C Program

We will use the C program shown in Fig. 6.2 to motivate our understanding of how this load and store model works.

1. Place the code into a file called `memory.C`
2. Compile it using: `gcc memory.C -o memory`
3. Run it: `./memory`

6.2.1 C Pointers

In the C code, we declared and initialized a *pointer* as `unsigned* p = array`. This means that `p` points to the address of the first element in `array`. We can now access any element of `array` using the notation `*p`. For example, the code:

```
p += 3; // move the pointer to the 3rd index element
```

will move `p` to index 3 in the array of integers, thus it is now pointing to the element that contains the value 4. Notice that `p += 3` is shorthand for `p += (3*w)` where `w` is the word size of the hardware.

This value can be modified by using the pointer-access notation as follows:

```
*p += 10; // add 10 to the value at the 3rd index
```

```

#include <stdio.h>

const int sz = 6;
unsigned array[ sz ] = { 9, 2, 3, 4, 8, 10 };
unsigned someValue = 55;

int main (int argc, char** argv) {
    unsigned* p = array;

    // update the 3rd element of the array
    p += 3;
    *p += 10;
    printf("the 3rd element is: %u \n", *p);

    // use p to access / modify the someValue;
    printf("someValue is %u \n", someValue);
    p = &someValue;
    *p = 123;
    printf("now someValue is %u\n", someValue);

    return 0;
}

```

Figure 6.2: Sample C program using pointers

This will change the value to which `p` points, from 4 to 11.

6.3 ARM Assembly “Pointers”

In ARM Assembly, we see that C pointers have a direct counterpart. This is achieved by selecting some register as the *base-register*. The base-register is initialized to the name of the `.data` object, as we show later. This base register is the direct equivalent of our C program pointer.

To begin, assume that the ARM Assembly `.data` section contains the following code:

```

.data
array: .word 9, 2, 3, 4, 8, 1
someValue: .word 55

```

When this array is placed into memory at program load time, the elements will be stored in *contiguous* memory, with each element occupying 4 bytes (a `.word`) as shown in Fig. 6.3.

6.3.1 Initializing the base-register

We first load the address of the array using the following syntax (we will give `r0` the role of base register), using the instruction `ldr`. For clarity, we also show the C code equivalent to the assembly instructions:

Address	Value
...	...
...	...
00 FF AB 00	00 00 00 09
00 FF AB 04	00 00 00 02
00 FF AB 08	00 00 00 03
00 FF AB 0C	00 00 00 04
00 FF AB 10	00 00 00 08
00 FF AB 14	00 00 00 0A
...	...
...	...

Figure 6.3: `array` in memory starting at address `00FFAB00`

```
// In C: unsigned* p = array;

// In ARM assembly:
ldr r0, =array
```

Note the use of the special character `=`. This character is used for *initializing* the base-register. This initialization does not need to be repeated (in most cases). As `array` starts at the memory location `00FFAB00`, `r0` now contains the value `00FFAB00`. We say that `r0` is *pointing to* the first element in the array:

$$\begin{array}{c}
 r0 \\
 \downarrow \\
 \begin{array}{|c|c|c|c|c|c|}
 \hline
 9 & 2 & 3 & 4 & 8 & 1 \\
 \hline
 \end{array} \\
 \begin{array}{cccccc}
 0 & 1 & 2 & 3 & 4 & 5
 \end{array}
 \end{array} \tag{6.1}$$

6.3.2 Pointing to the i -th element

Now we consider how to change the base-register so that it points to some arbitrary location (the i -th element).

```
// In C: p += 3;

// In ARM assembly 3 x 4 = 12:
add r0, #12
```

$$\begin{array}{c}
 r0 \\
 \downarrow \\
 \begin{array}{|c|c|c|c|c|c|}
 \hline
 9 & 2 & 3 & 4 & 8 & 1 \\
 \hline
 \end{array} \\
 \begin{array}{cccccc}
 0 & 1 & 2 & 3 & 4 & 5
 \end{array}
 \end{array} \tag{6.2}$$

Notice that to change the base register so that it points to the i -th element (in this example $i = 3$), we must add 12 to it. This is because we need to calculate how many *bytes* to move `r0` on by. Because the word size of our platform is 32-bit, and there are 4 bytes in 32 bits, we multiply our new index

Mode	Syntax	Address	Base-register <code>r0</code>
Offset	<code>ldr r1, [r0, V]</code>	<code>r0+V</code>	Unchanged
Pre-Index	<code>ldr r1, [r0, V]!</code>	<code>r0+V</code>	Changes <i>before</i> the location is read
Post-index	<code>ldr r1, [r0], V</code>	<code>r0</code>	Changes <i>after</i> the location is read

Figure 6.4: Addressing modes with using `ldr` instruction and the base register `r0`. Note `V` is either (a) a register, (for example, `ldr r1, [r0, r2]`), or (b) an immediate (for example, `ldr r1, [r0, #4]`)

(in the example it is 3) by 4 to get the next index position address when accessing integer arrays. This is in contrast to C, where we need to add 3.

There are other ways to achieve base register pointer changes. For example:

```
// In ARM assembly 3 x 4 = 12:
mov r1, #4
mov r2, #3
mul r3, r1, r2
add r0, r3
```

In C, when the code is compiled, the addition is 12 too, but this detail handled by the compiler, not the programmer.

6.4 Integer Addressing Modes

ARM Assembly has three different modes for accessing memory. You choose the correct mode depending on the situation. It is possible to mix and match them in a program, but this needs to be done with care so that you don't lose track of where the base-register is pointing to. These modes are now summarized in Table 6.4.

6.4.1 Offset Addressing

Use this mode when we do not want to change our base-register. For example, in situations where we want to flexibly access any location in the memory object by the addition of some constant.

Suppose our algorithm requires us to set certain elements in our array in a particular order. Let's say the third, first and fourth elements in `array` should be set with some number in `r2`

```
// using offset addressing
ldr r0, =array
mov r2, #0
str r2, [r0, #8]
str r2, [r0, #0]
str r2, [r0, #12]
```

In this sense we can “jump around” our array in any kind of random sequence, only knowing which element we need to access. Because the base-register does not change, there is no need to reset it after each “jump”.

► Offset Addressing: Random Access

The program below mimics the random-access pattern above using only registers and the stack as a stand-in for an array. Write three values to non-sequential offsets from **SP**, then read them back.

```
mov r2, #0
sub sp, sp, #16
str r2, [sp, #8]
str r2, [sp, #0]
str r2, [sp, #12]
ldr r3, [sp, #8]
ldr r4, [sp, #0]
ldr r5, [sp, #12]
add sp, sp, #16
```

Step through in PlayARM and watch the **Memory** panel after each **str**. Notice that **SP** never changes during the stores: that is the defining property of offset addressing. Check the **TLB** panel to see the virtual-to-physical address translation for each memory access.

On the other hand, a plain iteration over **array** is troublesome using offset addressing. Consider:

```
// using offset addressing to iterate
// to zero out the array is a bit troublesome
ldr r0, =array
mov r1, #0 // incrementer
mov r2, #0 // value to write
mov r3, #6 // array size

loop:
    cmp r1, r3
    strlt r2, [r0, r1, lsl #2]
    addlt r1, r1, #1
    blt loop
.data
array: .word 1,2,3,4,5,6,7
```

Here, we use **str r2, [r0, r1, lsl #2]**, which has the effect of shifting left (multiplying by 4), the incrementer in **r1** during each iteration (which generates the sequence, 0, 4, 8, ...). Obviously, this is extra work for the programmer to manage, so for plain 0, 1, 2, ..., $n - 1$ iteration tasks, we can use either pre-indexing or post-indexing. Let's consider post-index addressing first.

6.4.2 Post-index Addressing

As mentioned above, offset addressing is not a good choice for a simple iteration over each element of an array (for example, in linear search, or selection sort).

A much simpler approach, and the default approach used in C, is to incre-

ment the address stored in the base-register by some constant amount. In C, the notation `*ptr++` is used to dereference the memory address pointed to be `p` and then move `p` to the next element.

```
// in C, dereference the pointer
// and then move it on
unsigned r = *ptr++;
```

Assuming `ptr` contains `00FFAB00`, then the unsigned integer `r` will be assigned 9, and *immediately after*, `ptr` will be incremented to `00FFAB04`.

Of course, `*ptr++` is just shorthand for:

```
unsigned r = *ptr;
ptr++;
```

This is called *post-index* addressing (or post-fix addressing in C). This should be our default method when iterating over an array in sequential order. In ARM assembly, we implement post-index addressing as follows:

```
// using post-index addressing to iterate sequentially
// over array and zero out each element
ldr r0, =array
mov r2, #0 // value to write
mov r3, #6 // array size

loop:
    cmp r1, r3
    strlt r2, [r0], #4 // much neater syntax
    blt loop
```

Notice the simplification of the loop structure: we have a much neater syntax for our `str` instruction:

```
// much neater syntax for iteration
strlt r2, [r0], #4
```

versus

```
// more awkward syntax for iteration
strlt r2, [r0, r1, lsl #2]
addlt r1, r1, #1
```

Eliminating the incrementer (r1) and one instruction from the loop (addlt r1, r1, #1)

Post-Index Patterns

We now review some of the most common C programming post-index memory access patterns and their equivalent in ARM assembly.

Pattern 6.4.1 Initialize-Increment

In C we often see the following code pattern:

```
unsigned r = *ptr++;
```

You should understand that this code has **two** steps. Step 1., store the value found at the address pointed to by **ptr**, into the variable *r*. Step 2. **then** increment **ptr** to point to the next element in the array.

In ARM Assembly we have the exact same semantics:

```
ldr r2, [r0], #4
```

Step 1., load the value found at the address in *r0* into the register *r2*, and 2., **then** add *texttt4* to the value in *r0*.

Pattern 6.4.2 Assign-Increment

Another commonly encountered pattern is the assign increment pattern, for example:

```
*ptr++ = r;
```

becomes:

```
str r2, [r0], #4
```

Pattern 6.4.3 Read-Write-Increment

The following pattern arises where, in one statement, the value from a pointer is read, modified then written. Following this, the pointer is incremented. As follows:

```
*ptr++ += 3;
```

This pattern must be implemented as **three** instructions:

```
ldr r2, [r0] // load using offset addressing
add r2, r2, #3 // add our 3 as an immediate
str r2, [r0], #4 // write results, increment pointer
```

Of course, if you are writing a *C* program for the ARM platform, the compiler will convert `*ptr++ += 3;` into the three ARM instructions as shown above.

► Post-Index: Watch the Base Register Advance

Post-index addressing is the natural way to iterate sequentially. Run this program and watch `r0` increment by 4 after each `str` in the **Register File** panel.

```
sub sp, sp, #16
mov r0, sp
mov r2, #10
str r2, [r0], #4
mov r2, #20
str r2, [r0], #4
mov r2, #30
str r2, [r0], #4
mov r2, #40
str r2, [r0], #4
add sp, sp, #16
```

After each `str`, `r0` moves to the next slot *without* a separate `add` instruction. Check the **Memory** panel to confirm all four values (10, 20, 30, 40) are stored at consecutive addresses. Also watch the **TLB** panel: each access hits a different virtual page offset.

6.4.3 Pre-index Addressing

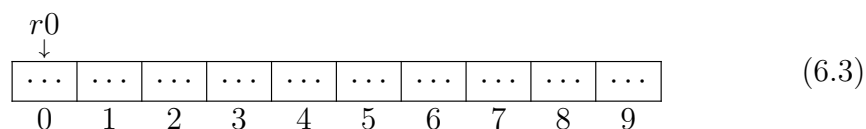
Post-index addressing has the interesting effect of leaving the base-register pointing to the next element in the array. This works well in cases where the next element in the array is *initialized*. However, it does not work well in cases when the next element is not initialized.

Consider a 10-element array of *uninitialized* ($\dots$) data:

```
ldr r0, =array

// the .bss section for uninitialized data
.section .bss
array: .skip 40
```

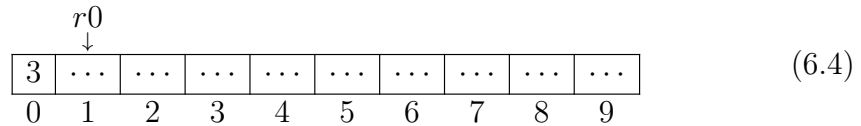
In the above example, 40-bytes have been reserved in the `.bss` segment. As you may remember from Chapter 2, the `.bss` segment is use for the placement of uninitialized data.



In Fig. 6.3, the base-register is pointing to the first uninitialized element. We can store a value into the first element using post-index addressing:

```
mov r1, #3
str r1, [r0], #4
```

`r0` now points to index 1, but this value is not initialized.



So, what happens here?

```
ldr r1, [r0]
```

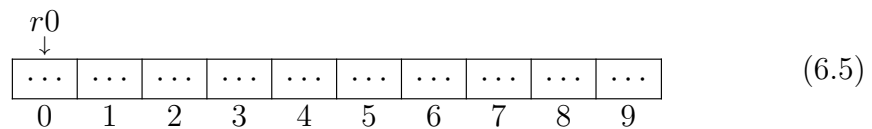
It is obvious that `ldr r1, [r0]` leaves `r1` with garbage. This is not desirable. The base-register is pointing to uninitialized memory. Hence, the post-index addressing into uninitialized produces undesirable results. Of course, we could easily solve this using a work-around:

```
ldr r1, [r0, #-4]
```

But this approach is awkward. We need another approach. Let's reset the base-register `r0`:

```
ldr r0, =array
```

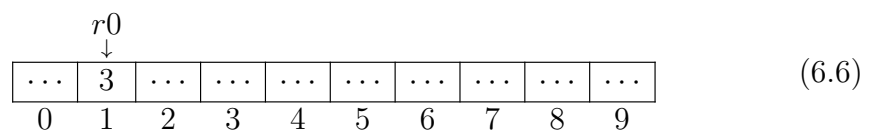
so now things look like this:



Now use pre-index as follows:

```
mov r1, #3
str r1, [r0, #4]!
```

and the array now looks like:



Now a `ldr` from the base-register will return a valid value:

```
ldr r1, [r0] //places 3 into r1
```

Because the base-register was moved *before* the write, the subsequent read is guaranteed to be initialized. Of course, the effect here is that the first element is skipped!

But the good news is that `ldr r1, [r0]` produces a properly initialized result every time. In fact, every pair of:

```
str r1, [r0, #4]!
ldr r1, [r0]
```

produces a *guaranteed* initialized value in `r1`, whereas, with

```
str r1, [r0], #4
ldr r1, [r0]
```

no such guarantee exists. In what circumstances might we want to use this pre-index addressing pattern? Use pre-index addressing when we want to move the base-register, write to an element and read that element back.

► Pre-Index: Write Then Read Back Safely

Pre-index addressing moves the base register *before* the memory access, guaranteeing the subsequent read returns an initialized value. Run the program below and compare what happens with pre-index vs post-index for the same slot.

```
sub sp, sp, #16
mov r0, sp
mov r1, #42
str r1, [r0, #4]!
ldr r2, [r0]
mov r1, #99
str r1, [r0, #4]!
ldr r3, [r0]
add sp, sp, #16
```

Step through in PlayARM and watch **r0** advance *before* each store. After each LDR, R2 and R3 should hold 42 and 99 respectively: never garbage. Check the **Memory** and **TLB** panels to see each translated address.

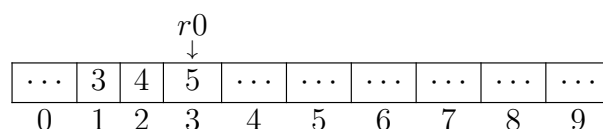
Stacks

Pre-index addressing is used to insert (or *push*) elements onto *stack*-type data structures. We discuss the details of stack-programming in Chapter 7. But we will briefly look ahead now.

Stacks are sometimes called LIFO (Last-In-First-Out) structures. The stack starts out empty and elements are added using pre-index addressing. Elements are removed (or *popped*) by using post-index addressing.

Consider this example:

```
ldr r0, =array
mov r1, #3
str r1, [r0, #4]! // stack "push"
mov r1, #4
str r1, [r0, #4]! // stack "push"
mov r1, #5
str r1, [r0, #4]! // stack "push"
```

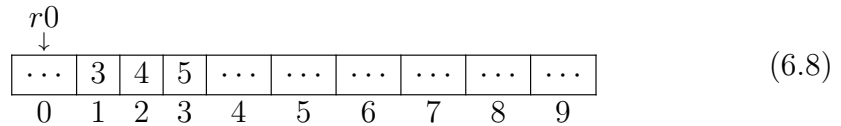


(6.7)

Now the stack has three elements, and the base-register is pointing to the *most recently added* element. We can now remove (*pop*) the elements as follows:

```
ldr r1, [r0], #-4 // stack "pop"
ldr r1, [r0], #-4 // stack "pop"
ldr r1, [r0], #-4 // stack "pop"
```

Notice, this leaves our stack-like structure looking like this:



which is exactly the configuration we had in 6.3.

Notice that when `r0` returns to its initial value (at offset 0), three elements remain in the array at positions 1,2 and 3. Does this matter? No, it does not matter. All that matters is where `r0` is pointing to. Everything following `r0` is considered to be free space.

6.5 Character Addressing Modes

In 6.4 we discussed how integer memory can be accessed. Of course, integer data is not the only type of data we may want to use in our assembly language programs. A *string* type is an array of 1-byte alpha-numeric character elements. Consider the following C code fragment:

```
char *string = "Hello World!";
```

The ARM assembly equivalent declaration is:

```
.data
string: .asciz "Hello World!"
```

Notice the declaration here: `.asciz`. This type is an array of ASCII characters (ie, a string), which may contain one or more elements. The `z` tells us that the string is terminated with a zero. This zero (the *null* character ASCII code 0) is automatically added to the end of our C and assembly strings during the compilation/assembly phase - we do not need to add it ourselves.

6.5.1 Iterating over strings

A common task is to iterate over a string looking for a particular value, or maybe looking for an uppercase or lowercase character. One problem we face is *how do we know when to stop iterating?*

The appearance of the null character signifies we have reached the end of the string in C:

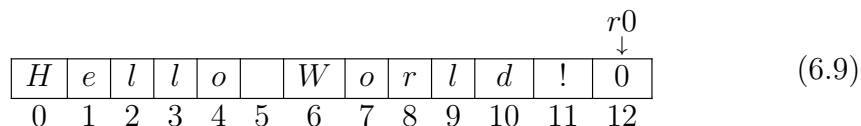
```
char string[] = "Hello World!";
char* cptr = string;
while(*cptr != 0) {
    // process the string
    // increment the pointer
    cptr++;
}
```

The ARM assembly equivalent is very similar - it must also test for the null character (ASCII 0). But because we are loading byte data and not integer data, we use the byte versions load and store instructions shown in Table 6.1:

```
ldrb r0, =string
ldrb r1, [r0], #1 //load the first character
loop:
    cmp r1, #0
    ldrneb r1, [r0], #1 //load the next character
    bne loop

loop_end:
    // the end of the loop
.data
string: .asciz "Hello World!"
```

The base-register is incremented using post-fix addressing and iteration stops when the null character is encountered:



Let us look carefully at this instruction `ldrneb r1, [r0], #1`. In ARM Assembly, the syntax `ldr<c>b` (where `<c>` is the condition `ne` not equal to) is referred to as a *Pre-UAL* syntax. This syntax has been superseded by UAL syntax, in which case the instructions is `ldrbne` (load register byte if not equal to). However, the pre-UAL syntax is the syntax used by *cpulator*, Hence this is the one we will use in these examples.

We should also note that in load or store byte instructions, the use of the immediate `#1` is correct because we are incrementing the base-register in steps of one byte (`#1`) and not one word (`#4`)

6.5.2 Zero-extending bytes

Looking again at this loop:

```
loop:
    cmp r1, #0
    ldrneb r1, [r0], #1
    bne loop
```

of course, we can see that a single byte of data is loaded into `r1` each time `ldrneb r1, [r0], #1` executes. As we know, all the registers in ARM-32 are 32-bits in size. This means that the byte of data stored in `r1` must be *zero-extended* to fit into the destination register. Consider how the ASCII value of 'A' ($65_{10} = 01000001_2$) would be handled:

31 – 24	23 – 16	15 – 8	7 – 0	
00000000	00000000	00000000	01000001	(6.10)
3	2	1	0	

As you can see, the 01000001_2 is right aligned into the LSB (byte 0), and the rest of the register is zero-filled to the MSB (bytes 1-3).

6.6 Revisiting the Program Counter

As introduced in Chapter 3 (Section 3.2.2), the Program Counter (`r15`) holds the address used during instruction fetch. However, its exact value at the point of execution is a common source of confusion in ARM assembly, and is worth examining more carefully here.

For many people, the program counter (`r15`) is a confusing register. When we first start programming in ARM assembly, we tend to think of this register as holding the address of the **c**urrently **e**xecuting **i**nstruction. However, this is not technically accurate.

Consider the following code sample:

```
mov r1, pc
mov r2, #20
mov r3, #30
```

We tend to think that `mov r1, pc` places the address of the instruction `mov r1, pc` into `r1`. This is not quite right. By the time the instruction is actually *executed*, the program counter is no longer holding the address of `mov r1, pc`. In fact, it is holding the address of `mov r3, #30`.

Why? The reason is simple: historically (but not anymore), instructions were processed in three distinct and concurrent stages. The so-called **F**etch, **D**ecode, and **E**xecute phases.¹ Thus, when the instruction was executed, the program counter has already moved to the **F**etch phase.

We can now add these phase letters to our code in order to understand where `pc` is at the point of `mov r1, pc`:

```
[E] mov r1, pc //execute, pc->Fetch
[D] mov r2, #20 //decode
[F] mov r3, #30 //fetch
```

In summary, when we see instructions like `mov r1, pc`, we should remember that this is really `pc+8`

¹we will learn more about these stages in Chapter 10

@ Pseudo-instruction written by the programmer:

```
ldr r0, =my_var
```

@ What the assembler emits (example):

```
ldr r0, [pc, #offset] @ load from address (PC + offset)
```

Because the offset is computed at *assemble time*, the code is **position-independent**: the same binary works correctly regardless of where in memory it is loaded, as long as the instruction and its data remain the same distance apart.

6.7 pc-Relative Addressing

So far we have used `ldr r0, =label` to load the address of a variable into a register. This is a *pseudo-instruction*: the assembler rewrites it behind the scenes using **PC-relative addressing**, a technique where an address is expressed as an offset from the current value of the Program Counter (**pc**).

6.7.1 ADR and ADRP

Two dedicated instructions give the programmer direct access to PC-relative address computation:

- **ADR** *Rd*, *label*: loads a PC-relative address within a range of ± 4 KB (ARM32) into *Rd*.
- **ADRP** *Rd*, *label*: ARM64 only; loads the *page* address of a label (aligns to 4 KB), enabling a two-instruction sequence to reach any 64-bit address:

```
// ARM64: load address of a global variable
adrp x0, my_var           // x0 = page base address of
my_var
add x0, x0, :lo12:my_var // x0 += offset within that page
ldr w1, [x0]             // w1 = value of my_var
```

PC-Relative Addressing

All address references in ARM machine code are ultimately encoded as offsets from **PC**. The `=label` pseudo-instruction, **ADR**, and **ADRP** are all different programmer-facing forms of the same underlying mechanism.

6.8 Literal Pools

A **literal pool** is a small region of data that the assembler embeds *inside the code section*, immediately after a function or at the end of a translation unit, to hold constant values that cannot be encoded directly into a 32-bit instruction.

ARM32 instructions are fixed at 32 bits wide. A data-processing instruction uses roughly 12 of those bits for the immediate operand, which means only constants expressible as an 8-bit value rotated by an even number of bits can

be embedded directly. A full 32-bit address (e.g. `0x00FFAB00`) cannot fit. The assembler solves this by:

1. Placing the full 32-bit constant in a nearby *literal pool*.
2. Replacing the pseudo-instruction with a PC-relative `ldr` that reads from that pool.

We can let the assembler place the literal pool in our code, or we can decide ourselves where it should be placed. The latter is accomplished by inserting the pseudo-command `.ltorg`

Let's look at an example. Notice, along with our standard ARM code, we see `.ltorg`. When the assembler encounters this it places the literal pool at that exact place.

```
ldr r0, =array
str r2, [r0, #4]
ldr r7, =0X00FFFFAA
add r5, r6, r7
.ltorg           // can be placed anywhere
add r5, r6, r7
ldr r1, =array
labelx:
add r5, r6, r7

.data
array: .word 1,2,3,4,5,6
```

What is in the literal pool? Any objects that we are seeking the address of (using the `=` prefix). So in the example here, the literal pool should consist of two objects, the constant `0X00FFFFAA` and the *address* of `array`. As we shall see, when looking at the disassembled code, the literal pool objects are computed as an offset from `pc+8`

6.8.1 Pool Placement and Range Limits

Now we study the disassembly of the code above. The file here is called `segments.txt` and can be found in the public repo here


```

0: e59f000c  ldr r0, [pc, #12] @ 14 <main+0x14>
4: e3a0200a  mov r2, #10
8: e5802004  str r2, [r0, #4]
c: e59f7004  ldr r7, [pc, #4] @ 18 <main+0x18>
10: e0865007  add r5, r6, r7
14: 00000000  .word 0x00000000
18: 00ffffaa  .word 0x00ffffaa
1c: e0865007  add r5, r6, r7
20: e59f1000  ldr r1, [pc] @ 28 <labelx+0x4>

00000024 <labelx>:
24: e0865007  add r5, r6, r7
28: 00000000  .word 0x00000000

```

The PC-relative offset in an ARM32 **ldr** is limited to ± 4 KB. If a function grows beyond that distance, the assembler will automatically insert an additional literal pool mid-function to stay within range. The assembler directive **.ltorg** can be used to force a pool at a specific location:

```

my_func:
    ldr r0, =big_constant
    @ ... many instructions ...
    .ltorg @ force a literal pool here
    @ ... more instructions ...

```

6.8.2 Viewing Literal Pools

You can inspect literal pools in a compiled binary with **objdump**:

```

[language=bash]
arm-linux-gnueabi-objdump -d my_program | less

```

Between function bodies you will see lines like:

```

00010234 <..text+0x34>:
10234: 00ffab00 .word 0x00ffab00

```

These **.word** entries are the literal pool values, sitting in the **.text** section alongside the instructions that use them.

Literal Pools

A literal pool is not a separate section: it lives inside **.text**. This is a deliberate design choice: keeping constants close to the instructions that reference them guarantees the PC-relative offset stays small.

6.9 Mixed Instructions and Data in ELF

The previous section revealed something surprising: ARM programs routinely store *data* (literal pool entries) inside the **.text** section, which is nominally

a *code* section. The ELF format is designed to allow this, and understanding why is important for reading disassemblies and writing correct linker scripts.

6.9.1 ELF Sections vs. Segments

Recall from Chapter 2 that an ELF file distinguishes between *sections* (used by the linker) and *segments* (used by the OS loader). The `.text` section is mapped into a read-only, executable **LOAD** segment. Because literal pool entries are also read-only (they are constants baked in at link time), placing them in `.text` is both *safe* and *efficient*: the loader does not need an extra segment.

6.9.2 ARM-Specific ELF Annotations

When **Thumb** code is mixed with ARM code in the same object file, the assembler must tell the linker which bytes are instructions and which are data. It does this with **mapping symbols**:

- `$a`: the following bytes are ARM (32-bit) instructions.
- `$t`: the following bytes are Thumb (16/32-bit) instructions.
- `$d`: the following bytes are data (e.g., a literal pool).

These symbols are visible in `objdump -t` output and allow debuggers, disassemblers, and the linker to correctly interpret every byte in the object file.

```
[language=bash]
$ arm-linux-gnueabi-objdump -t my_program | grep '\$'
00010220 l      .text  00000000 $a      @ ARM code starts here
00010234 l      .text  00000000 $d      @ data (literal pool)
           starts here
00010238 l      .text  00000000 $a      @ ARM code resumes
```

6.9.3 Implications for the Programmer

- Never branch into a literal pool. The bytes there are data values, not valid instructions; executing them produces undefined behaviour.
- When writing hand-crafted linker scripts, ensure the `.text` output section includes `*(.text*)` *without* splitting it in a way that separates a function from its literal pool, as this can violate the ± 4 KB range constraint.
- On ARM64 the situation is cleaner: **ADRP/add** sequences can reach any page in the 64-bit address space, so the compiler rarely needs to emit literal pools.

ARM32 vs ARM64

In ARM32, literal pools inside `.text` are common because the PC-relative range of `ldr` is limited to $\pm 4\text{KB}$. ARM64's `ADRP + add` pattern can address the entire 64-bit virtual address space in two instructions, largely eliminating the need for literal pools in normal code.

6.10 Chapter Summary

This chapter has introduced the process by which ARM assembly programs access the `.data` (static) segment of a program. We have shown the programmer must carefully choose the correct data access pattern, as well as understand how memory is accessed depending on the particular data type being used. The two data types are integer (declared as `.word`), and alphanumeric bytes (declared as `.asciz`).

You should develop an understanding of how the C programming pointer arithmetic model is implemented in ARM assembly. Developing a strong understanding of the similarities between C and assembly will make you a more well rounded software engineer, computer engineer, or general IT expert.

In section 6.4.3 we introduced the topic of *stacks*, and how we can understand these important data structures by studying how to add and remove elements using pre-index and post-index addressing respectively. In the next chapter we will discuss how stacks can be used to help with a variety of programming tasks that would otherwise be impossible without them.

Functions and The Stack

7.1 Introduction

In the previous chapter we introduced the concept of a stack. The idea of a stack should be familiar to all those who have studied discrete data structures.

As introduced in Chapter 6 (Section 6.4.3), the stack follows a **last-in-first-out (LIFO)** protocol: the most recently pushed element is always the first to be removed. In ARM, the stack is accessed via the **sp** (Stack Pointer) register, managed by the operating system.

To this general definition, we can add some ARM-specific stack observations:

1. the stack is used to store *short-lived* word data
2. it is primarily used to:
 - pass parameters from caller to callee label
 - save and restore general purpose registers
 - store label return address
 - store local variables
3. the programmer does not create the stack, the ARM microarchitecture provides access to the stack via the Stack Pointer register (**sp**) which in turn is managed by the operating system
4. the stack grows downward by default (high to low addresses) and is finite in size; for a full discussion of the Full Descending stack model and Linux thread stack limits see Chapter 2
5. as the thread executes, the stack grows and shrinks over time

No Stack Leaks

The stack should never *leak*: any words pushed onto the stack must eventually be popped. Failing to restore the stack pointer leaves the stack in a corrupted state and will cause unpredictable behaviour in subsequent function calls.

7.2 Sample C Code

In order to motivate our understanding, we will review a simple C code fragment that shows the key stack features:

```
void caller() {  
    int sum = callee(4,5,6);  
}  
  
// callee will return the sum  
// of its three parameters  
int callee(int a, int b, int c) {  
    int tmp = 0;  
    tmp = a + b + c;  
    return tmp;  
}
```

Although this is easy to implement in C, it is quite difficult to implement in ARM assembly. Let us look at the complexity involved.

1. **caller** prepares 3 parameters to share with the callee
2. **caller** invokes **callee**
3. **callee** retrieves the 3 parameters, one after the other
4. **callee** computes the sum of the three parameters and stores the result in **tmp**
5. **callee** prepares **tmp** to make it available to **caller**
6. **callee** returns to **caller**
7. **caller** uses the value returned from **callee**

As you can see, there is a lot going on here. None of the above steps would be possible without the stack. So now we will look at how to “port” this simple C program to ARM assembly by using the Stack Pointer (**sp**) register.

7.3 ARM Procedure Call Standard (APCS)

ARM defines a framework for all compilers and assembly code programmers to follow when passing parameters to a procedure (label/function). This framework is known as the ARM Procedure Call Standard (APCS).

The guide specifies conventions for passing arguments, returning values, using registers, managing the stack, and preserving program state across sub-routine calls. By following a common calling convention, separately compiled modules and libraries can interoperate correctly regardless of the compiler or programming language used.

The ARM Procedure Call Standard

- **r0-r3**: parameter/result passing (extra arguments are stacked) anyone can use these registers, but they are not saved across call.
- If **r0-r3** must be preserved, it is the caller's responsibility to do this
- **r4-r8, r10, r11**: are callee saved. This means that if the **callee** needs to use these registers in the procedure, it must first save them, and later restore them, before the procedure returns. Failure to do this will result in callee corrupted registers.
- If **r9, r12**: must be preserved, it is the caller's responsibility to do this

The ARM32 specification can be found here: [AAPCS32 Specification Repository](#)

In the example here, **caller** should prepare *three* integers for **callee**. According

Update this section

The following section **caller** prepares parameters, is totally wrong - need to differentiate between pass by reference and pass by value. The next section is mixing the two ideas but not clearly explaining it. **the following section must be rewritten**

7.4 ***caller prepares parameters*** UPDATE THIS!!

the values onto the stack, one by one, each time, growing the stack down by one word (**sp, #-4**). *Note the order in which the caller places the parameters on the stack: it does so from left to right when reading the function signature.*

```
caller:
    mov r1, #4
    str r1, [ sp, #-4 ]!
    mov r1, #5
    str r1, [ sp, #-4 ]!
    mov r1, #6
    str r1, [ sp, #-4 ]!
```

In the above example, let us assume the stack starts at address **00FFFFAA** and below is the empty stack, showing 6 free 32-bit slots

00FFFFAA		← sp	(7.1)
00FFFFA6			
00FFFFA2			
00FFFF9E			
00FFFF9A			
00FFFF96			

After `str r1, [ sp, #-4 ]!` has executed 3 times as above, the stack now looks like this:

00FFFFFFAA		← sp	(7.2)
00FFFFFFA6	4		
00FFFFFFA2	5		
00FFFFFF9E	6		
00FFFFFF9A			
00FFFFFF96			

► Watch the Stack Grow

Enter the program below into PlayARM and step through it one cycle at a time. Watch the **Memory** panel as each `str` executes — you will see the stack pointer decrement and the values 4, 5, 6 appear at successive addresses.

```
mov r1, #4
str r1, [sp, #-4]!
mov r1, #5
str r1, [sp, #-4]!
mov r1, #6
str r1, [sp, #-4]!
```

Check the **Register File** panel to confirm `sp` has moved down by 12 bytes (3 words) after all three stores.

As the stack is ready, **caller** invokes **callee** by using the branch-with-link instruction `bl`:

```
caller:
    mov r1, #4
    str r1, [ sp, #-4 ]!
    mov r1, #5
    str r1, [ sp, #-4 ]!
    mov r1, #6
    str r1, [ sp, #-4 ]!
    bl callee

callee:
    // how to access the 3 parameters?
```


7.5 Preserved and Unpreserved Registers

Caller-Saved vs. Callee-Saved Registers (ARM32)

When transferring execution control to a label, the ARM standard defines two register classes:

- **Unpreserved (caller-saved): `r0–r4`.** The callee may freely overwrite these. If the caller needs their values preserved across the call, the caller must push them onto the stack first.
- **Preserved (callee-saved): `r5–r14`.** If the callee uses any of these, it is responsible for saving and restoring them on the stack, typically immediately after saving `lr`.

Thus, the callee can directly overwrite `r0–r4` without concern for the consequences for the caller. If the caller has some content in `r0–r4` that must not be lost, then the caller must push these registers onto the stack before transferring control to the callee.

The remaining registers (`r5–r14`) must be saved and restored by the callee. That is, if the callee uses any or all of the preserved set, the callee is responsible for saving and restoring their state by placing them onto the stack right after the link register, and restoring their state before transferring control back to the caller using `bx lr`.

7.6 callee reads the parameters

How should `callee` read the parameters from the stack? Of course, there are 3 choices: offset, pre-index, post-index addressing. Which one should be used? First, we can eliminate pre-index addressing ¹. We are left with just offset or post-index addressing. Let us look at using post-index addressing first.

7.6.1 Post-index stack addressing

Let's see how `callee` can use post-index addressing to read back the three parameters:

```
callee:
    ldr r4, [ sp ], #4
    ldr r5, [ sp ], #4
    ldr r6, [ sp ], #4
```

After all three loads, the stack pointer has advanced back to its original position and the stack looks like this:

¹why is this?

00FFFFFFAA		← sp	(7.3)
00FFFFFFA6	4		
00FFFFFFA2	5		
00FFFFFF9E	6		
00FFFFFF9A			
00FFFFFF96			

Registers **r4**, **r5** and **r6** hold the parameters, and **sp** has returned to its original position (no leak). However, there are two drawbacks to this approach:

1. 3 registers are permanently allocated in the label for the parameters. This is wasteful. If a label had 10 parameters we would need 10 registers, which is impossible given ARM's register count.
2. Once **sp** has moved back, the three stack slots are gone and cannot be safely re-read.

For these two reasons, we *rarely* use post-index addressing for accessing variables from the stack. Instead we use offset addressing.

7.6.2 Offset stack addressing

With offset addressing, **sp** is *not* moved. Each parameter is read by adding a fixed byte offset to the current stack pointer:

```

callee:
    ldr r4, [ sp, #8 ]    // reads first parameter  (4)
    ldr r5, [ sp, #4 ]    // reads second parameter (5)
    ldr r6, [ sp, #0 ]    // reads third parameter  (6)

```

The stack is unchanged and **sp** stays at its current position:

00FFFFFFAA		← sp	(7.4)
00FFFFFFA6	4		
00FFFFFFA2	5		
00FFFFFF9E	6		
00FFFFFF9A			
00FFFFFF96			

The advantage is that the stack parameters are still accessible at any time during the callee's execution via their fixed offsets from **sp**. The callee must still adjust **sp** back before returning to avoid a stack leak.

► Offset vs. Post-Index Stack Addressing

Run the two programs below back-to-back in PlayARM. In both cases push the values 4, 5, 6 onto the stack first, then try each read strategy.

Step 1 — push three values:

```
mov r1, #4
str r1, [sp, #-4]!
mov r1, #5
str r1, [sp, #-4]!
mov r1, #6
str r1, [sp, #-4]!
```

Step 2 — read back with offset addressing (sp does not move):

```
ldr r4, [sp, #8]
ldr r5, [sp, #4]
ldr r6, [sp, #0]
```

After stepping through, confirm in the **Register File** that `r4=4`, `r5=5`, `r6=6`, and that `sp` has *not* changed.

7.7 callee saves lr

In Chapter 5 we discussed how execution flow can be controlled using the branching instruction `B`. We further refined this in Chapter 4 when we discussed conditional branching using instructions such as `ble`, `beq` etc.

When a branch instruction is executed, control jumps to the instruction address represented by the label name. In ARM32, there is no way to return from a label, and to continue execution from the instruction after the branch instruction. That is, there is no `return` keyword.

7.7.1 Orphaned instructions and return

Consider the example shown in Sample 7.1 to motivate our understanding, with instruction memory addresses added on the left hand side.

By the time the instruction at address `00FFFF28` executes, there is no way for `label3` to return to address `00FFFF1C`, so this instruction can never be executed. In fact, the instructions at `00FFFF1C` and `00FFFF0C` are *orphaned*, and will never be executed. Clearly, this is *not* how high-level programming languages work. Therefore, there must be a way for our assembly language to handle the natural requirement of a `return`.

7.7.2 Branch with link `bl` and the link register `lr`

To solve the problem of *where* a label should return to, we replace the `b` instruction with `bl`. The latter means branch *with link*. This causes a special register known as the link register (`lr`) to be updated with the address of the instruction following the `bl`, as shown:

```

main:
00FFFF00 mov r5, #5
00FFFF04 mov r6, #10
00FFFF08 b label2
00FFFF0C add r7, r5, r6

label2:
00FFFF10 mov r5, #11
00FFFF14 mov r6, #22
00FFFF18 b label3
00FFFF1C sub r7, r5, r6

label3:
00FFFF20 mov r5, #45
00FFFF24 mov r6, #2
00FFFF28 mul r7, r5, r6

```

Figure 7.1: Sample program with instruction addresses and label branching

```

main:
00FFFF00 mov r5, #5
00FFFF04 mov r6, #10
00FFFF08 bl label2
00FFFF0C add r7, r5, r6

label2:
00FFFF10 mov r1, #11
00FFFF14 mov r2, #22
00FFFF18 bl label3
00FFFF1C sub r3, r2, r1
00FFFF20 bx lr

label3:
00FFFF24 mov r1, #45
00FFFF28 mov r2, #2
00FFFF2C mul r3, r2, r1
00FFFF30 bx lr

```

Notice `bl` replaces `b` and `bx lr` is used to branch back to the address stored in `lr`. So we place `bx lr` at the end of our label.

Unfortunately, this does not quite work. Each time `bl` is used, it replaces the `lr` with the address of the instruction following. So although the instruction `bl label2` cause `lr` to be updated correctly, *the subsequent* `bl` instructions will overwrite it. The link register is assigned `00FFFF0C`, then it is assigned `00FFFF1C`. If a label calls a label, which calls a label etc. then *only the most recent* `lr` value is preserved, and all the others are lost. How should we solve this? By using the stack to store `lr` as the first thing a label does. We can rewrite the code as follows:

```

main:
00FFFF00 mov r1, #5
00FFFF04 mov r2, #10
00FFFF08 bl label2
00FFFF0C add r3, r2, r1

label2:
00FFFF10 str lr, [ sp, #-4 ]!
00FFFF14 mov r1, #11
00FFFF18 mov r2, #22
00FFFF1C bl label3
00FFFF20 sub r3, r2, r1
00FFFF24 ldr lr, [ sp ], #4
00FFFF28 bx lr

label3:
00FFFF2C str lr, [ sp, #-4 ]!
00FFFF30 mov r1, #45
00FFFF34 mov r2, #2
00FFFF38 mul r3, r2, r1
00FFFF3C ldr lr, [ sp ], #4
00FFFF40 bx lr

```

The above solution now properly handles the case of nested labels, and how to return from them using the stack to store temporary `lr` values. Notice the stack pointer is correctly reset at the end, and the callee label always saves the state of the `lr` as the *very first thing it does*

7.8 callee saves its working set of registers

When **callee** begins execution, it should, as mentioned in Section 7.7, save the `lr` on the stack as its *first* instruction. It should *then* save the set of working registers it will need in order to complete the label code *without clobbering the callers registers*. As mentioned previously, this is the task of saving the *preserved* registers.

This is a very important responsibility of **callee**. A callee that omits to save its working set of preserved registers will almost certainly clobber the registers of caller.

Consider again the code from Example 7.1:

```
main:
00FFFF00 mov r1, #5
00FFFF04 mov r2, #10
00FFFF08 b label2
00FFFF0C add r3, r2, r1

label2:
00FFFF0C mov r1, #11
00FFFF10 mov r2, #22
00FFFF14 b label3
00FFFF18 sub r3, r2, r1

label3:
00FFFF1C mov r1, #45
00FFFF20 mov r2, #2
00FFFF20 mul r3, r2, r1
```

Notice how, in each label, **callee** is overwriting the values that the **caller** has placed in the registers (**r1** and **r2**). Of course, the **caller** cannot predict how **callee** will use the registers. The code may be written by different programmers, or the code for **caller** may not be available in source code format to the developer of **caller**. The ARM best practice guide recommends that the caller should save any unreserved registers it uses, and the callee is responsible for saving and restoring the preserved registers it will be using in the label code. For example, the code in **label2** overwrites **r1** and **r2** of **caller**, thus clobbering them. We see that the code in **label3** does exactly the same thing for its **caller**. This kind of problem is easy to solve: **callee** should save its working set of registers on the stack before it begins, and restore the register set when it finishes:

```

main:
00FFFF00 mov r1, #5
00FFFF04 mov r2, #10
00FFFF08 b label2
00FFFF0C add r3, r2, r1

label2:
00FFFF20 str r1, [sp, #-4]
00FFFF24 str r2, [sp, #-4]
00FFFF28 str r3, [sp, #-4]
00FFFF2C mov r1, #11
00FFFF30 mov r2, #22
00FFFF34 b label3
00FFFF38 sub r3, r2, r1
00FFFF3C ldr r3, [sp], #4
00FFFF40 ldr r2, [sp], #4
00FFFF44 ldr r1, [sp], #4

label3:
00FFFF48 str r1, [sp, #-4]
00FFFF4C str r2, [sp, #-4]
00FFFF50 str r3, [sp, #-4]
00FFFF54 mov r1, #45
00FFFF58 mov r2, #2
00FFFF5C mul r3, r2, r1
00FFFF60 ldr r3, [sp], #4
00FFFF64 ldr r2, [sp], #4
00FFFF68 ldr r1, [sp], #4

```

Notice how the save/restore sequence follows the Last-In-First-Out (LIFO) semantics of the stack: save $[r1, r2, r3, \dots, rn]$ then restore $[rn, \dots, r2, r1]$. Of course, you should only save and restore the set of registers you will be using in your label. There is no need to save and restore all of them.

7.9 Stack Overflow and Underflow

The stack is a critical part of the ARM architecture, and managing its space effectively is vital to prevent runtime errors such as **stack overflow** and **stack underflow**. These errors can disrupt the execution of a program and lead to unpredictable behavior.

7.10 Stack Overflow

A **stack overflow** occurs when more data is pushed onto the stack than the available space can handle. This situation typically arises in recursive functions or when a large amount of memory is allocated to the stack without considering its limitations. In ARM, since the stack grows downwards, exceeding its bounds can cause it to overwrite other important data, leading to crashes or undefined behavior.

7.10.1 Causes of Stack Overflow

The primary causes of stack overflow are:

- **Deep Recursion:** Recursive functions call themselves, and each call pushes its return address and local variables onto the stack. Without a base case or if recursion depth exceeds the stack capacity, the stack will overflow as more data is pushed onto it.
- **Large Local Variable Allocation:** Functions that allocate large arrays or variables on the stack may exceed the available stack space. If this happens, the stack will overflow.

In ARM assembly, the stack grows from high memory to low memory addresses. When the stack grows beyond its allocated size, the **sp** (stack pointer) may overwrite crucial data, leading to undefined behavior and program crashes.

7.10.2 Example of Stack Overflow in Recursive Function

A common scenario that leads to stack overflow is a recursive function that does not have a base case or has excessive recursion depth. Each recursive call pushes more data to the stack, and if the function calls itself indefinitely or too many times, the stack will overflow.

Example: A function that recursively calls itself without a base case:

```
void recursive_call() {  
    recursive_call();  
}
```

In ARM assembly, this can look like:

```
recursive_call:  
    push {lr}           // Save lr on to the stack  
    bl recursive_call    // Call the function recursively  
    pop {lr}            // Restore lr from the stack  
    bx lr               // Return from the function
```

- Each recursive call adds its return address onto the stack using **push lr**.
- Without a base case, the function continues calling itself, leading to the stack growing larger
- Eventually the stack exceeds the kernel allocated size, causing a stack overflow.

7.10.3 Effects of Stack Overflow

When a stack overflow occurs, several issues can arise:

- **Memory Corruption:** The stack is supposed to store critical data such as return addresses, local variables, and saved registers. If the stack over-

flows, it overwrites this data, leading to corrupted values and unpredictable behavior.

- **Program Crashes or Undefined Behavior:** As the program may attempt to execute corrupted return addresses or access invalid memory locations, it can result in crashes or undefined behavior. This can cause the program to behave in unexpected ways or terminate unexpectedly.
- **Security Vulnerabilities:** Stack overflows are a common vulnerability exploited in buffer overflow attacks. An attacker may intentionally overflow the stack to inject malicious code and gain control over the program.

7.10.4 How to Prevent Stack Overflow

There are several strategies to prevent stack overflow in ARM assembly programming:

Limit Recursion Depth

Ensure that recursive functions have a proper base case to terminate the recursion. Avoid infinite recursion, and limit the recursion depth to prevent excessive stack growth.

Example: Recursive function with a base case:

```
recursive_call:
    cmp r0, #0           // Compare argument with 0
    beq end_recursion    // If base case reached, return
    push {lr}            // Save return address
    sub r0, r0, #1       // Decrement counter by 1
    bl recursive_call     // Recursive call
end_recursion:
    pop {lr}             // Restore return address
    bx lr                // Return from the function
```

In this example, the recursion halts once the argument `r0` reaches zero, preventing infinite recursion and stack overflow.

Optimize Stack Usage

Minimize stack usage by avoiding large local variables. Instead of allocating large arrays on the stack, allocate them dynamically in the heap.

Example: Using dynamic memory allocation instead of stack allocation:

```
// Avoid allocating large arrays on the stack:
sub sp, sp, #1000 // 1000 bytes on the stack
// Instead, use dynamic memory allocation on
// the heap to avoid stack overflow.
// Example in C: malloc() for large arrays
// along with the code to integrate libc calls
```

► Safe Iterative Sum — No Stack Growth

Instead of a recursive function (which needs `bl/bx lr`), use a loop to compute the sum $1 + 2 + \dots + 5$. This uses only the stack for storing the running total, with no risk of overflow.

```
mov r0, #5          // n = 5
mov r1, #0          // sum = 0
loop:
    add r1, r1, r0    // sum += n
    sub r0, r0, #1    // n--
    cmp r0, #0
    bne loop          // repeat while n != 0
str r1, [sp, #-4]!   // push final sum onto stack
```

Step through in PlayARM and watch `r1` accumulate the sum. After the `str`, check the **Memory** panel to confirm the result (15) is stored at the top of stack.

Use Tail Recursion

Tail recursion allows a function to reuse its current stack frame for recursive calls, thus avoiding the growth of the stack. A tail-recursive function does not require additional stack space for each recursive call, which helps prevent stack overflow.

Example of Tail Recursion:

```
tail_recursive:
    cmp r0, #0          // Check if base case
    beq end_tail_recursion
    sub r0, r0, #1      // Update argument
    bl tail_recursive   // Tail call (no extra stack frame
                        // needed)
end_tail_recursion:
    bx lr               // Return from function
```

In tail recursion, the function does not create a new stack frame for each recursive call, thus preventing stack overflow.

Use the Stack Wisely

Avoid unnecessary usage of the stack. Only push registers to the stack when they are necessary to preserve, and use registers for temporary variables when possible.

Example: Pushing only necessary registers to the stack:

```
// Avoid pushing unnecessary registers to the stack
push {r4, r5}          // Only push necessary registers
// Function logic
pop {r4, r5}           // Restore only the necessary
                        // registers
```

By limiting the number of pushed registers, the stack usage is minimized, reducing the risk of overflow.

Monitor Stack Usage

In embedded systems, monitoring the stack usage through system logs or dedicated tools can help detect potential stack overflows before they happen. A stack guard mechanism can be implemented to check for overflow conditions and raise exceptions if the stack is nearing its limit.

7.10.5 Conclusion on Stack Overflow

Stack overflow is a critical issue in ARM assembly and other low-level programming environments. It is commonly caused by excessive recursion or large allocations on the stack. By limiting recursion depth, using the stack wisely, and employing techniques like tail recursion, you can minimize the risk of stack overflow and ensure that your program executes efficiently and securely.

7.11 Prologue and Epilogue

Each function typically includes a **prologue** and an **epilogue** that manage the stack, save and restore registers, and adjust the stack pointer for local variable allocation.

The **prologue** and **epilogue** are essential for managing the stack during function calls in ARM assembly. These two sequences of instructions handle saving and restoring registers, allocating and deallocating space for local variables, and ensuring that control is properly returned to the caller.

7.11.1 Prologue: Setting Up the Stack Frame

The **prologue** is the first part of a function, responsible for setting up the stack frame. The stack frame stores the function's local variables, return address, and callee-saved registers. The prologue is crucial for establishing the function's execution environment.

- **Saving the LR:** The link register (LR) holds the return address, which tells the processor where to resume execution after the function finishes. It must be saved on the stack to ensure the function can return to the correct location.
- **Saving Callee-Saved Registers:** The registers **r4–r11** are callee-saved, meaning the callee is responsible for saving and restoring these registers if it uses them. This prevents the callee from inadvertently modifying the caller's state.
- **Adjusting the Stack Pointer (sp):** The **sp** is adjusted to allocate space for local variables. The amount of space depends on how many local variables the function needs to store.

7.11.2 Epilogue: Returning Control to the Caller

The **epilogue** is executed at the end of a function to clean up the stack and return control to the caller. It undoes the actions performed by the prologue and ensures that the function call does not leave the stack in an inconsistent state.

- **Restoring Callee-Saved Registers:** The registers saved in the prologue are restored to their original values. This ensures that the caller's state is preserved across function calls.
- **Deallocating Space for Local Variables:** The `sp` is adjusted to release the space allocated for local variables. This ensures the stack is returned to its previous state.
- **Restoring the LR and Returning:** The return address stored in `lr` is restored, and the program jumps back to the caller using the `bx lr` instruction.

Example of Prologue and Epilogue

Here is a simple example of how the prologue and epilogue are implemented in ARM assembly:

```
// Function Prologue
function_prologue:
    push {lr}           // Save return address (Link
                        // Register)
    push {r4-r7}        // Save callee-saved registers
    sub sp, sp, #16     // Allocate space for 16 bytes of
                        // local variables

    // Function Body (do some work here)

    // Function Epilogue
function_epilogue:
    add sp, sp, #16     // Deallocate 16 bytes of local
                        // variable space
    pop {r4-r7}         // Restore callee-saved registers
    pop {lr}            // Restore the return address (
                        // Link Register)
    bx lr               // Return to the caller
```

7.12 Register Allocation and Stack Management

ARM's 16 general-purpose registers (`r0–r15`) are introduced in Chapter 3. Their calling-convention roles—which registers are caller-saved, which are callee-saved, and how parameters are passed—are specified by APCS as described in Section 7.3 of this chapter. Keeping register usage within APCS guidelines is essential: accessing registers is far faster than memory, and minimising unnecessary stack spills improves performance significantly.

7.13 PC-Relative Addressing and Literal Pools

7.13.1 PC-Relative Addressing

In ARM assembly, many instructions that load a value or jump to a label encode the *offset* from the current program counter (PC) rather than an absolute address. This is called **PC-relative addressing**. It has two key advantages:

- **Position-independent code (PIC):** because only offsets are stored, the same binary can be loaded at any address.
- **Compact encoding:** a 12-bit immediate offset can reach ± 4 KB in ARM32 and a 19-bit offset reaches ± 1 MB in AArch64.

The assembler uses PC-relative addressing automatically when you write:

```
ldr r0, =myLabel    // assembler picks the closest literal
pool
b    someFunction    // encoded as offset from PC
```

At runtime, the processor computes the effective address as PC + offset.

ARM32 PC offset

In ARM32, when an instruction reads PC its value is the address of the *current instruction plus 8 bytes* (due to the three-stage pipeline pre-fetch). Keep this in mind when computing PC-relative offsets by hand.

7.13.2 Literal Pools

Literal pools are covered in detail in Chapter 6 (Section 6.7), including the `ldr rN, =value` pseudo-instruction, the 4 KB reach constraint, and the `.ltorg` directive. The PC-offset note above (ARM32 adds 8 bytes) applies equally when computing literal pool offsets by hand.

7.14 ARM32 vs. ARM64 Stack Conventions

ARM32 vs. ARM64: Stack & Calling Conventions		
Feature	ARM32 (AArch32)	ARM64 (AArch64)
Stack pointer	r13 / sp	x28 or sp (x31)
Link register	r14 / lr	x30 / lr
Frame pointer	r11 / fp (optional)	x29 / fp
Return	bx lr	ret (branches to x30)
Save/restore pair	push/pop (single regs)	stp/ldp (pairs, e.g. stp x29,x30,[sp,#-16]!)
Stack alignment	4-byte (word)	16-byte mandatory at all public call boundaries
Argument regs	r0–r3	x0–x7
Callee-saved	r4–r11	x19–x28
Key implication: AArch64 always saves x29 (FP) and x30 (LR) together as a pair, and sp must be 16-byte aligned before any bl/blr.		

7.15 Exercises on Programming the Stack

7.15.1 Exercise 1: Stack Management in Recursive Functions

Write a recursive function in ARM assembly to calculate the factorial of a number. Implement the function without a base case and observe how the stack grows, leading to a stack overflow. Then, modify the function to include a base case and explain how adding the base case prevents the overflow.

Instructions:

- Write the recursive function to compute the factorial without a base case.
- Simulate how the stack overflows when the recursion depth exceeds the available space.
- Modify the function to include a base case and explain how it fixes the overflow.

7.15.2 Exercise 2: Understanding Caller-Callee Stack Interaction

Given the C code below:

```
void caller() {  
    int sum = callee(4,5,6);  
}  
int callee(int a, int b, int c) {  
    int tmp = 0;  
    tmp = a + b + c;  
    return tmp;  
}
```

Task: Translate this C code to ARM assembly by using the stack to pass parameters from the caller to the callee, store local variables, and manage the return address.

Instructions:

- Implement the function ‘caller’ and ‘callee’ in ARM assembly, passing parameters through the stack.
- Use the ‘sp’ (Stack Pointer) and ‘LR’ (Link Register) properly to manage the stack and return address.

7.15.3 Exercise 3: Stack Overflow Prevention

Write a function in ARM assembly that uses a loop to sum integers from 1 to n. Compare the iterative solution with a recursive one and analyze how stack overflow could occur in the recursive version.

Instructions:

- Implement both iterative and recursive versions of the summing function.
- Simulate how the recursive function could lead to a stack overflow if ‘n’ is too large.
- Modify the recursive version to use tail recursion and explain how it reduces stack usage.

7.15.4 Exercise 4: Prologue and Epilogue Function Design

Write an ARM assembly function that uses a prologue to save registers and allocate space for local variables, and an epilogue to restore registers and deallocate the space.

Instructions:

- Implement a function that saves and restores registers (‘r4’ to ‘r7’) as part of the prologue and epilogue.
- Allocate space for local variables in the prologue and deallocate them in the epilogue.
- Ensure the function returns control to the caller correctly using the ‘bx lr’ instruction.

7.15.5 Exercise 5: Exploring Stack Underflow

Write a program that demonstrates stack underflow in ARM assembly. Underflow occurs when the stack pointer is adjusted incorrectly, and the program attempts to pop more data than was pushed onto the stack.

Instructions:

- Write a function that incorrectly manipulates the stack pointer by popping data without pushing it first.
- Simulate stack underflow by observing incorrect data restoration, or by analyzing how crashes occur when the stack is manipulated incorrectly.
- Explain how correct stack management can prevent stack underflow.

7.15.6 Exercise 6: Stack Size Limitation and Stack Guard

Implement a simple ARM assembly program that simulates exceeding the stack size limit. Show how a guard mechanism (e.g., checking if the stack pointer exceeds a set limit) can prevent stack overflow.

Instructions:

- Write a function that simulates allocating large data on the stack and exceeding the stack's size limit.
- Implement a stack guard that checks if the stack pointer exceeds the set limit and prevents overflow.
- Demonstrate how the guard mechanism can safely terminate the program when a potential overflow is detected.

Integrating libc

8.1 Linux System Calls

In the previous chapters we learned the basics of ARM32 assembly techniques and guidelines. The code we looked at was limited in terms of its *functionality* - it was limited to logical, mathematical and flow control instructions. The code did not require any services from the operating system.

These services are known as *system calls*, and a typical application will use many system calls in order to achieve its objectives. Indeed, an application program that does not use system calls can not achieve anything useful.

There are two ways for an assemble program to access the operating system services (1) by using system calls directly from the assembly, and (2) linking to a library of functions that provide an abstraction layer above the system call level. Such libraries are very useful because it allows us to reuse high-level abstractions rather having to build the scaffolding code around the service call. The difference is summarized in Fig. 8.1.

Directly using system calls from an application process is a difficult task. It requires the developer to execute a software interrept (`svc #0`) instruction, along with populating specific registers correctly. For example, the system call to print "Hello World\n" to the screen shown in Fig 8.2, and as you can see, the implementation in `print` is rather complex for such as simple tasks.

Using system calls for I/O, process management, memory allocation etc. is perfectly possible, but requires us to reinvent functionality that has already been solved and implemented elsewhere. As shown in Fig. 8.2, we will have difficulty creating a `printf` type function - one that supports parameter substitution within the string. For example, consider this simple C program:

As we can see clearly from Fig. 8.3, `printf` supports parameter substitution by way of the `%s`, `%d`, ... modifiers. How could we achieve this functionality using the `svc` instruction? With great difficulty!

In summary, although all the kernel functions for I/O, memory management, process management etc are accessible via system calls, we tend to avoid using this method because it requires us to build complex scaffolding code that adds to the overall complexity and management of the assembly application. Therefore, we leave it up to you to build your own `printf` function using service calls only.

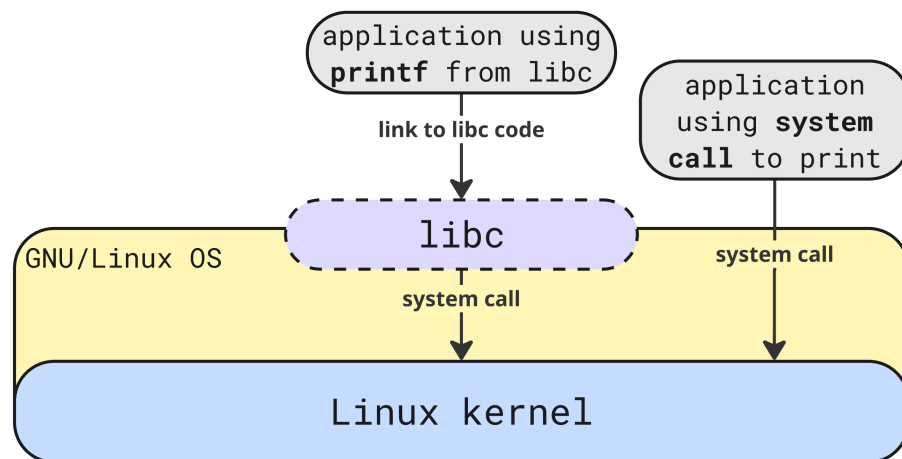


Figure 8.1: Application code can use either `libc` or system calls (`svc` to print to the screen)

```

print:
    mov r0, #1      // r0 gets 1=stdout
    ldr r1, =string // r1 gets the string address
    mov r2, #12     // r2 is the string length
    mov r7, #4      // r7 gets 4=print system call
    svc #0
.data
string: .asciz "Hello World\n"
  
```

Figure 8.2: Assembly code for printing to the screen using the system call instruction `svc`

```

int main(int argc, char* argv[]) {
    printf("num params: %d, \
    program name %s, \
    first param: %s\n", \
    argc, argv[0], argv[1]);
}
  
```

Figure 8.3: C code for printing to the screen using `printf` from `libc`

8.1.1 Linux libc

GNU/Linux `libc` contains over 400 functions¹ functions that can be called from user-layer applications. A complete discussion on `libc` is beyond the scope of this book. Readers interested in looking at `libc` in detail should refer to <https://www.gnu.org/software/libc/>

We will discuss one `libc` example next: the `printf` function shown in Fig. 8.3. `printf` can be used to print a string along with any number of *arguments* to the string. See <https://man7.org/linux/man-pages/man3/printf.3.html>.

8.1.2 Printing to the screen using `printf`

To make the example more interesting, we will use `printf` to print the following information to the screen:

- the number of command line parameters to the program
- the name of the program
- the first parameter to the program

Assuming that the program is called `a.out`, the running the following command:

```
$ ./a.out 55
num params: 2, program name ./a.out, first param: 55
```

This example demonstrates some interesting features. First, notice that we need to obtain command line parameters the OS passes into our application. In C, we use the standard main function signature `int argv, char *argv[]` to obtain the command line parameters passed in. But how do we get the equivalent of `int argv, char* argv[]` in ARM assembly? The convention is that:

- `r0` contains the `argv`
- `r1` is a base-register which points to an array of `char*` pointers. This is the definition of `char* argv[]`
- The stack pointer `sp` is *not* used to pass items from the command line into `main`

Based on the code in Fig 8.4 we see how to integrate `printf` from `libc`. We will briefly discuss this solution next.

Discussion of Fig 8.4

Let's focus on these two instructions with line numbers:

```
1. ldr r2, [ r1 ]
2. ldr r3, [ r1, #4 ]
```

¹excluding the `_` functions. If we include these functions, this number rises to over 3000

```

main:
    // save the return address from main
    push {lr}

    // in ARM32 r1 is the base-pointer
    // to an array of pointers to strings
    // zero-ith parameter - program name
    ldr r2, [ r1 ]
    // first parameter - program parameters
    ldr r3, [ r1, #4 ]

    // if have n parameters they are
    // accessed as r1 + (n * 4)
    // we are finished accessing r1
    // so we can prepare it for printf
    mov r1, r0
    ldr r0, =output_string

    // use puts or printf from libc
    bl printf

    // set up mains return parameters
    mov r0, #0

    // now return from main
    pop {pc}

.data
output_string:
    .asciz "num params: %d, program name %s, first param:
    %s\n"

```

Figure 8.4: Assembly code for printing to the screen using `printf` from `libc`

- line 1: `r1` is dereferenced (`[r1]`) we get `argv[0]` (which is *always* the name of the program).
- line 2: Using offset addressing to point to the next pointer and dereferencing (`[r1, #4]`) brings us to `argv[1]`.

Next, we look at how to prepare parameters for passing information `libc` functions. To better understand how to prepare parameters into `libc` you must carefully review the function prototype from the man pages:

```
int printf(const char *restrict format, ...);
```

Notice that the first parameter to `printf` is the formatting string. In our case, this string is:

```
num params:  %d, program name %s, first param:  %s
n
```

```
3. mov r1, r0
4. ldr r0, =output_string
5. bl printf
```

ARM32 has some specific rules around passing parameters into functions. For the first *four* parameters, registers `r0-r3` should be used. This means the stack is not involved. If there are more than four parameters, then all subsequent parameters should be pushed on to the stack from right-left ordering. However, here we have only 4 parameters, so `r0-r3` will be used, as follows:

1. line 3: place the value of `argv` into `r1`. (
2. line 4: set `r0` as the address of the formatting string
3. line 5: branch with link to `printf`

Graphically this is summarized in Fig 8.5, where the application (`a.out`) is invoked and the OS passes in command line parameters (in `r0` and `r1`). The application then process the parameters and sets up the registers correctly (`r0-r3`), then invokes `printf`

Finally

Finally, note this code fragment:

```
push {lr}
// .. call to printf
mov r0, #0
pop {pc}
```

as we know, all labes (and this includes `main`), should save the link register (`push {lr}`) as the first instruction. Notice when the `printf` routine returns we set `mov r0, #0`. This is used by the OS to understand if our program completed successfully or not.²). Finally, the link register value is popped and placed in the program counter (`pop {pc}`). This is effectively how a return takes place. Of course, when `main` returns the program is finished executing.

How is the value in `r0` used? We can use the linux parameter `$?` to check:

²where a non-zero value is used to indicate an error

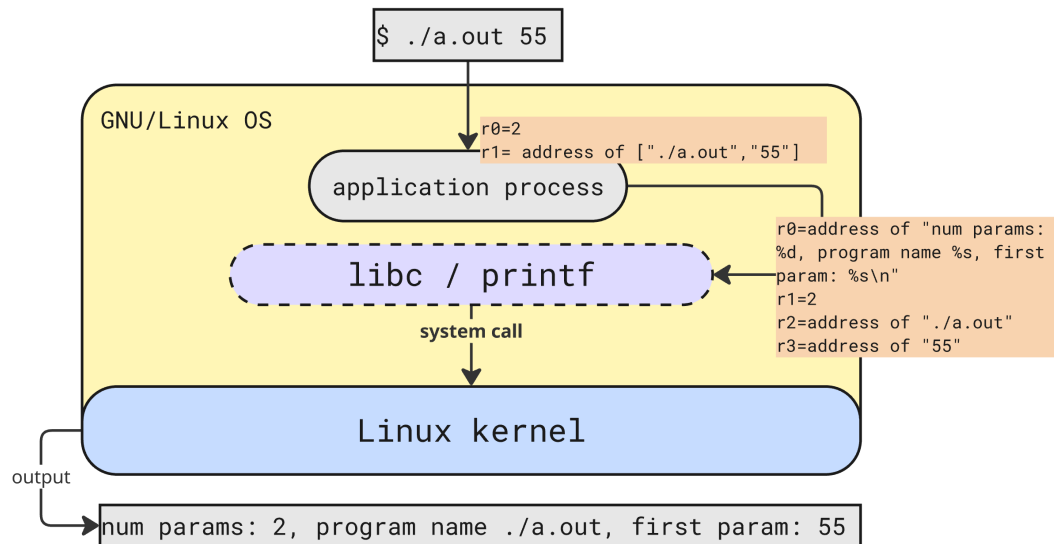


Figure 8.5: The application (**a.out**) is invoked and the OS passes in command line parameters (in **r0** and **r1**). The application then process the parameters and sets up the registers correctly (**r0-r3**), then invokes **printf**

```

$ ./a.out 55
num params: 2, program name ./a.out, first param: 55
$ echo $?
0

```

When we run **echo \$?** and get 0, we are receiving the 0 placed into **r0** prior to the return from **main**

8.2 Chapter Exercises using **libc** functions

Exercise 8.2.1

Write an assembly program to iterate over **argv** printing out each parameter (using **printf**) on a new line. For example:

```

$ ./a.out 55 66 77 88 99
param 1: 55
param 2: 66
param 3: 77
param 4: 88
param 5: 99

```

In **libc**, upper and lower case string manipulation is handled by **int toupper(int c)** and **int tolower(int c)** functions respectively. Using , write an assembly program to convert its use input from upper or mixed case, to all lower case. For example:

```

$ ./a.out MY NamE iS TOLOwer
lower_case: my name is tolower

```

The libc function `int atoi (const char *string)` takes a string pointer and converts it into an integer. For example, calling

```
int r = atoi("55")
```

would convert "55" to 55 in a C program. Write an assembly program that takes two parameters and adds them together, printing the result output:

```
$ ./a.out 55 65
Total is: 120
```

For this to work correctly you will need to use both `atoi` and `printf` from `libc`

Write an assembly language program to read from a the users name from the command line, and print a friendly greeting to the user. For example:

```
$ ./a.out
Hi! - what is your name? [ user enters Bob ]
Hi Bob, nice to meet you!
```

You will need to use `printf` and `gets` from `libs`

Write an assembly program to give the user 3 random numbers r_1, r_2, r_3 where $0 \leq r_k \leq 100$. You can store the random values in `r4-r6`. Print the results to the user. For example, here is some sample output:

```
$ ./a.out
Your 3 numbers are: 55, 22, 10
```

You will need to use `int rand (void)` and of course, `printf`

Binary Representation of Instructions

So far in this book we have been focused on the *higher-level* challenges of selection, iteration, branching, accessing memory, reading and writing the stack, and so on. Of course, even assembly language program consists of textual instructions that a CPU simply does not understand. To the CPU, the only understandable input is a word-size binary (or hexadecimal) number.

This naturally raises two related questions:

1. By what process is an assembly instruction translated into binary machine code?
2. What design constraints shape the representation of textual instructions in binary form?

In this chapter, we will address both of these important questions together as we proceed. We first begin with a motivating example.

9.1 Motivating Example

So we begin this chapter by noting that **each and every line of code we write in ARM32/64 *must* fit into one 32-bit field.**

For example, every line of code from this fragment (taken from Chapter 6) must fit into one 32-bit word in ARM7DTI:

```
ldr r0, =array
mov r1, #0
mov r2, #0
mov r3, #6

loop:
    cmp r1, r3
    strlt r2, [ r0, r1, lsl #2 ]
    addlt r1, r1, #1
    blt loop

.data
array: .word 1,2,3,4,5,6,7
```

It is the objective of this chapter to first explain the *process* by which instructions (like those shown) above, can be encoded in a 32-bit field. It

should be noted that in both ARM32- and 64- bit platforms, instruction length is 32-bits. So on 64-bit platforms one memory word contains two instructions, whereas on the 32-bit platform, one memory word contains one instruction.

As you will see, there is nothing mysterious about encoding instructions into *machine code*,

if we follow some basic structural patterns. These patterns are based on the *class* of instruction: being encoded. There are only 3: (a) **data processing** (b) **memory**, and (c) **branching**. Each of these will be covered in detail in Section 9.2

In the above example, we have eight instructions. At the end of the assembly process there will be 8 32-bit instructions in our ELF file too, although there will also be a lot of set-up and configuration instructions added by the assembler (/compiler) and the linker. However, *conceptually*, our 8 instructions in a text file will result in 8 executable 32-bit fields in our ELF file (again, assuming we are on the ARM7DTI platform).

So our 8 instructions will look like:

```
e59f0018
e3a01000
e3a02000
e3a03006
e1510003
b7802101
b2811001
baffffff
```

or in binary format:

```
111001011001111100000000000011000
111000111010000000001000000000000
111000111010000000010000000000000
1110001110100000000110000000000110
111000010101000100000000000000011
101101111000000000010000100000001
101100101000000010001000000000001
10111010111111111111111111111011
```

Now that we know *what* is happening, we will focus on *how* it happens: how a text instruction (eg `ldr r0, =array`) is converted into a binary representation (in this example, `111001011001111100000000000011000`)

9.2 Instruction Classes

ARM, in common with most ISAs, defines three classes of instructions:

1. **Data Processing (DP) Instructions:** the instructions that use the Arithmetic and Logical Unit (ALU) of the CPU to perform mathematical or register move operations. There are 16 different such instructions (in ARM32), for example: `add`, `sub`, `mul`, `mov` and so on. It also includes

logical instructions such as `and`, `orr` along with all the bitwise instructions, for example, `lsl`, `lsr`, `asr`, `ror`.

2. **Memory Instructions:** this smaller set of instructions, excluding the conditionals, signed and flag setting variants, has only four main classes that we will consider (again, there are others): `ldr`, `ldrb`, `str`, `strb`
3. **Branching Instructions:** this set of instructions contains three main variants (there are some others): `b`, `bl`, `bx`

9.3 Data Processing Instructions

The class of *data processing* instructions are those whose job it is to move and operate on data. For example, these are the all the `mov{cond}{S}` variants, along with all mathematical and logical instructions (the more common of which are shown in Table 3.1).

In some ways, data processing instructions are the most complex in terms of their 32-bit binary encoding. The encoding must allow for a great deal of variation in instruction format, for example:

```
mla r0, r1, r2, r3
addlts r4, r5, r6
and r5, r6, #1
sub r7, #4
```

Notice the variation here: `sub r7, #4` and `mla r0, r1, r2, r3` for example. Both of these instructions must be encoded using the data processing layout that we will discuss next. How this happens is very interesting.

9.3.1 Non-Multiply Instruction Format

There are eight different parts to the general (non-multiply) data processing instructions shown here. Along the top row, for each field, is the bit from:to pair, and along the bottom row is the number of bits for the field.

31:28	27:26	25	24:21	20	19:16	15:12	11:0
cond	00	I	cmd	S	Rn	Rd	source 2
4	2	1	4	1	4	4	12

We now discuss each of these in turn, from the most significant bit (31) to the least significant bit (0).

[31:28] **cond**: the *condition* code for the instruction. Used to determine if the instruction is to be executed. The decision is made by the micro-architecture in reference to the state of the `cpsr/nzcv` state. The full list of condition codes is shown in Table 9.1, left hand side

[27:26] **00**: the *operation* code for the instruction. **00** means *Data Processing*

[25] **I** the *immediate* bit. Where $I = 1$ the instruction contains an immediate value (for example `add r0, r1, #5`) and $I = 0$ there is no

immediate (for example, `add r0, r1, r2, add r0, r1, r2, lsl #2`

[24:21] **cmd**: the data processing operation to perform. With 4 bits for **cmd** there are only 16 possible variants as shown in Table 9.1 right hand side

[20] **S**: the *update-flags* bit, when $S = 1$ the output from the operation (for example, a negative number) is updated to the **NZCV** condition register (eg, `adds r0, r1, #-10`). Where $S = 0$ the register is not updated (eg, `add r0, r1, #-10`)

[19:16] **Rn**: the *first source register*. For example, in the instruction `add r0, r1, r2`, **Rn** is **r1**

[15:12] **Rd**: the *destination register*. For example, in the instruction `add r0, r1, r2`, **Rd** is **r0**

[11:0] **src2**: the second *source* parameters. There are 4 possible arrangements for **src2** as discussed in Table. ??

Notice an especially important fact: the ARM32 non-multiply data-processing instructions have a 12-bit field called **SRC2** ([11:0]) which is used to encode a variety of operands to the instruction. This section of the instruction is discussed in Table ??.

Code	Meaning
0000	Equal to (eq)
0001	Not equal (ne)
0010	Carry set (cs/hs)
0011	Carry clear (cc/lo)
0100	Negative (minus) (mi)
0101	Positive or zero (plus) (pl)
0110	Overflow set (vs)
0111	Overflow clear (vc)
1000	Unsigned higher (hi)
1001	Unsigned less than or same (ls)
1010	Signed greater than or equal to (ge)
1011	Signed less than (lt)
1100	Signed greater than (gt)
1101	Signed less than or equal to (le)
1110	Unconditional - always execute

Code	cmd
0000	and
0001	eor
0010	sub
0011	rsb
0100	add
0101	adc
0110	sbc
0111	rsc
1000	tst
1001	teq
1010	cmp
1011	cmn
1100	orr
1101	mov
1110	bic
1111	mvn

Table 9.1: On the left, the condition codes for 31 : 28 (there is no 1111 pattern). On the right, the arrangement for **cmd** (24 : 21)

Src2: The Second Source

The non-multiply data processing instructions have a region of the instruction known as second source (**src2**) from bit 11:0. The reason its called second source, is because **Rn** is the first source! The purpose of this 12-bit section is

to accommodate a variety of different second source arrangements. Here are 4 examples to illustrate this diversity, where the **31:12** is the same, and **11:0** is different. Note that sections in blue are hard-coded and not dependent on the operands of the instruction:

1. `add r5, r1, #2`: “Immediate” variant:

11:8	7:0
ror	Imm

2. `add r5, r1, r2`: “Register” variant:

11:8	7	6:5	4	3:0
0000	0	00	0	Rm

3. `add r5, r1, r2, lsl #3`: “Immediate shifted Register”

11:7	6:5	4	3:0
shamt5	sh	0	Rm

4. `add r5, r1, r2, lsl r3`: “Register shifted Register”

11:8	7	6:5	4	3:0
Rs	0	sh	1	Rm

9.3.2 Worked Examples

We will work through the examples from here to show how the in hexadecimal/binary generated by the assembler can be analyzed. As all four instructions share the same **[31:12]** we will briefly cover them first before looking at the **src2** variants.

1. *Immediate*: `add r5, r1, #2`

- (a) $31 : 28 = 1100$ - this is unconditional execution (Table 9.1, LHS)
- (b) $27 : 26 = 00$ - this is a data processing instruction
- (c) $25 = 1$ - there is an immediate present
- (d) $24 : 21 = 0100$ - the **add** function is **0100** (Table 9.1, RHS)
- (e) $20 = 0$ - there is no update to the flags register (**S**)
- (f) $19 : 16 = 0001$ - register **Rs** is **r1**
- (g) $15 : 12 = 0101$ - register **Rd** is **r5**
- (h) $11 : 8 = 0000$ - no rotate right required
- (i) $7 : 0 = 00000010$ - immediate value = 2

Solution: **1110 00 1 0100 0 0001 0101 0000 00000010** (*e2815002*)

2. *Register*: `add r5, r1, r2`

- (a) $31 : 28 = 1100$ - this is unconditional execution (Table 9.1, LHS)
- (b) $27 : 26 = 00$ - this is a data processing instruction

- (c) 25 = 0 - there is no immediate present
- (d) 24 : 21 = 0100 - the **add** function is 0100 (Table 9.1, RHS)
- (e) 20 = 0 - there is no update to the flags register (S)
- (f) 19 : 16 = 0001 - register Rn is r1
- (g) 15 : 12 = 0101 - register Rd is r5
- (h) 11 : 8 = 0000 - hard-coded by the assembler as there is no shift operation
- (i) 7 = 0 - hard-coded by the assembler as there is no shift operation
- (j) 6 : 5 = 0 - hard-coded by the assembler as there is no shift operation
- (k) 4 = 0 - hard-coded by the assembler as there is no shift operation
- (l) 3 : 0 = 0010 - encodes r2 for register Rm

Solution: 1110 00 0 0100 0 0001 0101 0000 0 00 0 0010 (e0815002)

3. *Immediate shifted Register:* **add r5, r1, r2, lsl #2**

- (a) 31 : 28 = 1100 - this is unconditional execution (Table 9.1, LHS)
- (b) 27 : 26 = 00 - this is a data processing instruction
- (c) 25 = 0 - there is no immediate present (the shift immediate does not count)
- (d) 24 : 21 = 0100 - the **add** function is 0100 (Table 9.1, RHS)
- (e) 20 = 0 - there is no update to the flags register (S)
- (f) 19 : 16 = 0001 - register Rn is r1
- (g) 15 : 12 = 0101 - register Rd is r5
- (h) 11 : 8 = 0000 - hard-coded by the assembler as there is no shift operation
- (i) 7 = 0 - hard-coded by the assembler as there is no shift operation
- (j) 6 : 5 = 0 - hard-coded by the assembler as there is no shift operation
- (k) 4 = 0 - hard-coded by the assembler as there is no shift operation
- (l) 3 : 0 = 0010 - encodes r2 for register Rm

Solution: 1110 00 0 0100 0 0001 0101 00010 00 0 0010 (e0815102)

4. *Register Shifted Register:* **add r5, r1, r2, lsl r3**

- (a) 31 : 28 = 1100 - this is unconditional execution (Table 9.1, LHS)
- (b) 27 : 26 = 00 - this is a data processing instruction
- (c) 25 = 0 - there is no immediate present (the shift immediate does not count)
- (d) 24 : 21 = 0100 - the **add** function is 0100 (Table 9.1, RHS)
- (e) 20 = 0 - there is no update to the flags register (S)
- (f) 19 : 16 = 0001 - register Rn is r1
- (g) 15 : 12 = 0101 - register Rd is r5
- (h) 11 : 8 = 0011 - the shift register Rs is r3
- (i) 7 = 0 - hard-coded by the assembler

- (j) 6 : 5 = 00 - `lsl` is encoded as 00
- (k) 4 = 1 - hard-coded by the assembler
- (l) 3 : 0 = 0010 - encodes `r2` for register `Rm`

Solution: 1110 00 0 0100 0 0001 0101 0011 0 00 1 0010 (`e0815312`)

9.3.3 Multiply instructions

In contrast to the non-multiply instructions, ARM7DTI multiply instructions have 10 different parts, two of which are hard-coded by the assembler and cannot be modified by the programmer. Again, we discuss each of them in turn, from the most significant bit (31) to the least significant bit (0). These are discussed in Table ???. Notice that these instructions do not have any `src2` sections.

9.4 Memory Instructions

We will now look at the *memory instruction* class. This class consists of a much smaller number of instructions. There are only *two*. The load register instruction `ldr`, and the store register instruction `str`. However, as you may remember from Chapter 6, there are quite a few variants possible. Prim988

Need to discuss how `sp` is handled. `ldr r5, [ sp ]` generates `e59d5000`
 1110 01 01100 1110 1010 10000000000000

Single-Cycle and Multi-Cycle Microarchitecture

10.1 Overview

Every processor executes the same fundamental sequence of events: fetching an instruction, decoding it, executing it, and saving the result. A **Microarchitecture** is the internal hardware organization that implements the instruction set architecture. From the pedagogical point of view, and for the sake of simplicity, the study of microarchitecture begins the **Single-Cycle Microarchitecture**. Later in the chapter we will refine this model into the **Multi-Cycle Microarchitecture**.

These two approaches are briefly contrasted here:

1. The **Single-Cycle Microarchitecture**, in which each instruction takes the same amount of time to complete, and the clock cycle is the length of the longest instruction path.
2. The **Multi-Cycle Microarchitecture**, in which each instruction is broken into discrete stages, each stage taking one or more clock cycles.

Both designs implement the same ISA, but they trade off simplicity, speed, and hardware utilization in different ways. This chapter explores both approaches in detail, focusing first on a clear and intuitive undergraduate understanding.

10.2 From ISA to Microarchitecture

10.2.1 What the ISA Describes

An **Instruction Set Architecture (ISA)** is the formal contract between software and hardware. It specifies *what* the processor must do when a well-formed instruction executes, without prescribing *how* the processor is built internally. In other words, the ISA is the interface that assembly programmers, compilers, and operating systems target; microarchitectures are free to implement the ISA in many different ways (single-cycle, multi-cycle, pipelined, out-of-order), as long as they preserve the externally visible behavior.

Programmer-visible state. At the heart of an ISA is the *architectural state*: the set of elements a programmer can reason about or observe:

- A defined set of **registers** (general-purpose and special-purpose such as PC, SP, and status/condition flags).
- The **memory address space** and how instructions and data reside in it.
- **System-visible** registers or state (e.g., control/status registers, exception vectors) that affect traps, interrupts, or privilege.

When an instruction completes, the ISA says exactly how this architectural state may change.

Instruction repertoire and semantics. The ISA enumerates the **operations** available (arithmetic/logic, compare, shift, load/store, branch/jump, system/privileged) and gives each one a precise *semantic meaning*. For example, an `add rd, rs1, rs2` updates `rd` with the integer sum of `rs1` and `rs2` modulo the register width and may set condition flags per the ISA's rules. These semantics must be exact enough that different implementations yield identical architectural results for the same program (ignoring explicitly undefined behavior).

Data types and addressing modes. ISAs define the **operand types** (e.g., 8/16/32/64-bit integers, floating point, packed SIMD) and the legal **addressing modes** (register, immediate, base+offset, scaled index, PC-relative, etc.). Addressing modes determine how effective addresses are formed for load/store instructions and how branch targets are computed.

Instruction format and encoding. To be executable, instructions must map to bits. The ISA specifies **encodings**: field widths and positions for opcode, register specifiers, immediates, function bits, and so forth. Some ISAs use fixed-length encodings (common in RISC designs), others use variable-length encodings, or a mixture (e.g., compressed subsets). Encodings matter for toolchains, fetch alignment, and decoding complexity, but do not dictate the internal hardware organization.

Memory and I/O model. Beyond simple reads/writes, the ISA provides a **memory model**: alignment constraints, endianness, atomicity guarantees, and the ordering rules that govern visibility of loads/stores across threads (the *consistency model*). It may also specify architectural mechanisms for **I/O** (memory-mapped I/O regions, special instructions, or system calls) and the rules for interacting with caches and translation (e.g., TLB maintenance instructions).

Control flow and exceptions. ISAs define **control-flow** primitives (conditional branches, indirect jumps, procedure calls/returns) and how **exceptions and interrupts** are raised and handled. This includes vector locations, privilege transitions, and what architectural state is saved/restored on an event. The specification must be precise so that compilers and operating systems can rely on consistent trap and return behavior.

Precision levels: defined, implementation-defined, undefined. A careful ISA distinguishes:

- **Well-defined behavior:** results and side effects are precisely specified (e.g., integer add wraparound).
- **Implementation-defined behavior:** allowed to vary by implementation but must be documented (e.g., latency or optional instruction subsets).
- **Undefined behavior:** intentionally unspecified; programs that rely on it are non-portable (e.g., executing a reserved encoding).

These categories give implementers latitude while keeping the programmer's model stable.

What the ISA *does not* mandate. Crucially, the ISA does *not* require a particular microarchitecture. It says nothing about pipeline depth, forwarding paths, cache sizes, or whether instructions complete in one or many cycles. Nor does it require a specific physical layout, timing closure method, or power-management scheme. All of those are implementation choices hidden beneath the architectural contract.

Related (but distinct) interfaces. Two interfaces sit next to the ISA and are often discussed with it:

- The **Application Binary Interface (ABI)**, which fixes calling conventions, register usage, stack layout, and relocation formats so that compiled code and libraries interoperate.
- The **System interface** (or *privileged architecture*), which specifies supervisor-mode features: virtual memory, page tables, exception levels, and control/status registers.

While the ABI and privileged architecture strongly influence software portability and OS design, they are conceptually layered on top of, or alongside, the base ISA.

Why this boundary matters. Because the ISA is the common contract, it enables diversity beneath (different cores, process nodes, power/performance trade-offs) and stability above (tools, compilers, operating systems, and applications). Understanding precisely *what* the ISA promises, and *what it leaves open*, is essential before studying how single-cycle or multi-cycle microarchitectures realize those promises in hardware.

10.2.2 What the Microarchitecture Implements

If the Instruction Set Architecture (ISA) defines *what* a processor must do, the **microarchitecture** defines *how* it is actually built to do it. It represents the internal organization and flow of data within the CPU, transforming abstract instructions into tangible sequences of hardware operations. While the ISA is fixed and visible to software, the microarchitecture is an engineering design choice that can vary widely among implementations of the same ISA.

Hardware building blocks. A microarchitecture is constructed from three broad categories of hardware components:

- **Storage elements** such as registers, latches, flip-flops, and memory arrays. These components hold the state of the machine: the current instruction, operands, intermediate results, and the program counter.
- **Combinational logic** such as adders, multiplexers, decoders, shifters, and Arithmetic Logic Units (ALUs). These perform the computations, comparisons, and selections that process the stored data.
- **Control logic**, typically implemented as finite state machines or microprograms, that coordinates the movement of data and the sequencing of operations in time.

Freedom of implementation. Two processors may implement the same ISA yet differ dramatically in their microarchitecture. For example:

- A simple educational RISC processor may use a **single-cycle** datapath: each instruction runs from start to finish in one long clock cycle.
- A higher-performance implementation may be **pipelined**, overlapping several instructions so that one completes on every clock tick.
- A superscalar design may contain multiple ALUs, multiple pipelines, and out-of-order execution logic that rearranges instructions dynamically to maximize throughput.

Despite these differences, all of these implementations produce the same observable results, because they obey the same architectural rules defined by the ISA.

Goals and trade-offs. Microarchitecture design involves balancing conflicting objectives:

- **Performance:** how many instructions the processor can execute per second,
- **Area:** the amount of silicon (and thus cost) consumed,
- **Power consumption:** the energy efficiency of the implementation.

Designers can choose simpler or more complex datapaths, deeper or shallower pipelines, and various caching or control techniques, depending on these constraints. The study of single-cycle and multi-cycle designs provides a foundation for understanding these trade-offs before progressing to advanced architectures.

10.2.3 The Instruction Execution Cycle

Regardless of its internal complexity, every processor follows the same general pattern when executing instructions. This sequence, often called the **instruction execution cycle**, divides the process into smaller, logical stages that reveal the flow of information through the datapath.

Stage 1: Instruction Fetch (IF). The processor begins by reading the next instruction from memory. The address of this instruction is stored in the **Program Counter (PC)**. The fetched instruction bits are loaded into an *instruction register*, and the PC is incremented (usually by 4 bytes in a 32-bit RISC architecture) so that it points to the next sequential instruction. In case of a branch or jump, the PC may later be updated with a new target address.

Stage 2: Instruction Decode (ID). The binary pattern of the instruction is examined by the control logic to determine what type of operation it specifies. At the same time, the **register file** is accessed to read the values of the source operands identified by the instruction's register fields. This stage translates “what to do” and “what data to use” into actual signals and data paths within the CPU.

Stage 3: Execute (EX). The operation is performed in the Arithmetic Logic Unit (ALU). Depending on the instruction, this stage may:

- Perform arithmetic or logical computation (e.g., addition, subtraction, AND, OR),
- Compute an effective address for a memory operation (e.g., base + offset),
- Evaluate a branch condition by comparing two register values.

The output of the ALU is an immediate numerical result or an address that will be used in the next stage.

Stage 4: Memory Access (MEM). If the instruction requires interaction with memory, the calculated address from the EX stage is used here.

- For a **LOAD** instruction, data is read from the data memory and temporarily stored in an internal register.
- For a **STORE** instruction, data from a register is written into memory at the computed address.

Instructions that do not access memory (such as arithmetic ones) simply bypass this stage.

Stage 5: Write-Back (WB). Finally, the result of the operation (either the ALU output or the data retrieved from memory) is written back into the destination register. This completes the instruction's lifecycle, returning the processor to a new stable state ready to begin the next fetch.

Putting the stages together. Together, these five stages define the flow of information through the processor. They separate the work of executing an instruction into manageable and reusable parts. The clear boundaries between stages allow engineers to reason about timing, optimize hardware reuse, and later extend the design into multi-cycle or pipelined implementations.



Figure 10.1: The five fundamental stages of instruction execution. Each processor, regardless of internal design, performs these conceptual steps.

► The 5-Stage Pipeline Live

PlayARM animates exactly the five stages above on an interactive canvas. Open PlayARM, enter the program below, and press **Step** (→) once per clock cycle. Watch each instruction bubble move from **IF** → **ID** → **EX** → **MEM** → **WB** across the pipeline canvas.

```

mov r0, #3
mov r1, #7
add r2, r0, r1
sub r3, r1, r0
  
```

Check the **Pipeline Activity** panel on the right to see which instruction occupies each stage at every cycle. Notice that once the pipeline is full, a new result is produced every cycle.

Historical note. This five-stage model originates from early RISC architectures developed in the 1980s (such as MIPS, SPARC, and ARM), where clarity and simplicity of data movement were primary goals. Even today, these same conceptual stages remain the organizing principle for most CPU designs, from simple teaching processors to complex out-of-order superscalar cores.

Conceptual importance. For undergraduate students, understanding this model is essential. It not only clarifies how instructions progress through hardware, but also builds intuition for timing, performance bottlenecks, and later topics such as pipelining and hazard management. Whether a CPU is built as a single-cycle or a deeply pipelined machine, these five stages are the unchanging rhythm of computation.

10.3 Part I: Single-Cycle Microarchitecture

10.3.1 Core Concept

The **single-cycle microarchitecture** represents the simplest complete hardware organization capable of running a full instruction set. Its defining idea is that *each instruction* (from fetching in memory to writing the final result back into a register) is completed entirely within one clock period.

- Every instruction consumes exactly one clock cycle, so the **cycles per instruction (CPI)** equals 1.
- The clock period, however, must be long enough to accommodate the *slowest* instruction in the ISA.

This design is elegant for learning because all the moving parts of a processor can be seen operating once per instruction. The concept helps students

visualize how data flows through the datapath, how control signals coordinate hardware components, and how an instruction's five stages connect. Yet this same simplicity introduces performance limitations: the entire processor must wait for the slowest possible instruction to finish before the next instruction begins.

The “race of the slowest” effect. Imagine three instructions:

- An **add** that only needs the ALU,
- A **LOAD** that uses both the ALU and data memory,
- A **BRANCH** that also computes and tests a target address.

Because the **LOAD** instruction touches more hardware and requires more propagation delay, the clock period must be long enough for it to complete safely. As a result, simpler instructions waste potential performance, idly waiting through that same long cycle.

Despite this inefficiency, the single-cycle model is a crucial foundation. It shows in one unified picture how the instruction fetch, decode, execute, memory, and write-back stages interconnect (concepts that remain valid for all modern CPU designs, regardless of later optimizations).

► Observing the “Race of the Slowest”

Run a mixed program containing both a fast **add** and a slower **ldr** (memory access). Use PlayARM's **Slow** speed setting and compare the **Control Signals** panel for each instruction.

```
mov r0, #10
mov r1, #20
add r2, r0, r1
str r2, [sp, #-4]!
ldr r3, [sp, #0]
cmp r3, r0
```

Notice that **add** activates **RegWrite** and **ALUOp** only, while **ldr** additionally activates **MemRead** and **MemToReg**. In a real single-cycle design, the clock must be slow enough for the **ldr** path even when **add** is running.

10.3.2 Main Hardware Components

Instruction Encoding and Decoding

Before any hardware block can act on an instruction, the CPU must extract meaning from its raw 32-bit binary representation. This process — called **instruction decoding** — is performed simultaneously by the control unit and the datapath the moment the instruction word exits instruction memory.

ARM 32-bit instruction format. Every ARM instruction occupies exactly 32 bits. The hardware slices this word into named fields whose positions

are fixed by the ISA:

Bits	[31:28]	[27:26]	[25:20]	[19:16]	[15:12]
Field	Cond	Class	Op/Funct	Rn	Rd/Rt

Bits	[11:0]
Field	Src2 (Rm or Imm12)

Cond [31:28]: A 4-bit condition code that determines whether the instruction executes at all. The condition flags (N, Z, C, V) set by a prior **cmp** or arithmetic instruction are checked against this field. The code **1110** means *always execute*, which is the default for most instructions.

Class [27:26]: Two bits that broadly classify the instruction: **00** = data-processing (ALU), **01** = memory (load/store), **10** = branch.

Op/Funct [25:20]: Further specifies the operation within the class (e.g., add vs. sub, with or without the S bit that updates flags). The control unit reads these bits to decide which ALU operation to perform and which datapath signals to assert.

Rn [19:16]: The 4-bit address of the *first source register* (also the base register for load/store). This value is wired directly to the **A1** port of the register file.

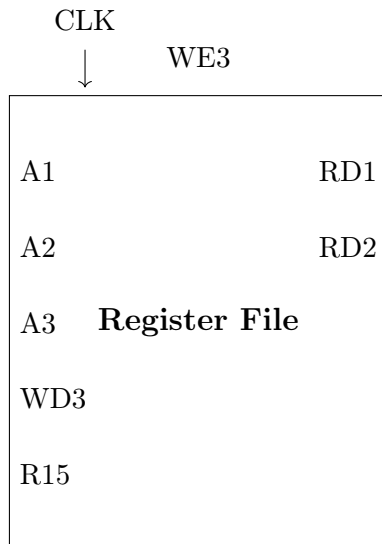
Rd/Rt [15:12]: For data-processing instructions, this is the *destination register* address, wired to **A3**. For load instructions it is the *target register* (where the loaded value is written); for store instructions it is the register whose value is sent to memory. A multiplexer selects the correct interpretation based on the instruction class.

Src2 [11:0]: The second operand, which takes one of two forms:

- **Register form:** bits [3:0] hold **Rm**, the address of the second source register, wired to **A2**. Bits [11:4] can encode a barrel-shifter amount applied to **Rm** before it reaches the ALU.
- **Immediate form:** when bit 25 of the instruction is set, the 12-bit field encodes an 8-bit constant rotated by a 4-bit amount, extending the effective immediate range without requiring a wider instruction word.

A multiplexer (controlled by bit 25) selects whether the ALU's second input comes from the register file or this sign/zero-extended immediate.

Decoding is wiring, not computation. Because every field occupies a fixed bit range, the hardware simply *connects* those wires to the appropriate inputs — no arithmetic is needed. The register-address fields (**Rn**, **Rd**, **Rm**) are connected directly to the address ports of the register file; the **Op/Funct** bits are connected to the control unit's lookup logic. Decoding in a fixed-length RISC design happens in one gate delay, which is one of the key reasons RISC architectures can sustain high clock frequencies.



Register File Ports in Detail

The register file diagram above uses a compact notation; each labeled wire carries a specific signal. Table 10.1 defines every port and maps it back to the instruction fields described above.

Table 10.1: Register File Port Definitions

Port	Width	Direction	Meaning
CLK	1 bit	Input	Clock; write occurs on the rising edge when WE3 is asserted
A1	4 bits	Input	Address of first read port; driven by Rn [19:16] from the instruction
A2	4 bits	Input	Address of second read port; driven by Rm [3:0] or Rt depending on instruction type
A3	4 bits	Input	Address of the write port; driven by Rd [15:12] (or Rt for a load)
WD3	32 bits	Input	Write data; the value to be stored into register A3
WE3	1 bit	Input	Write enable; asserted by the control unit when the instruction writes a result
RD1	32 bits	Output	Data read from the register addressed by A1 ; feeds ALU input 1
RD2	32 bits	Output	Data read from the register addressed by A2 ; feeds ALU input 2 or the store-data path
R15	32 bits	Output	Always outputs the current value of R15 (the PC); used for PC-relative loads and branch-link return addresses

How the ports connect to the datapath.

- RD1 is always routed to ALU input A.
- RD2 reaches a multiplexer: when **ALUSrc** is asserted, the MUX selects the sign-extended immediate instead; otherwise RD2 passes through.

- **WD3** comes from a second multiplexer: when **MemToReg** is asserted (a `ldr` instruction), the data read from memory is selected; otherwise the ALU result is selected.
- **WE3** is asserted for every instruction that modifies a register (data-processing and `ldr`), and de-asserted for `str` and branch instructions.
- **R15** is permanently tied to the PC register output so that instructions such as `ldr PC, [R0]` or `mov lr, pc` can read the current program counter without a separate path.

The read ports (**RD1**, **RD2**) are *asynchronous*: they respond to address changes combinatorially within the same cycle. The write port (**WD3** via **A3**) is *synchronous*: it latches the new value only on the rising clock edge and only when **WE3** = 1. This means a register can be read and written in the same cycle, but the new value is not visible until the *next* cycle — a deliberate choice that preserves the single-cycle semantics.

A minimal single-cycle CPU is composed of a small but complete set of hardware blocks. Each block performs a specific role in the instruction’s journey from memory to execution and back.

Program Counter (PC): A dedicated register that stores the address of the current instruction. After each instruction is fetched, the PC normally increments by 4 bytes (for a 32-bit ISA such as ARM), thereby pointing to the next sequential instruction. In case of a branch or jump, control logic overrides this default update and loads a new address.

Instruction Memory: A read-only memory (or instruction cache) that outputs the 32-bit instruction stored at the address given by the PC. It represents the program itself in machine code form. In hardware, this may be implemented as a simple ROM, flash, or instruction cache line connected to the PC’s output.

Register File: A fast storage array containing all the general-purpose registers defined by the ISA (typically 32 registers for ARM or MIPS-style designs). It allows two source operands to be read simultaneously (**rs1**, **rs2**) and one destination register to be written (**rd**) within the same clock cycle. The register file acts as the primary working area for arithmetic and logic operations.

Arithmetic Logic Unit (ALU): The computational heart of the datapath. It performs integer arithmetic (addition, subtraction, multiplication by shifting) and logical operations (AND, OR, XOR, NOT). The ALU receives its inputs directly from the register file or from an immediate value embedded in the instruction, depending on control signals. Its output is either a final result to be written back or an address used to access data memory.

Data Memory (DMEM): A separate memory module that stores program data (variables, arrays, the stack). Its interface exposes four signals:

- **A** — the 32-bit byte address, produced by the ALU.

- **WD** (write data) — the 32-bit value to store, sourced from register-file output **RD2**.
- **WE** (write enable) — asserted by the control unit for a **str** instruction; de-asserted for a **ldr**.
- **RD** (read data) — the 32-bit value fetched from address **A**, routed to the **WD3** multiplexer of the register file.

Like the register file's write port, **DMEM** writes are synchronous (clocked), while reads are asynchronous. Separating instruction and data memory follows the **Harvard architecture** principle common in RISC designs, allowing simultaneous instruction fetch and data access in the same cycle.

Control Unit: The combinational logic that converts instruction bits into the set of boolean control signals that orchestrate every other component. Given the **Class** [27:26], **Op/Funct** [25:20], and the condition flags from the ALU, the control unit asserts or de-asserts each of the following signals:

Signal	Effect when asserted
RegWrite	Enables write to register file (WE3)
MemWrite	Enables write to data memory (WE on DMEM)
MemToReg	Selects memory read data as write-back source (vs. ALU result)
ALUSrc	Selects sign-extended immediate as ALU second operand (vs. RD2)
RegSrc	Selects which instruction field drives A2 (e.g., Rm vs. Rt)
ImmSrc	Selects the immediate extension scheme (8-bit rotated vs. 12-bit offset)
ALUControl	2–3 bit code telling the ALU which operation to perform
PCSrc	Overrides the default PC+4 update with a branch target

Because these signals are generated combinatorially in a single-cycle design, the control unit is simply a *lookup table*: for each instruction encoding, a fixed pattern of outputs is hard-wired. There is no feedback, no clock, and no state — the signals stabilize within one gate delay of the instruction word arriving.

How these blocks interact. Each component plays its part once per cycle:

1. The PC supplies an address to the instruction memory.
2. The instruction memory outputs the corresponding instruction word.
3. The control unit interprets this instruction and sets the control lines.
4. The register file outputs operands for the ALU.
5. The ALU computes either a data result or an effective address.
6. The data memory is accessed if required.

7. The result is written back into the register file, and the PC updates for the next fetch.

All of these events occur within the same clock period (a remarkable coordination of parallel hardware activity).

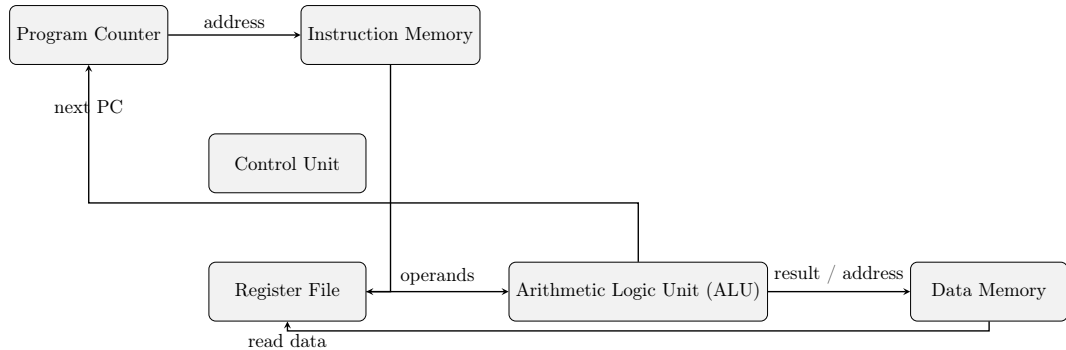


Figure 10.2: Simplified datapath for a single-cycle processor. Each instruction passes through this entire datapath within one clock period.

Why this model matters. For students, the single-cycle architecture is not primarily about performance; it is about comprehension. It reveals, in one unified snapshot, how every instruction flows through the processor and how hardware components cooperate under the guidance of control logic. Understanding this model builds the conceptual foundation required for the multi-cycle and pipelined designs that follow, where the same pieces are reused or overlapped to achieve higher efficiency.

10.3.3 Dataflow of an Instruction (Example)

To understand how all of the processor’s blocks work together, it helps to follow one instruction as it travels through the datapath. Consider a simple arithmetic instruction in a RISC architecture such as ARM or MIPS:

`add R3, R1, R2`

This instruction adds the contents of register **R1** and **R2**, and stores the result in **R3**. Even though it looks simple, it touches nearly every major part of the datapath. Let’s examine each action, step by step, as it occurs during the single clock cycle.

1. **Instruction Fetch (IF).** The **Program Counter (PC)** holds the address of the next instruction. At the beginning of the cycle, the PC’s output is sent to the **Instruction Memory**, which retrieves the 32-bit binary encoding of `add R3, R1, R2`. The fetched instruction is passed simultaneously to the **Control Unit** for decoding and to various parts of the datapath that will soon need its fields.
2. **Instruction Decode and Register Read (ID).** The **Control Unit** interprets the opcode bits and recognizes that this is an `add` instruction (an R-type operation requiring two register operands and one destination register). From the instruction fields, it extracts:

- **rs1** = R1 (first source),
- **rs2** = R2 (second source),
- **rd** = R3 (destination).

The **Register File** simultaneously outputs the values contained in R1 and R2 onto two internal buses. These become the ALU's inputs for this cycle.

3. **Execute (EX)**. Guided by the control signals, the **ALU** is configured to perform an addition operation. It receives the two operand values from the register file and computes the sum:

$$\text{ALU_Result} \leftarrow \text{R1_Value} + \text{R2_Value}.$$

The resulting value now exists on the ALU's output bus, ready to be written back. Because this instruction is purely arithmetic, no memory access is required.

4. **Memory Access (MEM)**. Since the **add** instruction does not involve a load or store, the data memory remains idle during this part of the cycle. Still, the datapath physically includes this stage because other instructions, such as **ldr** or **str**, will use it to interact with data memory.
5. **Write-Back (WB)**. The output from the ALU is sent to the **Register File**'s write port. Control signals assert the **RegWrite** line, and the destination register field (**rd** = R3) determines which register receives the value. On the next rising clock edge, R3 is updated with the sum computed by the ALU.
6. **PC Update**. In parallel with the above operations, the PC's incrementer calculates the address of the next instruction:

$$\text{PC}_{\text{next}} = \text{PC}_{\text{current}} + 4.$$

Unless a branch or jump overrides it, this new value is loaded into the PC at the end of the cycle, ensuring continuous sequential execution.

All six of these actions occur within one clock period. There are no intermediate registers to pause or pipeline the work; data flows smoothly through the combinational hardware of the datapath. This one-cycle “heartbeat” makes the design conceptually easy to trace but electrically demanding: the clock must be slow enough for the longest possible path (from PC through instruction memory, decoding, register file, ALU, and data memory) before the next rising edge arrives.

Key Observation

In a single-cycle processor, **the clock period is determined by the worst-case instruction path**. For example, a **LOAD** instruction must:

1. Fetch the instruction from memory,
2. Decode and read registers,
3. Compute an effective address in the ALU,
4. Access the data memory,
5. Write the loaded value back to the register file.

Because all of this happens in one cycle, the clock must be long enough to accommodate this slowest path. Consequently, even simple arithmetic instructions like **add** must wait through that same long period, limiting the processor's overall performance.

Visualizing the flow. A useful way to imagine the single-cycle datapath is as a continuous conveyor belt. At the start of each cycle, a new instruction enters at one end (via the PC and instruction memory). It passes through the decode and execute stations, possibly touching the data memory, and exits at the other end when its result is written back. Only after the entire belt has moved one full step (after the longest instruction has finished) can the next instruction begin. This visualization highlights both the clarity and the inefficiency of the single-cycle design.

10.3.4 Timing and Performance

In any synchronous processor, performance depends heavily on the duration of a single clock period. During each cycle, signals propagate through combinational hardware (adders, multiplexers, memories) before being latched into storage elements at the next clock edge. For the circuit to operate correctly, the clock period must be at least as long as the total propagation delay along the **longest path** that an instruction can take through the datapath.

Let the propagation delay of each major stage of the instruction cycle be:

$$T_{IF}, T_{ID}, T_{EX}, T_{MEM}, T_{WB},$$

corresponding respectively to instruction fetch, instruction decode, execution, memory access, and write-back. Then the minimum safe clock period is:

$$T_{clk} = T_{IF} + T_{ID} + T_{EX} + T_{MEM} + T_{WB} + T_{margin},$$

where T_{margin} represents extra time to cover setup and hold requirements of flip-flops, wiring delays, and other non-idealities.

The critical path. In a single-cycle microarchitecture, every instruction, no matter how simple or complex, must complete within this one fixed clock period. Therefore, the cycle time must accommodate the *slowest* instruction in the ISA, typically the one that traverses the greatest number of hardware

components. For most RISC processors, this is the **LOAD** instruction, because it includes:

1. Reading the instruction memory (fetch),
2. Decoding and reading registers,
3. Computing an effective address in the ALU,
4. Accessing the data memory,
5. Writing the loaded value back into the register file.

The delay through all of these elements defines the **critical path**, the longest combinational route between two registers in the design.

Clock period and frequency. The clock period T_{clk} directly determines the clock frequency:

$$f_{\text{clk}} = \frac{1}{T_{\text{clk}}}.$$

A longer critical path yields a smaller clock frequency and therefore lower overall performance. Even if 80% of the executed instructions are faster (for example, arithmetic operations that never touch memory), the processor still must wait for the slowest path. All instructions, regardless of complexity, share the same T_{clk} .

Example numerical estimation. Assume approximate stage delays (in picoseconds):

$$T_{\text{IF}} = 200, \quad T_{\text{ID}} = 150, \quad T_{\text{EX}} = 180, \quad T_{\text{MEM}} = 250, \quad T_{\text{WB}} = 40.$$

Including a 30 ps margin for setup and routing, the total clock period becomes:

$$T_{\text{clk}} = 200 + 150 + 180 + 250 + 40 + 30 = 850 \text{ ps}.$$

Hence, the maximum frequency is:

$$f_{\text{clk}} = \frac{1}{850 \times 10^{-12}} \approx 1.18 \text{ GHz}.$$

This frequency applies to *every* instruction, even though most do not require the full 850 ps of work.

Implications for CPI and instruction time. In a single-cycle processor, the cycles-per-instruction (**CPI**) is fixed at 1, since each instruction completes in one cycle. Therefore, the average time per instruction is simply:

$$T_{\text{instr}} = \text{CPI} \times T_{\text{clk}} = 1 \times T_{\text{clk}} = T_{\text{clk}}.$$

This makes timing analysis conceptually simple but leaves little room for optimization: once the clock period is set by the slowest instruction, no other instruction can execute faster.

Performance trade-offs. The simplicity of single-cycle timing brings two clear trade-offs:

- **Predictability:** Every instruction finishes in one cycle, simplifying control logic and performance analysis.
- **Inefficiency:** The clock must remain slow enough for the most complex instruction. Faster instructions waste time because they could have completed earlier but must wait for the fixed clock boundary.

Why this matters. Understanding timing at this level helps explain the motivation for more advanced architectures. The next chapter (multi-cycle design) breaks these same five stages into separate, shorter cycles, allowing the clock to run faster while reusing hardware between cycles. Later, pipelined designs will overlap multiple instructions in flight, turning the single-cycle bottleneck into a high-throughput engine. Nevertheless, the single-cycle timing equation above remains the starting point for all such performance reasoning.

10.3.5 Advantages and Disadvantages

Every engineering design involves trade-offs, and the single-cycle microarchitecture is no exception. Its conceptual simplicity makes it a favorite model for learning and experimentation, but that same simplicity comes at a cost in speed and scalability. The table below summarizes the major benefits and drawbacks, followed by a deeper discussion of each.

Advantages:

- **Conceptual clarity and simplicity.** Each instruction executes from start to finish in one clock cycle, making it easy to visualize how data moves through the processor. There are no pipeline hazards, control states, or overlapping stages to worry about; just one clean “pulse” of activity per instruction. This makes the single-cycle design ideal for students, educators, and hardware beginners to understand the relationships among the PC, datapath, and control signals.
- **Predictable timing.** Since every instruction consumes exactly one cycle ($CPI = 1$), timing analysis is straightforward. Execution time depends solely on the clock period, which in turn is determined by the slowest instruction’s path. This predictability simplifies debugging, verification, and performance measurement.
- **Ease of implementation on small platforms.** The single-cycle architecture maps neatly onto FPGA boards and educational simulators. Designers can implement it with minimal control logic, and the lack of sequential states means fewer synchronization errors. It is often used in undergraduate laboratories and early research prototypes to illustrate the essence of a working CPU.
- **Direct correspondence between ISA and hardware.** Because the datapath executes one instruction end-to-end, the mapping from ISA operations to hardware activity is intuitive. Each instruction’s effect can be traced visually through a single diagram, useful for teaching how assembly

code translates to electrical operations.

Disadvantages:

- **Long clock period (critical-path limitation).** The clock period must be long enough for the slowest instruction (typically a `LOAD` or `STORE`) to pass through the entire datapath. Simpler instructions, like `add` or `AND`, complete their work much faster but still must wait until the end of that long cycle. This lowers the maximum achievable frequency and wastes potential performance.
- **Inefficient hardware utilization.** Many hardware units sit idle during parts of the cycle. For instance, the data memory is only active for load and store instructions, yet it occupies silicon area and power in every cycle. Similarly, the ALU is unused when fetching or decoding instructions. This “all-at-once” execution means more hardware is built than is actively needed at any given moment.
- **Poor scalability for complex ISAs.** As instruction sets grow to include multi-cycle operations (e.g., division, floating-point, system calls), forcing every instruction to fit into a single clock period becomes impractical. Either the clock must be slowed dramatically, or specialized multi-cycle control must be added, defeating the simplicity that made the design attractive in the first place.
- **High power consumption at a given performance level.** Because all units switch on every cycle, even when not needed, the design wastes dynamic power. Real processors achieve energy efficiency by reusing or idling portions of the datapath between stages, something a single-cycle design cannot easily do.
- **No path to advanced optimizations.** The single-cycle model does not provide natural insertion points for pipelining or overlapping instructions. It serves well as a conceptual baseline, but practical processors must evolve to multi-cycle or pipelined structures to achieve competitive throughput.

Summary of the trade-off. In essence, the single-cycle processor favors *clarity* over *efficiency*. It is ideal for learning, verification, and small embedded use, but limited by the physical timing constraints of its longest instruction path. Engineers rarely use it in real commercial processors, yet every sophisticated CPU, no matter how complex, can still be understood as an evolution of this simple design.

10.4 Part II: Multi-Cycle Microarchitecture

10.4.1 Motivation

The single-cycle microarchitecture, while conceptually elegant, suffers from a fundamental performance limitation: the clock period must be long enough for the *slowest* instruction in the instruction set. This means that even the simplest operations are forced to wait through an unnecessarily long cycle. The processor performs many operations sequentially within that one period

(fetching, decoding, computing, accessing memory, and writing results) when most of those steps do not need to occur simultaneously.

Breaking the long cycle. The key idea behind the **multi-cycle microarchitecture** is to divide the work of executing an instruction into several smaller steps, each performed in its own clock cycle. By doing so, each cycle now contains fewer operations, the combinational delay per cycle becomes much shorter, and the clock can run at a higher frequency. Instead of completing one instruction in one long cycle, we complete it over several short cycles.

Single-cycle: One long step per instruction $\Rightarrow$ Multi-cycle: Several short steps per instruction

The total time per instruction might still be comparable, but the hardware is used more efficiently and scales better with complex operations.

Reusing hardware across cycles. In the single-cycle design, every major unit (such as the ALU, the instruction memory, and the data memory) was active once per instruction. This required separate hardware blocks for each role: one memory for instructions and another for data, one adder for PC incrementing and another inside the ALU, and so on. In a multi-cycle design, the same hardware resources can be **reused** across different cycles of the same instruction.

For example:

- The **ALU** can first compute $PC + 4$ during the fetch cycle and later compute an effective address for a **LOAD**.
- A single **memory unit** can be used both for fetching instructions (during one cycle) and for accessing data (during a later cycle).

This reuse greatly reduces hardware cost and power consumption, especially for teaching processors or small embedded designs.

Shorter clock, variable instruction time. By restricting each cycle to a smaller amount of logic, we can make the clock much faster. However, because different instructions may require different numbers of steps, the number of cycles per instruction (**CPI**) is no longer fixed at 1. A simple arithmetic instruction might take three cycles (fetch, decode, execute), whereas a load instruction might take five (fetch, decode, address calculation, memory access, write-back).

The average instruction time therefore becomes:

$$T_{\text{avg}} = \text{CPI}_{\text{avg}} \times T_{\text{cycle}},$$

where T_{cycle} is now much shorter than in the single-cycle case, but $\text{CPI}_{\text{avg}} > 1$. The overall performance gain depends on whether the product of these two terms is smaller than the single-cycle clock period.

Example illustration. Suppose the single-cycle CPU has a long clock period of 850 ps (set by the slowest load instruction). In a multi-cycle version, the work might be divided into five shorter steps, each with a cycle time of 250 ps. A load instruction might require all five cycles ($5 \times 250 = 1250$ ps), but a simple add might finish in three ($3 \times 250 = 750$ ps). While the slow instruction becomes slightly slower, the frequent short ones become significantly faster, improving average throughput.

Architectural implications. Moving to a multi-cycle design introduces several new engineering considerations:

- The datapath must now include temporary **intermediate registers** to hold values between cycles (such as the instruction, operands, and ALU results).
- The **control unit** must evolve from a simple combinational decoder to a finite state machine (FSM) that sequences through the proper control signals cycle by cycle.
- Timing analysis must now consider per-cycle critical paths rather than a single end-to-end delay.

Educational value. From a pedagogical standpoint, the multi-cycle microarchitecture marks an essential transition point:

- It introduces the concept of **time decomposition**: executing an instruction as a sequence of well-defined stages.
- It builds intuition for **hardware reuse** and the importance of control sequencing.
- It lays the groundwork for understanding **pipelining**, where these same stages are later overlapped across multiple instructions to achieve higher throughput.

In short, the motivation for the multi-cycle design is not merely performance; it is about engineering efficiency. It teaches how to achieve the same architectural behavior using fewer resources, shorter cycles, and more sophisticated control, setting the stage for real-world processor architectures.

10.4.2 Additional Internal Registers

When execution of a single instruction is stretched across multiple clock cycles, the processor must retain intermediate results between cycles. In the single-cycle design, temporary values naturally flow through combinational hardware within one long cycle and are not stored anywhere. However, once execution is split into distinct short cycles, the datapath needs dedicated **intermediate registers** to “remember” partial results that will be used in later cycles.

Purpose of internal registers. These internal registers serve as short-term holding areas between the main stages of the instruction cycle. They decouple the combinational logic of one stage from the next, allowing the processor to

reuse hardware components (such as the ALU or memory) over time without losing intermediate values. Effectively, they act as “checkpoints” between cycles, enabling the CPU to pause and continue the same instruction safely on the next clock edge.

The most common intermediate registers are described below.

- **IR (Instruction Register):** During the *fetch* cycle, the memory outputs the instruction word addressed by the Program Counter (PC). To avoid losing this instruction when the memory is reused for data access in later cycles, the instruction must be stored in a dedicated register called the **Instruction Register (IR)**. The control unit reads the opcode and other fields from the IR in subsequent cycles to generate appropriate control signals. This separation allows the memory to be reused for data operations while the instruction remains safely latched.
- **MDR (Memory Data Register):** The **Memory Data Register** (sometimes called *Memory Buffer Register*) holds data that has been read from, or is about to be written to, the memory. When a **LOAD** instruction fetches a word from memory, the value first enters the MDR, from which it will later be written into the destination register. Similarly, during a **STORE**, the value to be written is placed into the MDR before the actual memory write occurs. This mechanism stabilizes the data and provides a clean interface between the CPU core and the memory subsystem.
- **A and B (Operand Registers):** In the multi-cycle design, the **Register File** is accessed only during the *decode* stage. Once the operands are read, they are stored in two temporary registers, traditionally named **A** and **B**. These hold the source operands for use in later cycles, particularly during execution or memory address calculation. By capturing operand values early, the ALU can perform operations in later cycles even when the register file is busy with other tasks.
- **ALUOut (ALU Output Register):** After the ALU performs a computation (such as evaluating an address for a load/store or producing the result of an arithmetic operation), the result is latched into the **ALUOut** register. This stored value then becomes available for use in subsequent cycles. For example, during a **LOAD**, the address calculated in one cycle (stored in ALUOut) will be used in the next cycle to access memory. Similarly, for arithmetic instructions, ALUOut holds the final result until the write-back stage.

How these registers enable hardware reuse. Without these intermediate registers, each instruction would need its own dedicated ALU and memory path active simultaneously. By introducing them, the same hardware can perform multiple roles in different cycles:

- In the first cycle, the ALU computes $PC + 4$ (to advance to the next instruction).
- In the next cycle, the same ALU computes an effective address (**base** + **offset**) for a data access.
- Later, the same ALU might perform an arithmetic operation (**R1** + **R2**).

Each intermediate result is stored temporarily (in **ALUOut**, **IR**, or **MDR**) until it is needed again. Thus, the addition of these small registers transforms the datapath from a “one-pass conveyor belt” into a flexible, time-multiplexed machine.

Analogy. An intuitive way to visualize this is to imagine a small workshop where only one worker (the **ALU**) and one toolbox (the memory) are available. To build something complex, the worker performs one step at a time, saving intermediate parts on the workbench (the internal registers) before moving to the next step. This strategy allows the same tools to be reused efficiently instead of buying new ones for every task.

Design note. These registers are not visible to programmers; they belong to the microarchitecture, not the ISA. They do not correspond to architectural registers like **R0–R31**, but exist purely to coordinate timing and resource reuse. Their correct operation depends entirely on the sequencing logic of the control unit, which determines when each one should be loaded or read during instruction execution.

In summary, adding the **IR**, **MDR**, **A**, **B**, and **ALUOut** registers is what makes a multi-cycle design possible. They store the right pieces of information at the right times, enabling a single **ALU** and a single mem

10.4.3 State-by-State Operation

In the multi-cycle microarchitecture, the processor executes each instruction as a sequence of **states**, where each state corresponds to one short clock cycle. During a given state, specific control signals are asserted, allowing one well-defined operation (such as fetching an instruction, reading registers, or performing an **ALU** computation) to take place. At the end of each cycle, intermediate results are stored in internal registers (**IR**, **A**, **B**, **ALUOut**, **MDR**) to be used in the next state.

A typical controller for a simple RISC-style processor (such as MIPS or ARM-like subset) can be implemented with five key states. This model captures all the essential behavior of arithmetic, memory, and branch instructions, while demonstrating how hardware is reused efficiently from one cycle to the next.

State 0 – Instruction Fetch (IF). The processor begins every instruction by fetching its binary encoding from memory. The address used is the current value of the Program Counter (**PC**). During this cycle:

- The memory address input is connected to the **PC**.
- The memory performs a read operation, and the 32-bit instruction word is retrieved.
- The fetched instruction is latched into the **Instruction Register (IR)** so that it will be available even when the memory is later reused for data access.

- The ALU, operating in parallel, adds 4 to the current PC value, producing $PC + 4$. This result is stored back into the PC in preparation for sequential execution.

Control summary:

- Inputs: $PC \rightarrow$ Memory address bus
- Outputs: Memory data $\rightarrow$ IR
- Operations: ALU performs $PC + 4$; PC updated

This state uses both the memory and the ALU, but these components will later be reused for other functions. The IR now holds the instruction for subsequent decoding.

State 1 – Instruction Decode and Register Read (ID). With the instruction safely stored in the IR, the processor next determines what kind of operation is required. The control unit inspects the opcode and function fields, and simultaneously, the register file is read to obtain source operands.

- The two source registers specified in the IR (**rs1** and **rs2**) are read from the register file.
- Their values are stored into temporary registers **A** and **B**, respectively.
- If the instruction is a branch, the target address can be partially computed by adding the sign-extended immediate field to $PC + 4$. This helps reduce latency in the next state.

Control summary:

- IR opcode bits drive control decoding.
- Register file read enabled; $A \leftarrow R[rs1]$, $B \leftarrow R[rs2]$.

At the end of this state, the processor has decoded the instruction type and prepared all necessary operands for the execute stage.

State 2 – Execution / Address Calculation (EX). This state performs the core arithmetic or address computation required by the instruction. Different instruction types use the ALU for different purposes:

- **Arithmetic/Logic instructions (R-type):** The ALU executes the operation specified by the function field in the IR. Example:

$$ALUOut \leftarrow A \text{ op } B,$$

where **op** might be addition, subtraction, AND, OR, etc.

- **Load/Store instructions:** The ALU computes an effective address by adding a base register (**A**) to the sign-extended offset from the instruction. Example:

$$ALUOut \leftarrow A + \text{SignExt}(\text{Immediate}).$$

- **Branch instructions:** The ALU performs a subtraction ($A - B$) to check for equality. The Zero output of the ALU becomes a condition flag that

determines whether the PC should be updated with the branch target address.

All of these operations reuse the same ALU hardware, demonstrating efficient resource sharing. The result of this computation is stored in **ALUOut**, ensuring it remains available for the next state.

Control summary:

- ALU operation selected based on opcode/function.
- Destination: $\text{ALUOut} \leftarrow \text{ALU result}$.

State 3 – Memory Access (MEM). In this state, the processor interacts with the data memory: either reading from it (for loads), writing to it (for stores), or updating the PC (for branches).

- **Load instruction:** The address stored in ALUOut is sent to the memory address bus. The memory performs a read, and the returned data word is stored in the **Memory Data Register (MDR)**.
- **Store instruction:** The data value stored in temporary register B is sent to the memory's data input, while ALUOut provides the target address. A write signal is asserted, causing the memory to store the value.
- **Branch instruction:** If the ALU's zero flag from the previous state is true (meaning the branch condition is satisfied), the PC is updated with the branch target address computed earlier. Otherwise, the PC retains its sequential value ($\text{PC} + 4$).

Control summary:

- For load: $\text{MemRead}=1$, $\text{MDR} \leftarrow \text{Mem}[\text{ALUOut}]$.
- For store: $\text{MemWrite}=1$, $\text{Mem}[\text{ALUOut}] \leftarrow \text{B}$.
- For branch: Conditional PC update based on Zero flag.

At this point, the memory access or control flow change has been completed. Arithmetic instructions skip this stage because they do not involve data memory.

State 4 – Write-Back (WB). The final state of the instruction returns results to the architectural register file, making them visible to software. Depending on the instruction type:

- **R-type arithmetic/logic instructions:** The result previously stored in ALUOut is written to the destination register specified by the IR's **rd** field.

$$R[\text{rd}] \leftarrow \text{ALUOut}.$$

- **Load instructions:** The data fetched from memory, stored in the MDR during the previous cycle, is written back to the destination register:

$$R[\text{rd}] \leftarrow \text{MDR}.$$

No write-back occurs for store or branch instructions, since their results are either memory-side effects or PC updates.

Control summary:

- RegWrite signal asserted.
- Source of data selected: ALUOut (for ALU ops) or MDR (for loads).
- Destination register determined by `rd` field in IR.

After this state, the instruction is complete, and the control unit returns to State 0 to begin fetching the next instruction.

Putting the states together. These five states (IF, ID, EX, MEM, and WB) form the **control sequence** for all instructions. Each instruction type follows a different path through this state machine:

- Arithmetic instructions: IF → ID → EX → WB
- Load instructions: IF → ID → EX → MEM → WB
- Store instructions: IF → ID → EX → MEM
- Branch instructions: IF → ID → EX → MEM (conditional PC update)

Although each instruction requires multiple cycles, each cycle is short and efficient, and the same hardware is reused across states. This sequencing demonstrates the key virtue of the multi-cycle architecture: **doing more with less**, by carefully orchestrating time and control.

10.4.4 Advantages of the Multi-Cycle Approach

1. Shorter Clock Period. Each cycle includes only a portion of the datapath, so the clock period can be reduced dramatically.

2. Hardware Reuse. The same ALU and memory can be used across several cycles, saving area and cost.

3. Flexibility. Different instruction types can take different numbers of cycles.

4. Foundation for Pipelining. Splitting the instruction execution into well-defined steps sets the stage for overlapping those steps later.

10.4.5 Performance Comparison

Let T_{single} be the long single-cycle clock period and T_{multi} be the short multi-cycle period.

If the average number of cycles per instruction (CPI) is 4, and

$$T_{\text{multi}} = \frac{1}{3}T_{\text{single}},$$

then the average instruction time is:

$$4 \times \frac{1}{3}T_{\text{single}} = \frac{4}{3}T_{\text{single}}.$$

So even though each instruction takes more cycles, the shorter period nearly compensates. More importantly, this design scales better as instruction complexity increases.

10.4.6 Comparison Summary

Table 10.2: Single-cycle vs. Multi-cycle Comparison

Feature	Single-Cycle	Multi-Cycle
Clock period	Long (slowest instruction)	Short (per stage)
Cycles per instruction	1	Variable (3–5 typical)
Hardware reuse	Limited	High (same ALU reused)
Control	Simple, combinational	FSM-based, more complex
Performance scaling	Poor	Better for complex ISAs
Hardware cost	Higher	Lower (shared resources)

10.5 Putting It All Together

10.5.1 Evolution Toward Pipelining

Both architectures teach the same principle: instructions naturally divide into stages. In single-cycle, all stages happen sequentially within one period; in multi-cycle, stages occur over multiple periods; and in pipelining, stages overlap across multiple instructions.

Thus, multi-cycle design serves as the conceptual bridge between simple single-cycle and advanced pipelined processors.

► See the Pipeline Fill Up

Run a longer program and use PlayARM's **Play** button with **Normal** speed. Watch the **Pipeline Activity** panel after the first few cycles the pipeline becomes *full*: five different instructions occupy all five stages simultaneously, producing one result per cycle.

```

mov R0, #1
mov R1, #2
add R2, R0, R1
add R3, R1, R2
add R4, R2, R3
sub R5, R4, R0
cmp R5, R3
bne #-4

```

Count how many cycles pass before the first **add** reaches **WB**. Then observe how subsequent instructions complete every single cycle after that this is the throughput advantage of pipelining over single-cycle and multi-cycle designs.

10.6 Summary

- The **single-cycle** design executes one instruction per long cycle. It is conceptually simple but inefficient for complex ISAs.
- The **multi-cycle** design divides execution into shorter cycles, allowing hardware reuse and a faster clock.
- Both achieve functional correctness and implement the same ISA, but they differ significantly in performance and complexity.

Understanding these two models is essential before studying pipelining, which builds directly upon the cycle-based decomposition introduced here.

10.7 Exercises

Exercise 10.7.1 (Signal Control Mapping) Draw a table listing, for each instruction type (*R-type*, *Load*, *Store*, *Branch*), which control signals are asserted in each stage for both single-cycle and multi-cycle architectures.

Exercise 10.7.2 (Critical Path Calculation) Given component delays: $IMEM = 200\text{ ps}$, $RegFile = 120\text{ ps}$, $ALU = 150\text{ ps}$, $DMEM = 250\text{ ps}$, $WB = 40\text{ ps}$, estimate the clock period and frequency for single-cycle vs multi-cycle designs.

Exercise 10.7.3 (Datapath Sketch) Sketch a simplified datapath showing all key signals (*PC*, *IR*, *A*, *B*, *ALUOut*, *MDR*) and explain the data flow of a *Load* instruction in the multi-cycle design.

Exercise 10.7.4 (Hardware Efficiency) Why does hardware reuse matter in real-world CPU design? Give two examples where hardware reuse helps reduce chip area or energy consumption.

Exercise 10.7.5 (Conceptual Bridge) Explain in your own words why the multi-cycle architecture is considered the “bridge” between single-cycle and pipelined designs.

Pipeline Microarchitecture

Learning Objectives

By the end of this chapter you should be able to:

- Explain how pipelining increases throughput without reducing individual instruction latency.
- Identify RAW, WAW, and WAR data hazards in an instruction sequence and determine which require hardware intervention.
- Trace the forwarding multiplexer select signals (**ForwardAE**, **ForwardBE**) for any pair of consecutive instructions.
- Explain why a **ldr** followed immediately by a dependent instruction cannot be resolved by forwarding, and describe the stall mechanism that fixes it.
- Derive the stall and flush signals (**StallF**, **StallD**, **FlushE**, **FlushD**) from the complete Hazard Unit logic.
- Compute effective CPI given load-use hazard and branch misprediction rates.

11.1 Overview and Motivation

The single-cycle and multi-cycle designs studied in Chapter 10 execute one instruction at a time. Even the multi-cycle design, which shortens the clock period by breaking execution into stages, still completes one instruction before beginning the next. **Pipelining** breaks this constraint: by overlapping the execution of multiple instructions, the processor keeps all five stages busy simultaneously, dramatically increasing throughput.

The five execution stages (Fetch, Decode, Execute, Memory, and Write-Back) each operate on different hardware resources. Once an instruction leaves the Fetch stage, that stage is idle while the instruction moves to Decode. A pipelined processor immediately fills that idle slot with the next instruction's fetch. Up to five instructions can therefore be *in-flight* at any moment, one in each stage.

An assembly-line analogy. A car factory is a useful comparison. Rather than one team of workers building each car from start to finish before touching the next, a modern factory splits the work into stations: one bolts on the engine, the next paints the body, the next installs the electronics, and so on. As soon as a car moves from station 1 to station 2, a fresh car rolls in behind

it. At peak capacity, every station is working on a different car at the same time. The factory's *throughput* (cars finished per hour) is far higher than it would be if each car were built sequentially from scratch, even though the time to build any individual car has not changed.

A pipelined processor works the same way. Each pipeline stage is a station and each instruction is a car. Once the pipeline is full (after the first five cycles), one instruction finishes every single clock cycle.

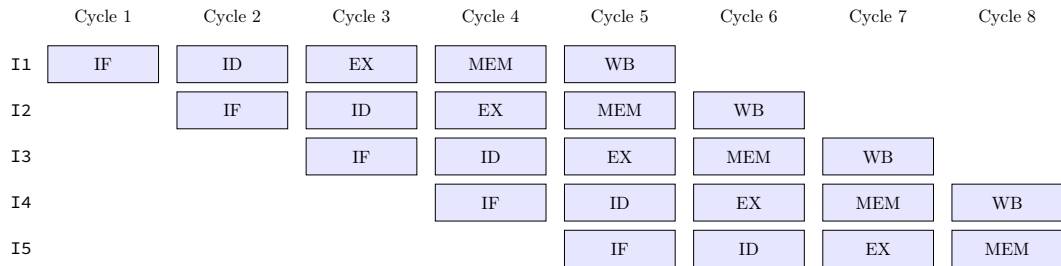


Figure 11.1: **Pipeline execution diagram.** Five instructions fill the five stages; once the pipeline is full, every stage is active every clock cycle.

Throughput vs. latency. Pipelining does *not* make any single instruction execute faster. Its latency (the number of cycles from fetch to write-back) stays at five. What changes is **throughput**: once the pipeline is full, one instruction completes every cycle, giving a steady-state CPI of 1. Compare this with the single-cycle design (CPI = 1 but a very long clock period) and the multi-cycle design (CPI $\approx$ 4–5, shorter period).

Pipelining Improves Throughput, Not Latency

A common misconception is that pipelining makes instructions run faster. It does not. An `add r1, r2, r3` still passes through all five stages and takes five clock cycles from fetch to write-back. What pipelining improves is **throughput**: once the pipeline is full, one instruction *completes* every cycle, even though each instruction individually still takes five cycles. Back to the car factory: adding more workers to each station does not build any car faster, but more cars come off the line per hour.

The cost: hazards. Overlapping instructions creates **hazards**, situations where a later instruction needs a result or a decision that an earlier instruction has not yet produced. The three classes of hazards (data, control, and structural) and their solutions make up most of this chapter.

11.2 The Five Pipeline Stages

We design a pipelined processor by subdividing the single-cycle processor into five pipeline stages and separating them with **pipeline registers** that store the signals between stages. Each stage is named by a single letter that becomes a suffix on all signals within it: **F** (Fetch), **D** (Decode), **E** (Execute), **M** (Memory), **W** (Write-back).

Fetch (F): The processor reads the 32-bit instruction from instruction memory at the address held in the Program Counter (PCF). `PCPlus4F` is computed and written back to the PC register so that the next instruction's address is ready for the following cycle.

Decode (D): The instruction word (`InstrD`) is examined by the control unit, which produces all control signals for the instruction. Simultaneously, the register file is read: bits [19:16] of `InstrD` select register A1, and bits [3:0] select register A2, producing `RD1D` and `RD2D`. The 12-bit immediate field is sign- or zero-extended to 32 bits (`ExtImmD`).

Execute (E): The ALU receives its two inputs (which may be forwarded from later stages; see Section 11.6) and computes a result (`ALUResultE`). For load/store instructions the result is the effective memory address; for arithmetic instructions it is the final value. Branch conditions are also evaluated here.

Memory (M): If the instruction is a `LDR`, data memory is read at `ALUResultM` to produce `ReadDataM`. If the instruction is a `STR`, `WriteDataM` is written to that address. All other instructions pass through this stage without touching memory.

Write-back (W): The result, either `ALUResultW` for arithmetic or `ReadDataW` for a load, is selected by the `MemToReg` multiplexer and written into the destination register (`WA3W`) of the register file. `RegWriteW` must be asserted by the control unit for the write to occur.

11.2.1 Tracing One Instruction Through All Five Stages

The following walkthrough follows `add r1, r2, r3` from fetch to write-back, one stage at a time. It shows exactly what each piece of hardware does and why signals must travel with the instruction through every pipeline register. Assume the instruction enters the Fetch stage in cycle 1.

Cycle 1, Fetch (IF): The processor reads the 32-bit instruction word from instruction memory at the address held in `PCF`. The encoding `0xE0811002` (condition AL, opcode ADD, `Rn=r2`, `Rd=r1`, `Rm=r3`) is placed into the IF/ID pipeline register. In the same cycle, `PCPlus4F = PCF + 4` is written back to the PC so the next instruction's address is ready. Note that the processor has not yet interpreted the instruction at this point; it is just a 32-bit pattern.

Cycle 2, Decode (ID): The control unit reads `Op[27:26] = 00` (data-processing group) and `Funct[24:20] = 00100` (ADD) from `InstrD`. It asserts `RegWrite=1`, `ALUSrc=0` (register second operand, not immediate), and `ALUControl = ADD`. The register file is read at the same time: bits [19:16] = `0010` select `r2`, giving `RD1D`; bits [3:0] = `0011` select `r3`, giving `RD2D`. The destination address `WA3D = 0001` (register `r1`) comes from bits [15:12]. The data values `RD1D`, `RD2D`, control signals, and `WA3D` are all latched into the ID/EX pipeline register.

Cycle 3, Execute (EX): The ALU receives **SrcAE** (the value of **r2**, possibly forwarded) and **SrcBE** (the value of **r3**). It computes **ALUResultE** = **r2** + **r3** and sets the condition flags (**N**, **Z**, **C**, **V**). This is the cycle where the result is ready. Any subsequent instruction that needs **r1** and has a forwarding path can receive it starting next cycle. **ALUResultE**, **WA3E** = 1, and the remaining control signals move into the EX/MEM pipeline register.

Cycle 4, Memory (MEM): Since this is an **ADD** instruction (not **ldr** or **str**), no data memory access occurs. **ALUResultM** passes through unchanged. **WA3M** = 1 and the control signals move into the MEM/WB pipeline register. Meanwhile, the Fetch stage is loading a completely different instruction. The **ADD** knows nothing about this.

Cycle 5, Write-back (WB): **MemToReg** = 0, so the result multiplexer picks **ALUResultW** as **ResultW**. With **RegWriteW** = 1, the register file writes **r2** + **r3** into register **r1** (selected by **WA3W** = 1). The instruction is now complete. The write happens on the falling clock edge, so any instruction simultaneously in Decode that reads **r1** will see the new value in the second half of this cycle.

Why WA3 Must Travel Through Every Stage

The destination register address **WA3** starts as **WA3D** in Decode and must be carried through the ID/EX, EX/MEM, and MEM/WB pipeline registers to reach Write-back as **WA3W**. If the hardware read **InstrD[15:12]** directly at Write-back time, it would get whatever instruction happens to be in Decode at that moment, which is a completely different instruction. The write would land in the wrong register. Every signal produced in one stage but consumed in a later one must be pipelined with its instruction for exactly this reason.

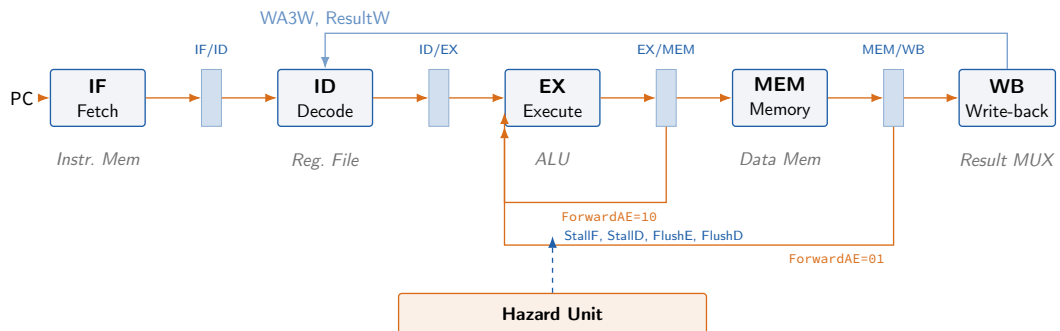


Figure 11.2: **Pipelined ARM datapath.** Pipeline registers (shaded blue) latch all signals between stages; orange forwarding paths route ALU results back to bypass the register file; the Hazard Unit (bottom) drives stall and flush signals.

► The Pipeline Live

Enter the following program and press **Step** repeatedly. Watch each instruction bubble move left-to-right through the **Pipeline Activity** panel.

```
ldr r2, [r0, #40]
add r3, r9, r10
sub r4, r1, r5
and r5, r12, r13
str r6, [r1, #20]
orr r7, r11, #42
```

Once the pipeline is full (after cycle 5), notice that a new instruction completes every single cycle. This is the throughput advantage of pipelining.

11.3 Pipelined Datapath

11.3.1 Pipeline Registers and Signal Naming

Between every adjacent pair of stages sits a **pipeline register** that captures all signals needed by the downstream stage. The four pipeline registers are named **IF/ID**, **ID/EX**, **EX/MEM**, and **MEM/WB**. On every rising clock edge, each register latches the outputs of the stage to its left and presents them to the stage on its right.

Every signal carries a stage suffix to make its position in the pipeline explicit. For example, the write-address for the register file starts as **WA3D** (in Decode), is passed through the pipeline registers to become **WA3E**, then **WA3M**, and finally **WA3W** in Write-back. This naming discipline makes it immediately clear, for any signal, exactly which stage's value it represents.

A fundamental rule follows from this: *all signals associated with a particular instruction must advance through the pipeline in unison*. Using, say, a Decode-stage address together with a Write-back-stage value produces incorrect results.

The Pipeline Register Rule

Every signal that belongs to an instruction must travel through the pipeline registers alongside that instruction, advancing by exactly one stage per clock cycle. Never use a signal that has already been overwritten by a newer instruction in an earlier stage. The stage suffix on every signal name (F, D, E, M, W) makes this rule visible in the code: only mix signals with the same suffix.

11.3.2 Pipelining the Write Address (WA3)

A subtle bug in a naive pipelined datapath illustrates why signal synchronization matters.

Consider the instruction `ldr r2, [r0, #40]` followed by `and r5,`

r12, r13. The write address for the load (**r2**, i.e., **WA3 = 2**) comes from bits [15:12] of the instruction. In the Write-back stage the correct destination is **r2**, but if the write address is taken directly from the Decode stage (**InstrD[15:12]**), by the time the load reaches Write-back, **InstrD** holds the *and* instruction, whose destination is **r5**. The result of the load would be incorrectly written to **r5**.

Fix: pipeline WA3 alongside the instruction. The destination register address must travel through all pipeline registers together with the data:

$$\text{WA3D} \xrightarrow{\text{ID/EX}} \text{WA3E} \xrightarrow{\text{EX/MEM}} \text{WA3M} \xrightarrow{\text{MEM/WB}} \text{WA3W}$$

In the Write-back stage, **WA3W** and **ResultW** are fed back together to the register file. This ensures the correct register is always written, regardless of what other instructions are simultaneously in earlier stages.

Signal Synchronization: The Most Common Pipeline Design Mistake

Forgetting to pipeline a signal alongside its instruction is one of the most frequent mistakes when first building a pipelined datapath. If a signal is *produced* in one stage but *consumed* in a later stage, it must travel through every pipeline register between them. The stage suffix on signal names (F, D, E, M, W) is the guide: signals with mismatched suffixes on opposite sides of an operation almost always indicate a bug.

11.3.3 Register File Write and Read Timing

In the pipelined processor the register file is *written* in the Write-back stage and *read* in the Decode stage, and both can happen in the *same cycle* once the pipeline is full.

To make this work safely, the register file is designed so that writes occur on the **falling edge** of CLK and reads occur in the **second half** of the cycle (after the write has settled). This means a value written in Write-back in cycle *n* is visible to a Decode read in the second half of cycle *n*, eliminating one potential source of stalls.

The shading convention in pipeline diagrams reflects this:

- Shading on the *right* half of an RF cell means the register file is **read** in that cycle.
- Shading on the *left* half means the register file is **written** in that cycle.
- No shading means the register file is idle.

11.3.4 PC Increment and Optimization

Each time an instruction is fetched, **PCPlus4F** is written simultaneously to the PC register and to the IF/ID pipeline register. On the next cycle, the ID stage sees this value as **PCPlus8D** (it is two instructions ahead of the current Fetch, because the Fetch stage has already incremented once more).

Rather than carrying a separate adder and pipeline register to compute `PCPlus8D` from scratch, the optimized datapath reuses `PCPlus4F` directly: `PCPlus8D = PCPlus4F` at that point in time. This eliminates one adder and one pipeline register, reducing area and power.

11.4 Pipelined Control

11.4.1 Deriving Control Signals

The pipelined processor uses the *same* control unit as the single-cycle design. In the Decode stage, the control unit examines the `Op` field (bits [27:26]) and `Func` field (bits [25:20]) of `InstrD` to produce the full set of control signals: `RegWrite`, `MemWrite`, `MemToReg`, `ALUSrc`, `RegSrc`, `ImmSrc`, `ALUControl`[2:0], and `PCSrc`.

Table 11.1 summarises what each signal does and which stage ultimately uses it.

Table 11.1: **Control signal summary.** Each signal’s source stage and the stage(s) that consume it.

Signal	Consumed in	Purpose
<code>ALUControl</code> [2:0]	Execute (EX)	Selects ALU operation (ADD, SUB, AND, ...)
<code>ALUSrc</code>	Execute (EX)	Chooses register vs. immediate as ALU input B
<code>MemWrite</code>	Memory (MEM)	Enables data memory write (<code>str</code>)
<code>MemToReg</code>	Write-back (WB)	Selects load data vs. ALU result to write back
<code>RegWrite</code>	Write-back (WB)	Enables register file write
<code>PCSrc</code>	Write-back (WB)	Triggers a PC update (branch or <code>bx</code>)

The control unit also checks whether the destination register is `R15` (the PC), because a write to `R15` is treated as a branch and requires special hazard handling.

11.4.2 Pipelining the Control Signals

Just like data signals, control signals must be pipelined along with the instruction they belong to. A control signal generated in the Decode stage that governs Write-back behavior (`RegWrite`, `MemToReg`) must be carried through the ID/EX, EX/MEM, and MEM/WB pipeline registers so that it arrives at Write-back at the same time as the data it controls.

For example, `RegWriteW` feeds back from the Write-back stage to the register file’s write-enable port. If `RegWrite` were not pipelined, the write-enable for the `ldr` instruction would arrive before its data, potentially writing a garbage value.

Concrete example. Consider `ldr r3, [r0, #8]`. The control unit produces `RegWrite=1` and `MemToReg=1` in the Decode stage (cycle 2). The instruction does not reach Write-back until cycle 6. Without pipelining, `RegWrite=1`

would enable a register write in cycle 2, four cycles before the data is ready. By carrying **RegWrite** through ID/EX, EX/MEM, and MEM/WB, the write-enable arrives at the register file in cycle 6 together with the loaded data, so the write happens at the right time.

11.5 Data Hazards

11.5.1 The Read-After-Write Problem

A **data hazard** occurs when an instruction tries to read a register that has not yet been written back by a previous instruction. The most common form is **Read-After-Write (RAW)**: an instruction produces a result that a subsequent instruction needs before Write-back has completed.

Consider the following sequence:

```
add r1, r2, r3      ; writes R1 in WB (cycle 5)
and r4, r1, r5      ; reads R1 in ID (cycle 3) -- WRONG
                    VALUE
orr r6, r1, r7      ; reads R1 in ID (cycle 4) -- WRONG
                    VALUE
sub r8, r1, r9      ; reads R1 in ID (cycle 5, 2nd half
                    ) -- OK
```

The **add** writes **R1** at the end of cycle 5 (Write-back). **and** reads **R1** in cycle 3, which is two cycles too early. **orr** reads **R1** in cycle 4, one cycle too early. Only **sub**, which reads in the second half of cycle 5, gets the correct value thanks to the falling-edge write timing described earlier.

The timeline below makes this concrete. Say the original value of **R1** is 7 and **add** produces 15.

Instruction	C1	C2	C3	C4	C5	C6	C7
add r1,r2,r3	IF	ID	EX	MEM	WB (writes 15)		
and r4,r1,r5		IF	ID reads 7!	EX	MEM	WB	
orr r6,r1,r7			IF	ID reads 7!	EX	MEM	WB
sub r8,r1,r9				IF	ID reads 15 (OK)	EX	MEM

■ correct ■ stale value read (old R1 = 7, not yet updated)

Figure 11.3: **RAW hazard timeline (no forwarding)**. **and** and **orr** read the register file before **add** has written back, so they see the stale value 7 instead of the correct value 15.

Why forwarding fixes this. By cycle 4, when **and** is in the Execute stage and needs **R1**, the result 15 is already sitting in the **EX/MEM** pipeline register (labelled **ALUOutM**). The forwarding multiplexer can route **ALUOutM** directly to the ALU input, bypassing the register file entirely. By cycle 5, when **orr** is in the Execute stage, the same result has moved to the **MEM/WB** register

(**ResultW**), and forwarding is still possible from there. Neither instruction needs to stall.

A second type, **Write-After-Write (WAW)**, occurs when two instructions both write to the same destination register. In an in-order 5-stage pipeline this resolves itself automatically: whichever instruction is farther along the pipeline writes first, and the later instruction's Write-back always overwrites it in the correct order. However, WAW becomes a serious problem in out-of-order processors, where instructions can complete out of sequence.

A third type, **Write-After-Read (WAR)**, also called an *anti-dependence*, occurs when a later instruction writes a register that an earlier instruction still needs to read. In a simple in-order pipeline, the reader always reaches the Decode (read) stage before the writer reaches Write-back, so WAR hazards do not arise. They become relevant in out-of-order designs where the writer might overtake the reader.

Table 11.2: **The three pipeline hazard types.** Structural, data, and control hazards with their causes and whether this pipeline must address each one.

Type	Trigger	Example
RAW (true dependence)	Read before write completes	<code>add r1,r2,r3</code> then <code>and r4,r1,r5</code>
WAW (output dependence)	Two writes to same register	<code>add r1,r2,r3</code> then <code>sub r1,r4,r5</code>
WAR (anti-dependence)	Write before earlier read	<code>add r4,r1,r2</code> then <code>sub r1,r5,r6</code>

11.5.2 Result Forwarding

Rather than stalling the pipeline until Write-back, the hardware can **forward** (bypass) the result directly from the stage where it is computed to the stage that needs it.

The `add` instruction above computes its ALU result at the end of cycle 3 (Execute stage). At cycle 4, when `and` reaches the Execute stage and needs **R1**, the result of `add` is sitting in the EX/MEM pipeline register (**ALUOutM**). A multiplexer can select **ALUOutM** instead of the register-file output, forwarding the value one stage early.

By cycle 5, when `orr` is in the Execute stage, `add`'s result has reached the MEM/WB register (**ResultW**), and forwarding is still possible.

Forwarding eliminates most RAW hazards without any stalls. There is one important exception, covered in Section 11.7.

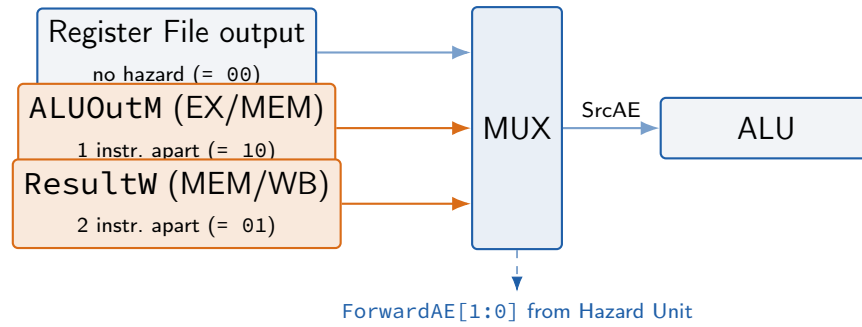


Figure 11.4: **Forwarding multiplexer for ALU input A (SrcAE).** ForwardAE selects the register-file value, the one-cycle-old EX/MEM result, or the two-cycle-old MEM/WB result; an identical mux handles input B via ForwardBE.

Forwarding Rule

Always forward from the *earliest* stage that holds the correct result. If both the Memory stage and the Write-back stage contain a matching destination register, the Memory stage has priority because its value is more recent. Fall back to the register file only when neither stage has a pending write to the source register.

11.6 The Hazard Unit

11.6.1 Architecture

The **Hazard Unit** is a small combinational block added to the pipelined datapath. It detects hazards and drives the forwarding-multiplexer select signals along with the stall and flush signals so that the pipeline always produces correct results.

Two multiplexers sit in front of the ALU, one per ALU input, each with a three-way select:

- 00: take the value from the register file (no hazard).
- 01: forward ResultW from the Write-back stage.
- 10: forward ALUOutM from the Memory stage.

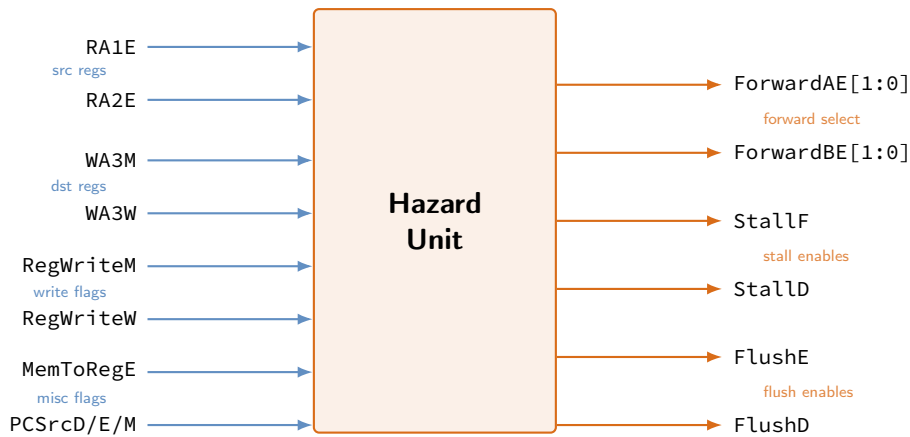


Figure 11.5: **Hazard Unit interface.** Inputs are in-flight register addresses and control flags; outputs drive the forwarding-select, stall, and flush signals.

11.6.2 Match Signals

The Hazard Unit computes four **match signals** by comparing the source register addresses in the Execute stage (RA1E, RA2E) against the destination register addresses in the Memory and Write-back stages (WA3M, WA3W):

$$\begin{aligned}\text{Match_1E_M} &= (\text{RA1E} == \text{WA3M}) \\ \text{Match_1E_W} &= (\text{RA1E} == \text{WA3W}) \\ \text{Match_2E_M} &= (\text{RA2E} == \text{WA3M}) \\ \text{Match_2E_W} &= (\text{RA2E} == \text{WA3W})\end{aligned}$$

To understand what these signals mean, recall where instructions are in the pipeline when the Hazard Unit checks them:

- The **current instruction** is in the Execute stage: it reads RA1E and RA2E as its source registers.
- The instruction **one step ahead** is in the Memory stage: it will write to WA3M.
- The instruction **two steps ahead** is in the Write-back stage: it will write to WA3W.

When a match signal is asserted, the instruction currently in Execute needs a value that an older instruction has computed but not yet written to the register file. The Hazard Unit then activates the appropriate forwarding path so the result is taken from a pipeline register rather than the register file.

Match_2E_M is the tightest RAW dependency the pipeline can still resolve without stalling, the one-instruction-apart case:

```
add r5, r4, r2    ; WA3M = r5
add r6, r4, r5    ; RA2E = r5  -- forwarded from
    Memory stage
```

11.6.3 Forwarding Logic

If both Memory and Write-back stages contain a matching destination register, Memory has priority because it holds the *more recently executed* instruction. Forwarding from the wrong stage (Write-back when Memory has the fresher value) would produce incorrect results. The forwarding logic for ALU input A is:

```

if (Match_1E_M && RegWriteM)
    ForwardAE = 10;    // SrcAE = ALUOutM  (1 instr
                        apart)
else if (Match_1E_W && RegWriteW)
    ForwardAE = 01;    // SrcAE = ResultW  (2 instr
                        apart)
else
    ForwardAE = 00;    // SrcAE from register file (no
                        hazard)

```

The `&& RegWriteM` and `&& RegWriteW` guards are important: just because two instructions share a register address does not mean a hazard exists. If the instruction in the Memory stage has `RegWrite=0` (e.g., a `str` or a `cmp`), it is not writing any result, so there is nothing to forward. Without these guards, the hardware would forward garbage values to the ALU.

The logic for `ForwardBE` is identical except it checks `Match_2E_M` and `Match_2E_W`.

Forwarding the store data. A subtlety often missed: `ForwardBE` serves double duty for a `str` instruction. In an ARM `str rT, [rBase, #off]`, register `rT` is the *data to be stored*, not an ALU operand. It enters the pipeline as the register-file read on port B (`RD2E`) and travels through the datapath alongside the computed address. If a preceding instruction wrote `rT` and has not yet reached Write-back, `ForwardBE` selects the forwarded value as `WriteDataE`, ensuring the correct data reaches data memory in the MEM stage. Without this path, a `str` immediately following an arithmetic instruction that produces its store data would write a stale value to memory.

Example trace. Consider the sequence:

```

add r5, r4, r2    ; Instruction A
sub r6, r5, r1    ; Instruction B (needs r5)
and r7, r5, r3    ; Instruction C (also needs r5)

```

When instruction B is in Execute (cycle 4), instruction A is in Memory. `Match_1E_M` is asserted (`RA1E = r5 = WA3M`) and `RegWriteM = 1`, so `ForwardAE = 10` and instruction B's ALU reads `ALUOutM` from the EX/MEM register. When instruction C is in Execute (cycle 5), instruction A is in Write-back. `Match_1E_W` is asserted and `RegWriteW = 1`, so `ForwardAE = 01` and instruction C's ALU reads `ResultW`. Neither B nor C stalls; both hazards are resolved by forwarding.

11.7 Load-Use Hazard and Pipeline Stalls

11.7.1 Why `ldr` Cannot Be Forwarded

For arithmetic instructions the ALU result is available at the end of the Execute stage and can be forwarded to the *next* instruction's Execute stage one cycle later. A `ldr` instruction, however, does not produce its result until the end of the Memory stage, one cycle later than the ALU result. A `ldr` followed immediately by a dependent instruction therefore creates a hazard that forwarding alone cannot fix.

```
ldr r2, [r0, #40]    ; result available end of cycle 4
                    (MEM)
orr r3, r2, r1       ; needs r2 at start of cycle 4 (
                    EX) -- impossible
```

The cycle count below shows why:

Instruction	C1	C2	C3	C4	C5	C6
<code>ldr r2,[r0,#40]</code>	IF	ID	EX	MEM <i>result ready here</i>	WB	
<code>orr r3,r2,r1</code>		IF	ID	EX <i>needs r2 here!</i>	MEM	WB

■ normal stage ■ conflict: value needed before it is ready

Figure 11.6: **Load-use timing conflict.** The `ldr` result is ready at the end of MEM (cycle 4), but `orr` needs it at the start of EX in the same cycle, so forwarding alone cannot help.

The root problem is that the `ldr` result comes from data memory, which is accessed in the MEM stage. An arithmetic instruction's result comes from the ALU in the EX stage, one cycle earlier. This one-cycle difference is exactly what makes the load-use hazard impossible to resolve with forwarding alone.

The `ldr` has a **two-cycle latency**: a dependent instruction cannot use its result until two cycles after the load. One pipeline stall cycle must be inserted.

Load-Use Hazard Rule

A `ldr` instruction followed immediately by an instruction that uses the loaded register cannot be resolved by forwarding alone. The hardware inserts exactly one stall cycle (a bubble). The easiest software fix is to reorder code so that an independent instruction fills the slot between the load and its first consumer. This is called *load-delay slot scheduling*.

11.7.2 Stalls and Bubbles

A **stall** freezes the Fetch and Decode pipeline registers (their contents hold unchanged on the next clock edge) while simultaneously **flushing** the Execute

pipeline register by clearing all its control signals to zero. The flushed Execute stage propagates through the pipeline as a **bubble**, a NOP-like slot that performs no action and modifies no architectural state.

Instruction	C1	C2	C3	C4	C5	C6	C7
ldr r2,[r0,#40]	IF	ID	EX	MEM	WB		
orr r3,r2,r1		IF	ID	stall	EX	MEM	WB
sub r4,r5,r6			IF	stall	ID	EX	MEM

Figure 11.7: **Load-use hazard with one stall cycle.** `orr` and `sub` are each delayed one cycle; a bubble propagates through Execute while Fetch and Decode are frozen.

■ active ■ stall/bubble

Stalling one stage always requires stalling *all earlier stages* too, so that no instruction already in the pipeline is lost. The pipeline register immediately after the stalled stage is cleared (flushed) to prevent the current instruction from being executed a second time.

Why All Earlier Stages Must Stall Together

When the pipeline stalls to wait for a result, it is not enough to freeze just the one waiting stage. If only the Decode stage were frozen while Fetch kept running, the instruction in the Fetch stage would overwrite the stalled instruction in the IF/ID pipeline register on the next clock edge, and that instruction would be lost. A stall works like a traffic jam: when one car stops, every car behind it must stop too. Every stage *upstream* of the stall point (including Fetch) holds its current instruction until the hazard is resolved.

11.7.3 Hazard Unit: LDR Stall Logic

The Hazard Unit detects the load-use hazard by inspecting the instruction in the Execute stage. If it is a `ldr` (`MemToRegE` is asserted) and its destination register (`WA3E`) matches either source of the instruction currently in the Decode stage, a stall is required:

```
Match_12D_E = (RA1D == WA3E) || (RA2D == WA3E);
LDRstall    = Match_12D_E && MemToRegE;

StallF      = LDRstall;
StallD      = LDRstall;
FlushE      = LDRstall;
```

The `EN` (enable) inputs of the Fetch and Decode pipeline registers are de-asserted when `StallF` and `StallD` are set, freezing those stages. The Execute pipeline register has a synchronous `CLR` input that zeroes all control signals when `FlushE` is set.

11.7.4 The Compiler's Role: Avoiding Hazards in Software

The hardware must handle every hazard correctly, but the *compiler* (or a careful assembly programmer) can often eliminate hazards entirely by reordering instructions. This is called **instruction scheduling** or **load-delay slot scheduling**.

Before scheduling (1 stall cycle):

```
ldr  r2, [r0, #40]    @ load r2
orr  r3, r2, r1        @ uses r2 immediately -- STALL
    inserted here
add  r4, r3, r5
```

After scheduling (0 stall cycles):

```
ldr  r2, [r0, #40]    @ load r2
add  r4, r3, r5        @ independent -- moved up to
    fill the slot
orr  r3, r2, r1        @ uses r2 (now ready, no stall
    needed)
```

By moving the independent **add** into the slot between the load and its first consumer, the compiler eliminates the stall without changing the program's result. This technique is called **filling the load-delay slot**.

When scheduling is not possible. Sometimes there is no independent instruction to fill the slot. For example, at the start of a function the first thing is often loading several arguments from memory; there may be nothing to interleave. In these cases the hardware stall is unavoidable. Modern compilers try very hard to schedule loads early in a basic block so that the result is ready by the time it is needed.

Hardware vs. Software Hazard Handling

The Hazard Unit always guarantees correctness. The programmer never needs to worry about getting wrong results from a pipeline hazard. What the hardware cannot avoid entirely is the time cost of stalls. A well-written compiler (or careful assembly programmer) reduces stalls by scheduling independent instructions between a load and its first use, keeping the pipeline as busy as possible.

11.8 Control Hazards

11.8.1 The Branch Problem

A **control hazard** arises because the pipelined processor does not know which instruction to fetch next when a branch enters the pipeline. The branch de-

cision (whether to take the branch and, if so, what the target address is) is not resolved until the Execute stage, which is cycle 3 for a branch fetched in cycle 1. By then, two instructions have already been fetched behind the branch.

The same problem applies to any instruction that modifies the PC:

```

bx lr          ; PC update known in Writeback (cycle
5)
ldr pc, [sp]   ; PC update known in Writeback (cycle
5)

```

These cases require the pipeline to be stalled until the new PC is known, which can cost up to four cycles per occurrence.

An everyday analogy. Picture driving down a highway toward a fork. A sign tells you which lane to take, but it only becomes legible 50 metres before the split, yet the lane you are in was chosen 100 metres back. Branch prediction works the same way. The processor commits to fetching two instructions before knowing which path is correct. If the guess is wrong, those two fetched instructions must be discarded (flushed) and the correct instructions fetched instead. The goal is simply to be right as often as possible.

11.8.2 Predict-Not-Taken

Rather than stalling every time a branch is encountered, the processor employs a simple **branch prediction** strategy: *predict that branches are not taken* and continue fetching the sequential instructions.

If the prediction is correct (branch not taken), no penalty is incurred. If the branch *is* taken, the two instructions fetched behind the branch are wrong and must be **flushed** from the pipeline by clearing the Decode and Execute pipeline registers (**FlushD** and **FlushE**). These discarded instruction cycles are called the **branch misprediction penalty**.

Instruction	C1	C2	C3	C4	C5	C6	C7
b target (<i>taken</i>)	IF	ID	EX	MEM	WB		
add r6, ... (<i>wrong path</i>)		flush	flush				
sub r7, ... (<i>wrong path</i>)			flush				
target: mov ... (<i>correct</i>)				IF	ID	EX	MEM

■ active stage ■ flushed (wrong-path instruction discarded)

Figure 11.8: **Branch misprediction penalty (predict-not-taken)**. Two wrongly-fetched instructions are flushed when the taken branch is resolved in EX (end of cycle 3), giving a 2-cycle penalty.

Predict-Not-Taken Strategy

Assume every branch is *not taken* and continue fetching instructions sequentially. If the branch turns out to be not taken, no cycles are lost. If the branch *is* taken, flush the two wrong-path instructions from the Fetch and Decode stages and redirect the PC to the branch target. With Execute-stage resolution this costs exactly **2 cycles** per misprediction.

11.8.3 Resolving Branches in the Execute Stage

Adding a multiplexer before the PC register allows the branch destination address to be selected from `ALUResultE` in the Execute stage (cycle 3), rather than waiting until Write-back (cycle 5). The control signal `BranchTakenE` is asserted when the branch condition is satisfied.

When `BranchTakenE` is asserted:

- The PC is loaded with the branch target from `ALUResultE`.
- The two instructions in the Fetch and Decode stages are flushed (`FlushD = FlushE = 1`).

This cuts the misprediction penalty from four cycles to **two cycles**.

11.8.4 Handling PC Writes from Write-Back

Instructions that write to R15 (`bx lr, pop {pc}`) produce the new PC value only in Write-back. While such an instruction works its way through the pipeline, the Fetch stage must be **stalled** to avoid fetching from the wrong address. `PCWrPendingF` signals that a PC-write is in flight somewhere in the pipeline:

```
PCWrPendingF = PCSrcD || PCSrcE || PCSrcM;
```

When `PCSrcW` is finally asserted in Write-back, `StallF` is released and the correct instruction is fetched, but `FlushD` is held for one extra cycle to discard whatever was in the Fetch stage during the stall.

11.8.5 Complete Hazard Unit Control Logic

Combining data and control hazard handling, the full set of stall and flush signals is:

```
PCWrPendingF = PCSrcD || PCSrcE || PCSrcM;
```

```
StallD = LDRstall;
```

```
StallF = LDRstall || PCWrPendingF;
```

```
FlushE = LDRstall || BranchTakenE;
```

```
FlushD = PCWrPendingF || PCSrcW || BranchTakenE;
```

StallD: Hold the Decode stage (stop **InstrD** from advancing) during a load-use hazard.

StallF: Hold the Fetch stage during a load-use hazard or while a PC-write is pending in the pipeline.

FlushE: Clear the Execute stage to insert a bubble after a stall or a correctly-predicted-not-taken branch that was in fact taken.

FlushD: Clear the Decode stage when a PC write is completing or a taken branch is detected.

11.8.6 Reducing the Branch Penalty Further

With additional logic, the branch decision can be moved from the Execute stage to the **Decode stage**, reducing the misprediction penalty to just **one cycle**. The branch target is computed as:

$$\text{PCBranchD} = \text{PCPlus8D} + \text{ExtImmD}$$

and **BranchTakenD** is evaluated using the ALU flags produced by the *previous* instruction (**ALUFlagsE**).

The trade-off is a potential increase in the critical path if the flag signals arrive late from the preceding Execute stage. Whether this optimization improves overall performance depends on the branch frequency and the clock timing budget.

11.8.7 Beyond Predict-Not-Taken: Smarter Branch Predictors

Predict-not-taken is simple but limited. It fails every time a branch is actually taken, which is about 55–65% of the time in typical loop-heavy code. Real processors use more sophisticated predictors that reach 95%+ accuracy.

2-bit saturating counter. Rather than always predicting not-taken, the processor keeps a small table of **2-bit counters**, one per branch (indexed by the branch's PC address). Each counter has four states:

Table 11.3: **2-bit saturating counter states.** Two consecutive surprises are required before the prediction flips, making the predictor robust to isolated outliers.

State	Prediction	Transition
11 (Strongly Taken)	Taken	Taken → 11, Not-taken → 10
10 (Weakly Taken)	Taken	Taken → 11, Not-taken → 01
01 (Weakly Not-Taken)	Not taken	Taken → 10, Not-taken → 00
00 (Strongly Not-Taken)	Not taken	Taken → 01, Not-taken → 00

One misprediction does not flip the prediction. A loop branch that is taken 99 times and falls through on the last iteration will be mispredicted just once. The counter needs two consecutive surprises before it changes its prediction, which is why loop branches are handled well.

Branch history table (BHT). A **branch history table** stores one 2-bit counter per branch, indexed by the low bits of the branch's PC address. A 1 KB BHT with 2-bit entries holds 4,096 predictions, which covers most branches in a typical program.

Global history. The most accurate predictors correlate the outcome of the current branch with the outcomes of recent previous branches using a global history register. This lets the predictor notice patterns such as: whenever branch A was taken and branch B was not, branch C is almost always taken. Modern ARM Cortex-A processors use variants of this idea and reach misprediction rates below 5% on many workloads.

Why Branch Prediction Matters More with Deeper Pipelines

The deeper the pipeline, the more instructions are fetched on the wrong path before the misprediction is detected, so the penalty grows. This 5-stage pipeline has a 2-cycle penalty. A modern out-of-order processor with a 14-stage pipeline can pay 14–20 cycles per misprediction. At that scale, going from 90% to 95% prediction accuracy roughly halves the branch penalty overhead. Branch predictors are one of the most actively researched areas in processor design for exactly this reason.

11.9 Exceptions in a Pipelined Processor

11.9.1 The Problem: Multiple In-Flight Instructions

An **exception** (or **interrupt**) is an event that disrupts the normal flow of execution: a memory fault, an undefined instruction, a divide-by-zero, or an external hardware signal. In a single-cycle processor, handling one is straightforward: finish the current instruction, then jump to the exception handler.

In a pipelined processor, however, up to five instructions are executing simultaneously. When an exception occurs, the processor must answer several difficult questions:

- Which instruction caused the exception?
- What should happen to the instructions that were already fetched and partially executed *after* the faulting instruction?
- What state was the processor in when the exception occurred, and can execution be resumed correctly after the handler returns?

11.9.2 Precise vs. Imprecise Exceptions

The gold standard for exception handling is a **precise exception**: the processor saves its state as if every instruction *before* the faulting instruction had fully completed, and *no* instruction after it had executed at all. This gives the illusion that the pipeline does not exist: from the software’s point of view, execution stopped cleanly at the faulting instruction.

Imprecise exceptions (used in some older or simpler designs) allow some instructions after the fault to complete, and some before it to remain incomplete. Imprecise exceptions are much harder to reason about and make it nearly impossible to resume execution after an exception.

Precise Exceptions

A precise exception guarantees that at the moment the handler is invoked, the processor state reflects exactly all instructions before the faulting instruction (committed) and none of the instructions after it (squashed). This is the model used by all modern ARM processors.

11.9.3 Implementing Precise Exceptions in a 5-Stage Pipeline

When an exception is detected in stage S , the processor must:

1. **Flush** all instructions in stages *after* S (they must not modify architectural state).
2. **Complete** all instructions in stages *before* S normally (they were fetched before the faulting instruction and must commit).
3. **Save the PC** of the faulting instruction (so the handler knows where to resume or report the fault).
4. **Jump to the exception vector**, a fixed address in memory where the operating system’s exception handler begins.

In the ARM architecture, the processor saves the current PC into the **Link Register** (r14) and the processor status flags into the **Saved Program Status Register** (SPSR), then switches to a privileged mode and begins executing the handler.

Example: undefined instruction. Suppose the Decode stage encounters a 32-bit pattern it does not recognise as a valid ARM instruction.

- The instruction in Fetch (behind the bad instruction) is flushed immediately.
- The bad instruction itself is flushed from Decode after the exception address is saved.
- Any instruction in Execute, Memory, or Write-back that was fetched *before* the bad instruction completes normally.
- The PC of the bad instruction is written to the link register, and control transfers to the undefined-instruction exception vector.

Exceptions Are Not the Same as Pipeline Flushes for Branches

A branch misprediction flush (Section 11.8) is a *performance* correction: the processor speculatively fetched the wrong instructions and discards them before they do any harm. An exception flush is a *correctness* event: something truly wrong happened, and the operating system must be notified. The mechanism (clearing pipeline registers) looks similar in hardware, but the purpose and software handling are entirely different.

11.10 Structural Hazards

A **structural hazard** occurs when two instructions simultaneously require the *same hardware resource* and the hardware cannot satisfy both requests in the same cycle. Unlike data hazards (wrong value) or control hazards (wrong instruction), structural hazards arise purely from resource contention.

11.10.1 The Memory Conflict

The most common structural hazard in a pipeline is a **memory access conflict**: the Fetch stage reads an instruction from memory in the same cycle that the Memory stage reads or writes data.

Table 11.4: **Structural hazards in the 5-stage ARM pipeline.** Each hazard and the hardware solution that eliminates it.

Potential conflict	Stage A	Stage B	Resolved by
Memory access	Fetch reads <i>instruction</i> memory (IF)	Memory stage reads/writes <i>data</i> memory (MEM)	Harvard architecture: separate instruction and data memories
Register file R/W	Decode reads register file (ID)	Write-back writes register file (WB)	Falling-edge write, second-half read (Section 11.3)

11.10.2 Harvard Architecture Eliminates the Memory Hazard

The introduction says the pipeline has *separate instruction and data memories*. This is the **Harvard architecture**: instruction fetch and data memory access use independent memory banks (in real processors, separate L1 instruction and data caches). Because the two accesses go to entirely different hardware, there is no contention, and no structural hazard arises.

Why Harvard Architecture Matters

A single unified memory would create a structural hazard every time a load or store instruction is in the Memory stage at the same cycle that any other instruction is being fetched. Separate instruction and data memories (or caches) allow both accesses to proceed in parallel, eliminating this hazard without any stalls.

11.10.3 Register File Read/Write in the Same Cycle

Write-back and Decode both access the register file in the same cycle once the pipeline is full. This would be a structural hazard if both operations required the port simultaneously in the same clock phase. Writing on the **falling clock edge** and reading in the **second half** of the cycle means the write settles before the read samples the value, so both can proceed without conflict. This is described in detail in Section 11.3.

11.10.4 Summary

In the 5-stage ARM pipeline as designed in this chapter, *no structural hazard requires a stall*. Both potential conflicts are eliminated by the choice of Harvard architecture and the dual-phase register file. This is why the chapter’s hazard discussion focuses entirely on data hazards and control hazards.

11.11 Real-World ARM Pipelines

The 5-stage pipeline studied in this chapter is a clean, pedagogical design that captures the essential ideas. Real ARM processors extend this foundation in several important ways.

From 5 Stages to Modern ARM Pipelines

Processor	Stages	Key additions over 5-stage
ARM7TDMI (1994)	3	No forwarding; classic RISC simplicity
ARM9TDMI (1997)	5	Closest to this chapter’s design
ARM Cortex-A5 (2009)	8	Superscalar 2-wide, branch predictor
ARM Cortex-A7 (2011)	8	Dual-issue in-order, improved predictor
ARM Cortex-A53 (2012)	8	Advanced predictor, 2-wide decode
ARM Cortex-A76 (2018)	13	4-wide out-of-order, deep speculation
ARM Cortex-X4 (2023)	15+	6-wide decode, large instruction window

Each additional pipeline stage allows a *shorter clock period* (higher frequency) by splitting the slowest stage into two. The Cortex-A7’s 8-stage pipeline runs at up to 1.6 GHz, while the ARM9’s 5-stage design peaked around 300 MHz. More stages allow a shorter clock period, but a deeper pipeline means a larger branch misprediction penalty, typically 8–15 cycles instead of 2. This is exactly why sophisticated branch predictors (Section 11.8) are so important in modern processors.

What “superscalar” means. A **superscalar** processor issues more than one instruction per cycle by duplicating pipeline resources (two ALUs, two decode units, etc.). A 2-wide superscalar has an ideal CPI of 0.5, meaning two instructions complete every cycle. Most modern ARM Cortex-A processors are 2-wide to 6-wide superscalar, which is why their throughput far exceeds the 5× speedup that a simple 5-stage pipeline can offer.

Out-of-order execution. In the pipeline covered in this chapter, instructions always execute in program order. An **out-of-order** processor can execute instruction 5 before instruction 4 if instruction 4 is waiting for a result but instruction 5 already has all its inputs ready. This hides latency and keeps execution units busy, at the cost of significantly more complex hardware for tracking dependencies and maintaining precise exceptions.

11.12 Complete Pipeline Trace: Putting It All Together

The best way to consolidate everything from this chapter is to trace a short program completely, cycle by cycle, and identify every forwarding path, every stall, and every flush. This is the style of problem most commonly seen on exams.

11.12.1 The Program

```

; Assume r0 = base address, r9 = 5
ldr  r1, [r0, #0]    @ I1: load
add  r2, r1, r9      @ I2: uses r1  (load-use hazard
    with I1)
sub  r3, r2, r9      @ I3: uses r2  (RAW, forwardable
    from I2)
cmp  r3, #0          @ I4: uses r3  (RAW, forwardable
    from I3)
bne  skip            @ I5: branch (taken)
add  r4, r3, r2      @ I6: wrong path -- will be
    flushed
sub  r5, r1, r9      @ I7: wrong path -- will be
    flushed
skip:
mov  r6, #1          @ I8: correct branch target

```

11.12.2 Cycle-by-Cycle Table

Instr.	C1	C2	C3	C4	C5	C6	C7	C8	C9	C10	C11	C12
I1 <code>ldr r1</code>	IF	ID	EX	MEM	WB							
I2 <code>add r2,r1</code>		IF	ID	stall	EX fwd	MEM	WB					
I3 <code>sub r3,r2</code>			IF	stall	ID	EX fwd	MEM	WB				
I4 <code>cmp r3</code>				stall	IF	ID	EX fwd	MEM	WB			
I5 <code>bne skip</code>						IF	ID	EX	MEM	WB		
I6 <code>add r4 (wrong)</code>							flush	flush				
I7 <code>sub r5 (wrong)</code>								flush				
I8 <code>mov r6,#1</code>									IF	ID	EX	MEM

■ active stage
 ■ stall / bubble
 ■ flushed (wrong-path)
 fwd = forwarded result

Figure 11.9: **Complete pipeline trace (eight instructions).** One load-use stall, three forwarded results, and a 2-cycle branch flush across twelve clock cycles.

11.12.3 Pipeline Snapshot at Cycle 6

A **pipeline snapshot** shows the state of every stage at one instant in time. Reading the table column by column gives the snapshot; reading row by row gives the instruction's journey. At cycle 6 (highlighted column in Table 11.9), the pipeline contains:

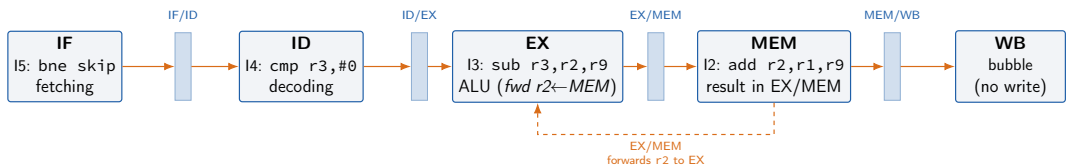


Figure 11.10: **Pipeline snapshot at cycle 6.** Four instructions occupy four different stages simultaneously: I5 fetching in IF, I4 decoding in ID, I3 executing in EX (receiving I2's result via EX/MEM forwarding), I2 in MEM, and the load-use bubble in WB.

11.12.4 Annotated Walkthrough

Cycle 3 (I1 in EX, I2 in ID): The Hazard Unit sees `MemToRegE = 1` (I1 is a load) and `RA1D = r1 = WA3E`. `LDRstall` is asserted: I2 and I3 are frozen; a bubble is inserted into EX. This stall also pushes I3 and I4 back by one cycle each.

Cycle 5 (I1 in WB, I2 in EX): I1's load result is now in the MEM/WB register (`ResultW`). `Match_1E_W` is asserted; `ForwardAE = 01`. I2's ALU reads `ResultW` directly; the load value arrives just in time.

Cycle 6 (I2 in MEM, I3 in EX): I3 needs r2 (written by I2). I2's result is in EX/MEM; `Match_1E_M` is asserted; `ForwardAE = 10`.

Cycle 7 (I3 in MEM, I4 in EX): I4 (`cmp r3, #0`) needs `r3` from I3. I3's result is in the EX/MEM register; `ForwardAE = 10`. Meanwhile I5 is in ID being decoded, and I6 is being speculatively fetched.

Cycle 8 (I4 in MEM, I5 in EX): I5 (`bne skip`) evaluates its branch condition in the Execute stage. `BranchTakenE` is asserted at the end of this cycle. `FlushD = FlushE = 1`: I6 (in Decode) and I7 (in Fetch) are erased. The PC is redirected to the address of `skip`.

Cycle 9 onwards: I8 begins fetching from the correct branch target. No further hazards.

Counting cycles and CPI.

- Useful instructions committed: I1, I2, I3, I4, I5, I8 = **6 instructions**. I6 and I7 are flushed and never modify architectural state.
- I8 enters IF in cycle 10 and completes Write-back in cycle 14, giving a total execution time of **12 cycles** from first fetch to when I8 reaches MEM (the last column in the trace above). Using the full execution window from C1 to I8's WB:

$$\text{CPI}_{\text{trace}} = \frac{14}{6} \approx 2.3$$

- For a long program the pipeline-fill overhead is negligible and the standard formula applies:

$$\text{CPI}_{\text{actual}} = 1 + \frac{\text{wasted cycles}}{\text{instructions}} = 1 + \frac{1 + 2}{6} = 1.5$$

where the 3 wasted cycles are 1 load-use stall and 2 branch-flush cycles.

- This dense cluster of hazards is a worst case; typical programs spread hazards out and achieve $\text{CPI} \approx 1.2\text{--}1.5$.

11.13 Performance Analysis

11.13.1 Ideal CPI and Throughput

In the absence of hazards, a pipelined processor achieves $\text{CPI} = 1$: one instruction completes every clock cycle. The clock period is set by the longest single pipeline stage (not the longest instruction), so the period is much shorter than in the single-cycle design.

If the single-cycle clock period is T_{single} and the pipeline has $N = 5$ stages, the ideal pipeline period is approximately $T_{\text{pipe}} \approx T_{\text{single}}/N$. The ideal speedup over the single-cycle design is therefore $N = 5\times$.

11.13.2 Pipeline Efficiency

Pipeline efficiency measures how close the actual pipeline performance is to the ideal:

$$\text{Efficiency} = \frac{\text{Ideal CPI}}{\text{Actual CPI}} = \frac{1}{\text{CPI}_{\text{actual}}}$$

Efficiency of 1.0 (100%) means no stall cycles were wasted. Efficiency of 0.78 means 22% of all cycles were spent stalling.

Using the worked example from Section 11.13.4 (CPI = 1.28):

$$\text{Efficiency} = \frac{1}{1.28} \approx 0.78 = 78\%$$

This is a reasonable figure for a simple in-order pipeline. Modern out-of-order processors achieve effective efficiencies above 90% on many workloads by hiding latency through dynamic scheduling and deep branch prediction.

Alternatively, pipeline efficiency can be expressed as:

$$\text{Efficiency} = \frac{\text{Busy stages}}{\text{Total stage-cycles}} = \frac{N \times I}{N \times (I + \text{Stall cycles})}$$

where I is the number of instructions and N is the pipeline depth. As I grows large (long programs), the startup transient (first N cycles) becomes negligible and efficiency approaches $1/\text{CPI}_{\text{actual}}$.

11.13.3 Stall Penalties

Stalls degrade throughput. Each stall cycle adds 1 to the effective CPI:

$$\text{CPI}_{\text{actual}} = 1 + \text{stalls per instruction}$$

Typical stall sources and their approximate frequencies in real programs:

- **Load-use hazards (LDR stall):** approximately 5–10% of instructions are loads followed immediately by a dependent instruction, contributing ≈ 0.05 – 0.10 stall cycles per instruction.
- **Branch mispredictions (2-cycle penalty):** branches occur in roughly 15–20% of dynamic instructions; with a predict-not-taken strategy and a 50–60% taken rate, the misprediction penalty contributes ≈ 0.15 – 0.25 stall cycles per instruction.

Even with these penalties, a pipelined processor typically achieves an effective CPI of 1.2–1.5, which is far superior to the 4–5 CPI of the multi-cycle design.

11.13.4 Worked Example: Computing Effective CPI

Let us walk through a concrete calculation using realistic program statistics.

Given a program with the following profile:

- Total dynamic instructions: 100,000
- LDR instructions: 10% of all instructions (10,000 loads)
- Load-use hazards: 60% of loads are followed immediately by a dependent instruction ($0.10 \times 0.60 = 0.06$ stall cycles per instruction on average)
- Branch instructions: 20% of all instructions (20,000 branches)
- Taken branches: 55% of branches (misprediction rate = 55%, since we predict not-taken)
- Branch misprediction penalty: 2 cycles per misprediction

Step 1: Calculate stall cycles per instruction.

$$\text{LDR stall contribution} = 0.10 \times 0.60 \times 1 \text{ cycle} = 0.060$$

$$\text{Branch stall contribution} = 0.20 \times 0.55 \times 2 \text{ cycles} = 0.220$$

$$\text{Total stalls per instruction} = 0.060 + 0.220 = 0.280$$

Step 2: Effective CPI.

$$\text{CPI}_{\text{actual}} = 1 + 0.280 = \mathbf{1.28}$$

Step 3: Speedup over single-cycle. If the single-cycle design has clock period T_s and $\text{CPI} = 1$, while the pipelined design has clock period $T_s/5$ and $\text{CPI} = 1.28$:

$$\text{Speedup} = \frac{1 \times T_s}{1.28 \times T_s/5} = \frac{5}{1.28} \approx \mathbf{3.9\times}$$

Despite hazard penalties consuming 0.28 out of an ideal CPI of 1, the shorter clock period still yields nearly a 4 $\times$ speedup over the single-cycle design. This illustrates why pipelining is so effective even in the presence of hazards.

Reducing Stalls Matters More at Scale

In this example, branch mispredictions (0.22 stalls/instr) cost more than load-use hazards (0.06 stalls/instr). This is a realistic ratio in programs with many conditional branches (like loops). Real processors invest heavily in branch predictors precisely because the branch penalty dominates. Even a 2% improvement in prediction accuracy translates directly into a measurable speedup at scale.

11.13.5 Comparison Summary

Table 11.5: **Performance comparison: single-cycle, multi-cycle, and pipelined.**

Feature	Single-Cycle	Multi-Cycle	Pipelined
Clock period	Long (T_s)	Short ($T_s/5$)	Short ($T_s/5$)
Ideal CPI	1	4–5	1
Hazard overhead	None	None	0.2–0.5 CPI
Hardware reuse	Low	High	Medium
Control complexity	Simple	FSM	Hazard Unit + FSM
Throughput (instructions/s)	$1/T_s$	$1/(4 \cdot T_s/5)$	$\sim 5/T_s$

► Observing Stalls and Forwarding

Enter the following program in PlayARM and step through it cycle by cycle.

```

ldr  r2, [r0, #40]
orr  r3, r2, r1      @ load-use hazard: stall
    inserted
add  r4, r2, r3
sub  r5, r4, r2
b    skip
add  r6, r3, r4      @ branch misprediction:
    flushed
add  r7, r4, r5      @ branch misprediction:
    flushed
skip:
mov  r8, #0

```

In the **Pipeline Activity** panel:

- After the `ldr`, watch for a *bubble* (grey slot) inserted in the Execute stage.
- After the `b skip`, watch for two instructions flushed from Fetch and Decode.
- For the `add` and `sub`, verify in the **Forwarding** panel that results are being forwarded rather than read from the register file.

11.14 Summary

- **Pipelining** overlaps the execution of multiple instructions by dividing the datapath into five stages (IF, ID, EX, MEM, WB) separated by pipeline registers. It improves *throughput*, not the latency of any individual instruction.
- All signals are suffixed by their pipeline stage (F, D, E, M, W) and must advance through the pipeline in unison with their instruction. Mismatched suffixes in an expression almost always indicate a bug.
- The **WA3** write-address must be pipelined through all stages to prevent incorrect register writes. Without this, Write-back would use the destination register of whatever instruction happens to be in Decode at that moment.
- The register file writes on the **falling clock edge** so that a Write-back write and a Decode read can coexist within the same cycle without a structural hazard.
- **Data hazards** (RAW: Read-After-Write) arise when an instruction reads a register that a preceding instruction has not yet written. The **Hazard Unit** resolves most RAW hazards by *forwarding* results from the EX/MEM or MEM/WB pipeline register directly to the ALU inputs.
- **Forwarding priority**: if both the Memory stage and Write-back stage have a matching destination register, always forward from Memory, since

it holds the more recent value.

- A **load-use hazard** (`ldr` followed immediately by a dependent instruction) cannot be resolved by forwarding because the loaded value is not available until the end of MEM, which is the same cycle the dependent instruction needs it in EX. One stall cycle (bubble) is inserted.
- **Control hazards** arise from branch instructions; a **predict-not-taken** strategy with Execute-stage branch resolution limits the misprediction penalty to **two cycles**. If prediction succeeds, zero cycles are wasted.
- PC-modifying instructions (`bx`, `pop {pc}`) stall the Fetch stage until the new PC is available in Write-back, using the `PCWrPendingF` signal.
- **Structural hazards** (resource conflicts) are eliminated by design: separate instruction and data memories remove the Fetch/MEM memory conflict; dual-phase register-file timing removes the Decode/Write-back conflict. No structural stalls are needed.
- The complete stall/flush logic is encoded in five signals: `StallF`, `StallD`, `FlushE`, `FlushD`, and `PCWrPendingF`.
- Realistic programs achieve an effective CPI of roughly 1.2–1.5, giving a speedup of approximately 3–4× over the single-cycle design after accounting for hazard penalties.

11.15 Exercises

Exercise 11.15.1 (Pipeline Diagram) Draw a cycle-by-cycle pipeline diagram for the following six-instruction sequence, marking any stall cycles and flushed instructions. Assume a predict-not-taken branch policy with Execute-stage resolution.

```
ldr r1, [r0, #0]
add r2, r1, r3
cmp r2, #10
bne loop
sub r4, r2, r1
mov r5, #0
```

Exercise 11.15.2 (Forwarding Paths) For each instruction pair below, state whether forwarding is sufficient (no stall needed), a stall is required, or there is no hazard at all. Justify your answer by identifying which match signal is asserted.

- `add r5, r2, r3` followed immediately by `sub r7, r5, r1`
- `ldr r5, [r0, #4]` followed immediately by `and r6, r5, r2`
- `mov r3, #42` followed two instructions later by `add r4, r3, r1`

Exercise 11.15.3 (Hazard Unit Logic) Write out the complete forwarding logic for `ForwardBE` (ALU input B) in pseudo-code, following the same structure as the `ForwardAE` logic shown in Section 11.6.

Exercise 11.15.4 (Branch Penalty Calculation) *A program has 10,000 instructions. 18% are branches; of those, 55% are taken. The processor uses predict-not-taken with a 2-cycle misprediction penalty. Compute:*

- (a) *The total number of wasted cycles due to branch mispredictions.*
- (b) *The effective CPI (assuming no other hazards).*
- (c) *The speedup compared with the same program on a stall-on-every-branch implementation (4-cycle penalty, always stalling).*

Exercise 11.15.5 (Design Trade-off) *Moving branch resolution from the Execute stage to the Decode stage reduces the misprediction penalty from two cycles to one cycle. Explain why this optimization might not improve performance in all cases, and describe the condition under which it could actually hurt performance.*

Exercise 11.15.6 (Signal Tracing) *Trace the instruction `ldr r4, [r1, #12]` through all five pipeline stages. For each stage, list:*

- (a) *which hardware unit is active (instruction memory, register file, ALU, data memory, or result MUX),*
- (b) *what value is computed or transferred, and*
- (c) *which signals are latched into the outgoing pipeline register.*

How does your trace change if the instruction immediately after is `add r5, r4, r2` (a load-use hazard)? Draw the modified pipeline diagram showing the inserted stall.

Exercise 11.15.7 (Forwarding Decision Table) *For each of the following three-instruction sequences, determine the value of `ForwardAE` and `ForwardBE` when the third instruction is in the Execute stage. Show which match signals are asserted and whether `RegWrite` is set.*

- (a) `add r1, r2, r3` $\rightarrow$ `sub r5, r6, r7` $\rightarrow$ `and r8, r1, r9`
- (b) `str r1, [r0]` $\rightarrow$ `mov r3, #5` $\rightarrow$ `orr r4, r1, r2`
- (c) `add r1, r2, r3` $\rightarrow$ `add r1, r4, r5` $\rightarrow$ `sub r6, r1, r7`

In case (c), which instruction's value does `ForwardAE` select, and why?

Exercise 11.15.8 (Hazard Identification Drill) *For each of the ten instruction pairs below, classify the relationship as one of: (N) No hazard, (F) RAW hazard resolved by forwarding (no stall), (S) RAW hazard requiring a stall, or (C) Control hazard. Briefly justify each answer.*

#	Instruction A	Instruction B (immediately after)	Type?
1	<code>add r1, r2, r3</code>	<code>sub r4, r1, r5</code>	
2	<code>ldr r1, [r0, #4]</code>	<code>add r2, r1, r3</code>	
3	<code>mov r1, #42</code>	<code>orr r5, r6, r7</code>	
4	<code>ldr r1, [r0, #4]</code>	<code>str r1, [r0, #8]</code>	
5	<code>add r3, r1, r2</code>	<code>cmp r3, #0</code>	
6	<code>cmp r3, #0</code>	<code>beq done</code>	
7	<code>str r4, [r0]</code>	<code>add r5, r4, r6</code>	
8	<code>ldr r2, [r1, #0]</code>	<code>ldr r3, [r2, #0]</code>	
9	<code>add r1, r2, r3</code>	<code>add r1, r4, r5</code>	
10	<code>mul r5, r3, r4</code>	<code>sub r6, r5, r1</code>	

Exercise 11.15.9 (CPI Optimisation by Instruction Scheduling) *The following code fragment has two load-use hazards.*

```

ldr  r1, [r0, #0]    @ load array[0]
add  r2, r1, r9      @ uses r1 (hazard 1)
ldr  r3, [r0, #4]    @ load array[1]
sub  r4, r3, r9      @ uses r3 (hazard 2)
add  r5, r2, r4      @ uses r2 and r4

```

- Draw the pipeline diagram for this code fragment and mark the two stall cycles. How many total cycles does it take?
- Reorder the instructions to eliminate both stalls (without changing the program's result). Verify that your reordered sequence has no hazards.
- Draw the pipeline diagram for your reordered sequence. How many cycles does it now take?
- Calculate the CPI for the original code and the reordered code. What is the speedup?

12

CPU Caches

13

Virtual Memory

Parallel Architectures

Conclusion

This concludes our introduction to computer architecture.